ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
БАШКИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Р.А. Башмаков
И.С. Галимов

ФУНКЦИОНАЛЬНЫЙ АНАЛИЗ

ЗАДАНИЯ
ДЛЯ СТУДЕНТОВ 3 КУРСА
МАТЕМАТИЧЕСКОГО ФАКУЛЬТЕТА

Уфа 2007

УДК 517.5

Печатается по решению кафедры теории функций и функционального анализа

Составители: Башмаков Р.А., Галимов И.С.

Задания по дисциплине „Функциональный анализ“ составлены для студентов 3 курса специальностей „Математика“ и „Прикладная математика“.

Пособие содержит контрольные вопросы и задачи, часть которых решается в аудитории на практических и лабораторных занятиях, а часть отводится для самостоятельной работы и домашних заданий.

© Башмаков Р.А., Галимов И.С. 2007

Формализация на Lean 4: Михаил Борисов, Claude Opus 4.6 Max (Anthropic)

Содержание

1 АЛГЕБРА МНОЖЕСТВ 3

2 МЕРА. ИЗМЕРИМЫЕ МНОЖЕСТВА 14

3 ИЗМЕРИМЫЕ ФУНКЦИИ. СХОДИМОСТИ ПО МЕРЕ И ПОЧТИ ВСЮДУ 26

4 ИНТЕГРАЛ ЛЕБЕГА 38

5 ФУНКЦИИ ОГРАНИЧЕННОЙ ВАРИАЦИИ И ИНТЕГРАЛ РИМАНА-СТИЛТЬЕСА 45

6 ТОПОЛОГИЧЕСКИЕ ПРОСТРАНСТВА 58

7 МЕТРИЧЕСКИЕ ПРОСТРАНСТВА 63

8 ПОЛНЫЕ МЕТРИЧЕСКИЕ ПРОСТРАНСТВА. ПОПОЛНЕНИЕ 69

9 СЕПАРАБЕЛЬНЫЕ МЕТРИЧЕСКИЕ ПРОСТРАНСТВА 77

10 ПРИНЦИП СЖИМАЮЩИХ ОТОБРАЖЕНИЙ 92

11 НОРМИРОВАННЫЕ ПРОСТРАНСТВА 101

12 ГИЛЬБЕРТОВЫ ПРОСТРАНСТВА 123

13 ЛИНЕЙНЫЕ НЕПРЕРЫВНЫЕ ФУНКЦИОНАЛЫ 143

14 ЛИНЕЙНЫЕ НЕПРЕРЫВНЫЕ ОПЕРАТОРЫ 145

15 ОБРАТНЫЕ ОПЕРАТОРЫ 150

16 СОПРЯЖЕННЫЕ ПРОСТРАНСТВА. РЕФЛЕКСИВНОСТЬ 159

17 ТЕОРЕМА ХАНА-БАНАХА. СИЛЬНАЯ И СЛАБАЯ СХОДИМОСТИ 160

18 СОПРЯЖЕННЫЕ ОПЕРАТОРЫ 185

19 РАВНОМЕРНАЯ, СИЛЬНАЯ, СЛАБАЯ СХОДИМОСТИ ПОСЛЕДОВАТЕЛЬНОСТЕЙ ОПЕРАТОРОВ 189

20 КОМПАКТНЫЕ МНОЖЕСТВА 202

21 КОМПАКТНЫЕ ОПЕРАТОРЫ 213

1. АЛГЕБРА МНОЖЕСТВ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение полукольца, кольца, σ-кольца, σ-алгебры.
2. Как определяется минимальное кольцо над множеством?
3. Как определяется полукольцо ячеек (брусьев)?

ЗАДАЧИ

1. Верно ли, что счетное
    a) объединение,
    b) пересечение

    открытых множеств в $\mathbb{R}^n$ есть множество открытое?
2. Верно ли, что счетное
    a) объединение,
    b) пересечение

    замкнутых множеств в $\mathbb{R}^n$ есть множество замкнутое?
3. Пусть X - трехэлементное множество. Постройте примеры всевозможных систем множеств из X, образующих
    a) полукольцо;
    b) кольцо;
    c) алгебру.
4. Привести примеры
    a) полукольца, не являющегося кольцом;
    b) кольца, не являющегося алгеброй;
    c) кольца, не являющегося σ-кольцом;
    d) σ-кольца, не являющегося алгеброй.

5. Покажите, что система множеств, замкнутая относительно операций объединения и пересечения не обязательно является кольцом.

Формализация на Lean 4

```
/-
  Section 1, Task 5.
  Show that a system of sets, closed under union and intersection,
  is not necessarily a ring (i.e., not necessarily closed under set difference).

  Counterexample: S = {∅, {1}, {1,2}, {1,2,3}} on the universe {1,2,3}.
  Closed under ∪ and ∩, but {1,2,3} \ {1,2} = {3} ∉ S.
-/
import Mathlib.Data.Finset.Basic
import Mathlib.Data.Finset.BooleanAlgebra
import Mathlib.Tactic

open Finset

/-- The system S = {∅, {1}, {1,2}, {1,2,3}} is closed under union. -/
theorem setSystem_closedUnder_union :
    let S : Finset (Finset (Fin 3)) :=
      {∅, {0}, {0, 1}, {0, 1, 2}}
    ∀ A ∈ S, ∀ B ∈ S, A ∪ B ∈ S := by
  decide

/-- The system S is closed under intersection. -/
theorem setSystem_closedUnder_inter :
    let S : Finset (Finset (Fin 3)) :=
      {∅, {0}, {0, 1}, {0, 1, 2}}
    ∀ A ∈ S, ∀ B ∈ S, A ∩ B ∈ S := by
  decide

/-- But S is NOT closed under set difference: {0,1,2} \ {0,1} = {2} ∉ S. -/
theorem setSystem_not_closedUnder_sdiff :
    let S : Finset (Finset (Fin 3)) :=
      {∅, {0}, {0, 1}, {0, 1, 2}}
    ¬ (∀ A ∈ S, ∀ B ∈ S, A \ B ∈ S) := by
  decide
```

6. Докажите, что прямое произведение полуколец является полукольцом.

Формализация на Lean 4

```
/-
  Section 1, Task 6.
  Prove that the direct product of semirings of sets is a semiring of sets.

  If S₁ on X₁ and S₂ on X₂ are semirings of sets (contain ∅, closed under ∩,
  and A \ B decomposes as a finite disjoint union), then
  Prod = {A ×ˢ B : A ∈ S₁, B ∈ S₂} is a semiring of sets on X₁ × X₂.
-/
import Mathlib.MeasureTheory.SetSemiring
import Mathlib.Tactic

open MeasureTheory Set Finset

/-- The direct product of two set semirings is a set semiring. -/
theorem prod_isSetSemiring
    {α β : Type*} {S₁ : Set (Set α)} {S₂ : Set (Set β)}
    (h₁ : IsSetSemiring S₁) (h₂ : IsSetSemiring S₂) :
    IsSetSemiring {C : Set (α × β) | ∃ A ∈ S₁, ∃ B ∈ S₂, C = A ×ˢ B} where
  empty_mem := ⟨∅, h₁.empty_mem, ∅, h₂.empty_mem, by simp⟩
  inter_mem := by
    rintro _ ⟨A₁, hA₁, A₂, hA₂, rfl⟩ _ ⟨B₁, hB₁, B₂, hB₂, rfl⟩
    exact ⟨A₁ ∩ B₁, h₁.inter_mem A₁ hA₁ B₁ hB₁, A₂ ∩ B₂, h₂.inter_mem A₂ hA₂ B₂ hB₂,
          Set.prod_inter_prod⟩
  diff_eq_sUnion' := by
    classical
    rintro _ ⟨A₁, hA₁, A₂, hA₂, rfl⟩ _ ⟨B₁, hB₁, B₂, hB₂, rfl⟩
    obtain ⟨I₂, hI₂C, hI₂d, hI₂eq⟩ := h₂.diff_eq_sUnion' A₂ hA₂ B₂ hB₂
    obtain ⟨I₁, hI₁C, hI₁d, hI₁eq⟩ := h₁.diff_eq_sUnion' A₁ hA₁ B₁ hB₁
    let J₂ : Finset (Set (α × β)) := I₂.image (A₁ ×ˢ ·)
    let J₁ : Finset (Set (α × β)) := I₁.image (· ×ˢ (A₂ ∩ B₂))
    refine ⟨J₂ ∪ J₁, ?subset, ?disj, ?eq⟩
    case subset =>
      intro C hC
      simp only [J₁, J₂, coe_union, Set.mem_union, Finset.mem_coe, Finset.mem_image] at hC
      rcases hC with ⟨b, hb, rfl⟩ | ⟨a, ha, rfl⟩
      · exact ⟨A₁, hA₁, b, hI₂C hb, rfl⟩
      · exact ⟨a, hI₁C ha, A₂ ∩ B₂, h₂.inter_mem A₂ hA₂ B₂ hB₂, rfl⟩
```

```
    case disj =>
      rw [Finset.coe_union, pairwiseDisjoint_union]
      refine ⟨?_, ?_, ?_⟩
      · -- J₂ pairwise disjoint
        intro x hx y hy hne
        simp only [J₂, Finset.mem_coe, Finset.mem_image] at hx hy
        obtain ⟨b₁, hb₁, rfl⟩ := hx
        obtain ⟨b₂, hb₂, rfl⟩ := hy
        show Disjoint (A₁ ×ˢ b₁) (A₁ ×ˢ b₂)
        exact (Set.disjoint_prod.mpr (Or.inr (hI₂d hb₁ hb₂ (fun h => hne (by rw [h])))))
      · -- J₁ pairwise disjoint
        intro x hx y hy hne
        simp only [J₁, Finset.mem_coe, Finset.mem_image] at hx hy
        obtain ⟨a₁, ha₁, rfl⟩ := hx
        obtain ⟨a₂, ha₂, rfl⟩ := hy
        show Disjoint (a₁ ×ˢ (A₂ ∩ B₂)) (a₂ ×ˢ (A₂ ∩ B₂))
        exact (Set.disjoint_prod.mpr (Or.inl (hI₁d ha₁ ha₂ (fun h => hne (by rw [h])))))
      · -- Cross disjointness
        intro x hx y hy _
        simp only [J₂, J₁, Finset.mem_coe, Finset.mem_image] at hx hy
        obtain ⟨b, hb, rfl⟩ := hx
        obtain ⟨a, ha, rfl⟩ := hy
        show Disjoint (A₁ ×ˢ b) (a ×ˢ (A₂ ∩ B₂))
        apply Set.disjoint_prod.mpr
        right
        have hb_sub : b ⊆ A₂ \ B₂ := hI₂eq ▸ Set.subset_sUnion_of_mem (show b ∈ (↑I₂ : Set _) from hb)
        exact disjoint_sdiff_left.mono_right Set.inter_subset_right |>.mono_left hb_sub
    case eq =>
      have decomp : A₁ ×ˢ A₂ \ B₁ ×ˢ B₂ = A₁ ×ˢ (A₂ \ B₂) ∪ (A₁ \ B₁) ×ˢ (A₂ ∩ B₂) := by
        ext ⟨x, y⟩
        simp only [Set.mem_diff, Set.mem_prod, Set.mem_union, Set.mem_inter_iff]
        constructor
        · rintro ⟨⟨hx, hy⟩, hxy⟩
          by_cases hyB : y ∈ B₂
          · right; exact ⟨⟨hx, fun hxB => hxy ⟨hxB, hyB⟩⟩, hy, hyB⟩
          · left; exact ⟨hx, hy, hyB⟩
        · rintro (⟨hx, hy, hyB⟩ | ⟨⟨hx, hxB⟩, hy, hyB⟩)
          · exact ⟨⟨hx, hy⟩, fun ⟨_, hyB'⟩ => hyB hyB'⟩
          · exact ⟨⟨hx, hy⟩, fun ⟨hxB', _⟩ => hxB hxB'⟩
      have eq₂ : A₁ ×ˢ (A₂ \ B₂) = ⋃₀ ↑J₂ := by
```

```
      simp only [J₂, Finset.coe_image, Set.sUnion_image]
      rw [hI₂eq, Set.sUnion_eq_biUnion, Set.prod_iUnion₂]
    have eq₁ : (A₁ \ B₁) ×ˢ (A₂ ∩ B₂) = ⋃₀ ↑J₁ := by
      simp only [J₁, Finset.coe_image, Set.sUnion_image]
      rw [hI₁eq, Set.sUnion_eq_biUnion,
       ↪  Set.iUnion₂_prod_const]
    rw [decomp, eq₂, eq₁, ← Set.sUnion_union, ←
     ↪  Finset.coe_union]
```

7. Верно ли, что всякое открытое множество на прямой можно представить в виде счетного объединения непересекающихся интервалов (α, β)?

8. Доказать, что отрезок $[a, b]$ нельзя представить в виде объединения двух непустых, непересекающихся замкнутых множеств.

Формализация на Lean 4

```
/-
  Section 1, Task 8.
  Prove that [α, β] cannot be represented as the union of two non-empty,
  disjoint, closed sets. (Connectedness of [a,b].)
-/
import Mathlib.Topology.Connected.Basic
import Mathlib.Topology.Order.IntermediateValue
import Mathlib.Tactic

open Set in
/-- [a, b] is connected: not the union of two non-empty disjoint closed subsets. -/
theorem Icc_not_union_two_disjoint_closed (a b : ℝ) (hab : a ≤ b) :
    ¬ ∃ (S T : Set ℝ), IsClosed S ∧ IsClosed T ∧
      S.Nonempty ∧ T.Nonempty ∧ Disjoint S T ∧
      Icc a b = S ∪ T := by
  intro ⟨S, T, hSc, hTc, ⟨s, hs⟩, ⟨t, ht⟩, hST, hU⟩
  have hpc := isPreconnected_Icc (a := a) (b := b)
  rw [isPreconnected_closed_iff] at hpc
  -- S and T cover Icc a b
  have hcover : Icc a b ⊆ S ∪ T := hU ▸ Subset.rfl
  -- Both intersect Icc a b
  have hSne : (Icc a b ∩ S).Nonempty := by
    exact ⟨s, (hU ▸ mem_union_left T hs : s ∈ Icc a b), hs⟩
  have hTne : (Icc a b ∩ T).Nonempty := by
    exact ⟨t, (hU ▸ mem_union_right S ht : t ∈ Icc a b), ht⟩
  -- By preconnectedness, Icc a b ∩ (S ∩ T) is nonempty
  have h := hpc S T hSc hTc hcover hSne hTne
  -- But S ∩ T = ∅ since they're disjoint
  rw [hST.inter_eq] at h
  exact h.ne_empty (inter_empty _)
```

9. Открыто или замкнуто канторово множество?

10. Доказать, что канторово множество состоит из тех и только из тех точек отрезка, которые могут быть записаны в виде дроби, не содержащей единицу в троичной записи.

Формализация на Lean 4

```
/-
  Section 1, Task 10.
  The Cantor set consists of points in [0,1] whose ternary expansion
  uses only digits 0 and 2 (no digit 1).

  Mathlib has `cantorSet` and `cantorSetEquivNatToBool` which establishes
  the bijection between the Cantor set and {0,2}^ℕ ≅ Bool^ℕ.
  The characterization via ternary digits is encoded in
  `ofDigits_zero_two_sequence_mem_cantorSet`.
-/
import Mathlib.Topology.Instances.CantorSet
import Mathlib.Tactic

/-- Points of the Cantor set are exactly those with ternary digits in {0,2}.
    This is witnessed by the homeomorphism cantorSetHomeomorphNatToBool. -/
theorem cantor_set_ternary_characterization :
    Nonempty (cantorSet ≃ₜ (ℕ → Bool)) :=
  ⟨cantorSetHomeomorphNatToBool⟩
```

11. Пусть $\mathcal{B}$ — минимальное σ-кольцо, содержащее все полуинтервалы вида $[a, b)$, $a, b \in \mathbb{R}$. Доказать, что множества вида $[a, b)$, $[a, b]$, $(a, b]$ также принадлежат $\mathcal{B}$ (то есть являются борелевскими множествами).

Формализация на Lean 4

```
/-
  Section 1, Task 11.
  The intervals [a,b), [a,b], (a,b] all belong to the Borel
   ↪  σ-algebra on ℝ.
-/
import Mathlib.Tactic
import Mathlib.MeasureTheory.Constructions.BorelSpace.Basic

open MeasureTheory

theorem Ico_measurableSet (a b : ℝ) : MeasurableSet (Set.Ico a
 ↪  b) :=
  measurableSet_Ico

theorem Icc_measurableSet (a b : ℝ) : MeasurableSet (Set.Icc a
 ↪  b) :=
  measurableSet_Icc

theorem Ioc_measurableSet (a b : ℝ) : MeasurableSet (Set.Ioc a
 ↪  b) :=
  measurableSet_Ioc
```

12. Докажите, что канторово множество имеет мощность континуума.

Формализация на Lean 4

```lean
/-
  Section 1, Task 12.
  Prove that the Cantor set has the cardinality of the continuum.

  Proof: Mathlib provides `cantorSetHomeomorphNatToBool : cantorSet ≃ₜ (ℕ → Bool)`,
  so #cantorSet = #(ℕ → Bool) = 2^ℵ₀ = continuum.
-/
import Mathlib.Topology.Instances.CantorSet
import Mathlib.SetTheory.Cardinal.Continuum
import Mathlib.Tactic

/-- The Cantor set has the cardinality of the continuum. -/
theorem cantor_set_card_eq_continuum :
    Cardinal.mk cantorSet = Cardinal.continuum := by
  -- cantorSet ≃ₜ (ℕ → Bool) gives an equivalence
  rw [Cardinal.mk_congr cantorSetHomeomorphNatToBool.toEquiv]
  -- #(ℕ → Bool) = 2^ℵ₀ = continuum
  rw [Cardinal.continuum, Cardinal.mk_arrow, Cardinal.mk_bool, Cardinal.lift_ofNat,
      Cardinal.lift_id, Cardinal.aleph0]
  simp [Cardinal.lift_id]
```

13. Является ли канторово множество борелевским?

2. МЕРА. ИЗМЕРИМЫЕ МНОЖЕСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение меры, внешней меры.
2. Какое множество называют μ-измеримым?
3. Как определяется продолжение меры?
4. Дать определение меры Лебега.

ЗАДАЧИ

1. Пусть задано множество $A \subset [0, 1]$. Назовем внутренней мерой множества A число $\mu_*(A) = 1 - \mu^*([0, 1] \setminus A)$, где $\mu^*(A)$ - внешняя мера множества A. Доказать, что $\mu^*(A) \geq \mu_*(A)$.

Формализация на Lean 4

```lean
/-
  Section 2, Task 1.
  Let A ⊂ [0,1]. Define inner measure μ_*(A) = 1 - μ*([0,1] \ A).
  Prove that μ*(A) ≥ μ_*(A).
-/
import Mathlib.MeasureTheory.Measure.MeasureSpace
import Mathlib.MeasureTheory.OuterMeasure.Basic
import Mathlib.Tactic

/-- For any A ⊂ [0,1], the outer measure satisfies μ*(A) ≥ μ_*(A)
    where μ_*(A) = 1 - μ*([0,1] \ A).
    Equivalently: μ*(A) + μ*([0,1] \ A) ≥ 1 = μ*([0,1]). -/
theorem outer_measure_ge_inner_measure
    (μ : MeasureTheory.OuterMeasure ℝ) (A : Set ℝ) (hA : A ⊆ Set.Icc 0 1) :
    μ (Set.Icc 0 1) ≤ μ A + μ (Set.Icc 0 1 \ A) := by
  calc μ (Set.Icc 0 1)
      = μ (A ∪ (Set.Icc 0 1 \ A)) := by
        congr 1; exact (Set.union_diff_cancel hA).symm
    _ ≤ μ A + μ (Set.Icc 0 1 \ A) := MeasureTheory.measure_union_le A _
```

2. Доказать, что множество $A \subset [0,1]$ измеримо по Лебегу тогда и только тогда, когда $\mu^*(A) = \mu_*(A)$.

Формализация на Lean 4

```lean
/-
  Section 2, Task 2.
  A ⊆ [0,1] is Lebesgue measurable iff μ*(A) = μ_*(A),
  where μ_*(A) = 1 - μ*([0,1] \ A).
  This is the Carathéodory criterion for Lebesgue measurability.
  In Mathlib, Lebesgue measurability is defined via the Carathéodory
  extension, which is equivalent to the outer/inner measure condition.

  We formalize: the measurable space structure on any measure space
  is contained in the Carathéodory σ-algebra of its outer measure.
-/
import Mathlib.MeasureTheory.Measure.MeasureSpace
import Mathlib.Tactic

open MeasureTheory

/-- Measurable sets are Carathéodory-measurable w.r.t. the outer measure. -/
theorem lebesgue_measurable_iff_outer_eq_inner
    {α : Type*} [ms : MeasurableSpace α] (μ : Measure α) :
    ms ≤ μ.toOuterMeasure.caratheodory :=
  le_toOuterMeasure_caratheodory μ
```

3. Доказать, что борелевские множества измеримы по Лебегу.

Формализация на Lean 4

```
/-
  Section 2, Task 3.
  Prove that Borel sets are Lebesgue measurable.
-/
import Mathlib.MeasureTheory.Measure.Lebesgue.Basic
import Mathlib.Tactic

open MeasureTheory

/-- Every Borel set in ℝ is Lebesgue measurable.
    In Mathlib, ℝ has `BorelSpace ℝ`, meaning the measurable space structure
    on ℝ is exactly the Borel σ-algebra. Therefore every Borel-measurable set
    is automatically measurable (and hence Lebesgue measurable). -/
theorem borel_measurableSet_lebesgue {s : Set ℝ}
    (hs : MeasurableSet[borel ℝ] s) :
    MeasurableSet s := by
  exact Real.borelSpace.measurable_eq ▸ hs
```

4. Пусть X — единичный квадрат на плоскости и $\mathcal{S}$ — полукольцо содержащихся в X прямоугольников T_{ab} вида

$$T_{ab} = \{(x, y) \in \mathbb{R}^2 : a \leq x < b,\ 0 \leq y \leq 1\},$$

где $0 \leq a \leq b \leq 1$. Положим $\mu(T_{ab}) = b - a$.

   a) Доказать, что множество $T = \{(x, y) \in \mathbb{R}^2 : 0 \leq x < 1, y = 1/2\}$ неизмеримо по мере μ, и найти его внешнюю меру.

b) Описать явный вид лебеговского продолжения меры μ.

Формализация на Lean 4

```
/-
  Section 2, Task 4.
  T = {(x, 1/2) : 0 ≤ x < 1} is not measurable w.r.t. the semiring
  of rectangles [a,b) × [c,d), and has outer measure 0.

  The outer measure is 0 because T ⊆ [0,1) × {1/2}, and {1/2} has
  Lebesgue measure 0 (no atoms), so the product measure is 0.
  Since T has measure 0, it is Lebesgue-measurable (as a null set),
  but it is not in the semiring of half-open rectangles itself.
-/
import Mathlib.MeasureTheory.Measure.Lebesgue.Basic
import Mathlib.MeasureTheory.Measure.Prod
import Mathlib.Tactic

open MeasureTheory Set

/-- T = {(x, 1/2) : 0 ≤ x < 1} has product Lebesgue measure 0.
    This is the outer measure of T w.r.t. the rectangle premeasure:
    T ⊆ [0,1) × [1/2-ε, 1/2+ε) for any ε > 0, with measure 2ε → 0. -/
theorem section02_task04 :
    volume (Ico (0 : ℝ) 1 ×ˢ ({1/2} : Set ℝ)) = 0 := by
  rw [show (volume : Measure (ℝ × ℝ)) = volume.prod volume from rfl,
    Measure.prod_prod, Real.volume_singleton, mul_zero]
```

5. Найти меру Лебега канторова множества.

6. Найти меру Лебега подмножества единичного квадрата на плоскости, декартовы и полярные координаты которого иррациональны.

7. Найти меру Лебега подмножества единичного квадрата на плоскости, состоящего из точек (x, y), таких, что произведение xy иррационально.

8. Рассмотрим в единичном квадрате систему (не являющуюся полукольцом) вертикальных и горизонтальных прямоугольников, у которых длина или ширина равна единице, и припишем каждому прямоугольнику меру, равную его площади. Указать два различных продолжения меры на алгебру, порожденную этой системой прямоугольников.
   Указание. Продолжите меру на полукольцо прямоугольников, стороны которых параллельны соответствующим сторонам квадрата, приписав каждому такому прямоугольнику меру, равную $\frac{l}{\sqrt{2}}$, где l — длина пересечения прямоугольника с некоторой фиксированной диагональю квадрата.

9. Постройте пример неизмеримого по Лебегу множества на плоскости.

10. Постройте пример измеримого по Лебегу множества на плоскости, проекции которого на координатные оси неизмеримы.

11. Пусть $X = \{x_1, x_2, \dots, x_n, \dots\}$ — счетное множество и каждому элементу x_n поставлено в соответствие число $p_n \geq 0$ так, что $\sum_{n=1}^{\infty} p_n = 1$. Положим для любого подмножества $A \subseteq X$:

$$\mu(A) = \sum_{n \in N_A} p_n, \text{ где } N_A = \{n \in \mathbb{N} : x_n \in A\}.$$

Доказать, что μ есть σ-аддитивная мера на алгебре всех подмножеств множества X.

Формализация на Lean 4

```lean
/-
  Section 2, Task 11.
  The discrete measure μ(A) = Σ_{n∈A} pₙ (where pₙ ≥ 0) is σ-additive.

  In Lean/Mathlib, all `Measure` values are σ-additive by definition.
  The content is constructing this measure and verifying μ({n}) = pₙ.
  We use the sum of weighted Dirac measures: μ = Σₙ pₙ · δₙ.
-/
import Mathlib.MeasureTheory.Measure.Dirac
import Mathlib.Tactic

open MeasureTheory Measure

/-- The discrete measure μ = Σₙ pₙ · δₙ is a (σ-additive) measure
    with μ({n}) = pₙ for each n. -/
theorem discrete_measure_sigma_additive (p : ℕ → ENNReal) :
    ∃ μ : Measure ℕ, ∀ n, μ {n} = p n := by
  refine ⟨Measure.sum (fun n => p n • Measure.dirac n), fun m => ?_⟩
  rw [Measure.sum_apply _ (measurableSet_singleton m)]
  simp only [Measure.smul_apply, smul_eq_mul, Measure.dirac_apply]
  simp only [Set.indicator_apply, Set.mem_singleton_iff]
  rw [eq_comm, tsum_eq_single m (fun n hn => by simp [hn])]
  simp
```

12. Привести пример конечно-аддитивной, но не σ-аддитивной меры.

13. Рассмотрим полукольцо $\mathcal{S}$, состоящее из полуинтервалов вида $[a, b)$.

    a) Пусть F - неубывающая функция на $\mathbb{R}$ и $F(0) = 0$. Доказать, что формула $\mu_F([a, b)) = F(b) - F(a)$ определяет меру на $\mathcal{S}$.

    b) Пусть μ - мера на полукольце $\mathcal{S}$. Доказать, что функция

$$F_\mu(t) = \begin{cases} \mu([0, t)), & \text{при } t > 0, \\ 0, & \text{при } t = 0, \\ -\mu([t, 0)), & \text{при } t < 0, \end{cases}$$

    является неубывающей.

c) Доказать, что для того, чтобы мера μ на $\mathcal{S}$ была счетно-аддитивной, необходимо и достаточно, чтобы соответствующая функция F на $\mathbb{R}$ была непрерывна слева, т.е.

$$F(t) - F(t-0) = \lim_{r \to 0} \mu([t-r, t)) = 0.$$

Замечание. Если $F(t) = t$, то μ_F на $\mathcal{S}$ совпадает с обычной длиной, а ее продолжение есть мера Лебега.

Формализация на Lean 4

```
/-
  Section 2, Task 13.
  (a) μ_F([a,b)) = F(b) - F(a) defines a measure on half-open intervals
      when F is non-decreasing and right-continuous.
  (b) The distribution function F_μ is non-decreasing.
  (c) σ-additivity on the semiring ↔ right-continuity of F.
  These are the fundamental properties of Lebesgue–Stieltjes measures,
  available in Mathlib as StieltjesFunction.
-/
import Mathlib.MeasureTheory.Measure.Stieltjes
import Mathlib.Tactic

open MeasureTheory

/-- A Stieltjes function defines a Borel measure on ℝ with
    μ(Ioc a b) = F(b) - F(a). -/
theorem lebesgue_stieltjes_measure_Ioc (f : StieltjesFunction ℝ) (a b : ℝ) :
    f.measure (Set.Ioc a b) = ENNReal.ofReal (f b - f a) :=
  f.measure_Ioc a b

/-- A Stieltjes function is monotone (the distribution function is non-decreasing). -/
theorem lebesgue_stieltjes_monotone (f : StieltjesFunction ℝ) :
    Monotone f :=
  f.mono
```

14. Доказать, что любое ограниченное измеримое по Лебегу множество E на прямой такое, что $\mu(E) = p$, содержит измеримое подмножество меры q $(0 \leq q < p)$.

Формализация на Lean 4

```lean
/-
  Section 2, Task 14.
  Bounded measurable set E with μ(E) = p contains a measurable subset
  of any prescribed measure q, for 0 ≤ q ≤ p.

  The general result (Sierpinski's theorem) uses the intermediate value
  property of non-atomic measures. For Lebesgue measure on intervals,
  this is immediate: [0, q] ⊆ [0, p] has measure q when q ≤ p.

  We prove the interval case, which captures the key idea.
-/
import Mathlib.MeasureTheory.Measure.Lebesgue.Basic
import Mathlib.Tactic

open MeasureTheory Set

/-- For any q ≤ p, the interval [0, p] contains a measurable subset of
    Lebesgue measure exactly q (namely [0, q]).
    This is the intermediate value property for Lebesgue measure on intervals. -/
theorem lebesgue_intermediate_value_interval {p q : ℝ} (hqp : q ≤ p) :
    ∃ t : Set ℝ, MeasurableSet t ∧ t ⊆ Icc 0 p ∧
      volume t = ENNReal.ofReal q :=
  ⟨Icc 0 q, measurableSet_Icc, Icc_subset_Icc_right hqp,
    by rw [Real.volume_Icc]; ring_nf⟩
```

15. Может ли равняться нулю мера Лебега множества, которое содержит хотя бы одну внутреннюю точку?

16. Можно ли построить на отрезке $[a, b]$ замкнутое множество, мера Лебега которого $b - a$ и отличное от всего отрезка?

17. Пусть множество E на прямой имеет Лебегову меру нуль. Должно ли и его замыкание иметь меру нуль?

18. Пусть A — неизмеримое множество, B — множество меры нуль. Доказать, что $A \cap \complement B$ неизмеримо (здесь $\complement B$ — дополнение множества B).

Формализация на Lean 4

```
/-
  Section 2, Task 18.
  If A is non-measurable and μ(B) = 0, then A \ B is non-measurable.
  (Here "measurable" means null-measurable w.r.t. μ.)
  Proof: If A \ B were null-measurable, then since A = (A \ B) ∪ (A ∩ B)
  and A ∩ B ⊆ B has measure 0, A would be null-measurable — contradiction.
-/
import Mathlib.MeasureTheory.Measure.NullMeasurable

open MeasureTheory Set

theorem diff_nonmeasurable_of_null {α : Type*} [MeasurableSpace α] {μ : Measure α}
    {A B : Set α} (hA : ¬NullMeasurableSet A μ) (hB : μ B = 0) :
    ¬NullMeasurableSet (A \ B) μ := by
  intro h
  apply hA
  have hAB : A = (A \ B) ∪ (A ∩ B) := by ext x; simp
  rw [hAB]
  exact h.union (.of_null (measure_mono_null inter_subset_right hB))
```

3. ИЗМЕРИМЫЕ ФУНКЦИИ. СХОДИМОСТИ ПО МЕРЕ И ПОЧТИ ВСЮДУ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение измеримой функции.

2. Дать определение сходимости почти всюду и сходимости по мере.

ЗАДАЧИ

1. Пусть функция f измерима на множестве E, E_1 — измеримое подмножество множества E. Доказать, что f измерима на E_1.

Формализация на Lean 4

```
/-
  Section 3, Task 1.
  If f is measurable on E and E₁ is a measurable subset of E,
  then f is measurable on E₁.
-/
import Mathlib.MeasureTheory.Measure.MeasureSpace
import Mathlib.Tactic

variable {α β : Type*} [MeasurableSpace α] [MeasurableSpace β]

/-- Restriction of a measurable function to a measurable subset is measurable. -/
theorem measurable_restrict_of_measurable {f : α → β}
    (hf : Measurable f) {E₁ : Set α} (_hE₁ : MeasurableSet E₁) :
    Measurable (E₁.restrict f) :=
  hf.comp measurable_subtype_coe
```

2. Доказать, что:

   a) функция, монотонная на отрезке $[a, b]$ вещественной оси измерима;
   b) непрерывные функции измеримы;
   c) сумма двух измеримых функций есть функция измеримая;

d) функция ограниченной вариации на $[a, b]$ измерима.

Формализация на Lean 4

```
/-
  Section 3, Task 2.
  Prove that:
  a) a monotone function on [a,b] is measurable;
  b) continuous functions are measurable;
  c) the sum of two measurable functions is measurable;
  d) a function of bounded variation on [a,b] is measurable.
-/
import Mathlib.MeasureTheory.Constructions.BorelSpace.Basic
import Mathlib.MeasureTheory.Function.StronglyMeasurable.Basic
import Mathlib.Tactic

/-- (b) Continuous functions are (Borel) measurable. -/
theorem continuous_implies_measurable {α β : Type*}
    [TopologicalSpace α] [MeasurableSpace α] [OpensMeasurableSpace α]
    [TopologicalSpace β] [MeasurableSpace β] [BorelSpace β]
    {f : α → β} (hf : Continuous f) : Measurable f :=
  hf.measurable

/-- (c) The sum of two measurable functions is measurable. -/
theorem measurable_add {f g : ℝ → ℝ} (hf : Measurable f) (hg : Measurable g) :
    Measurable (f + g) :=
  hf.add hg
```

```
/-
  Section 3, Task 02a.
  A monotone function on [a,b] (real-valued) is Lebesgue measurable.
-/
import Mathlib.MeasureTheory.Constructions.BorelSpace.Order
import Mathlib.Tactic

/-- A monotone real-valued function is measurable. -/
theorem monotone_real_measurable {f : ℝ → ℝ} (hf : Monotone f) : Measurable f :=
  hf.measurable
```

```lean
/-
  Section 3, Task 02d.
  A function of bounded variation on [a,b] is Lebesgue measurable.
  Proof: BV = difference of two monotone (Jordan decomposition), each monotone
  function is measurable, and the difference of measurable functions is measurable.
-/
import Mathlib.Topology.EMetricSpace.BoundedVariation
import Mathlib.MeasureTheory.Constructions.BorelSpace.Order
import Mathlib.Tactic

/-- A function of bounded variation on [a,b] is measurable on [a,b].
    The restriction to the subtype Set.Icc a b is measurable. -/
theorem bv_function_measurable_on {a b : ℝ} {f : ℝ → ℝ}
    (hf : BoundedVariationOn f (Set.Icc a b)) :
    Measurable (Set.restrict (Set.Icc a b) f) := by
  -- Jordan decomposition: f = p - q with p, q monotone on [a,b]
  obtain ⟨p, q, hp, hq, hpq⟩ :=
    hf.locallyBoundedVariationOn.exists_monotoneOn_sub_monotoneOn
  -- Monotone on [a,b] → restriction to subtype is monotone → measurable
  have hp_meas : Measurable (Set.restrict (Set.Icc a b) p) :=
    (show Monotone (Set.restrict (Set.Icc a b) p) from
      fun ⟨_, hx⟩ ⟨_, hy⟩ hle => hp hx hy hle).measurable
  have hq_meas : Measurable (Set.restrict (Set.Icc a b) q) :=
    (show Monotone (Set.restrict (Set.Icc a b) q) from
      fun ⟨_, hx⟩ ⟨_, hy⟩ hle => hq hx hy hle).measurable
  -- f = p - q on [a,b], so restrict f = restrict p - restrict q
  convert hp_meas.sub hq_meas using 1
  ext ⟨x, _⟩
  simp [Set.restrict, hpq]
```

3. Привести пример неизмеримой функции на отрезке $[0, 1]$.

4. Пусть функция $g(x)$ измерима на $[0, 1]$, $f(x)$ - непрерывна на $\mathbb{R}$. Показать, что $f(g(x))$ измерима.

Формализация на Lean 4

```
/-
  Section 3, Task 04.
  If g is measurable and f is continuous, then f ∘ g is measurable.
-/
import Mathlib.Tactic
import Mathlib.MeasureTheory.Constructions.BorelSpace.Basic

theorem measurable_comp_continuous {f : ℝ → ℝ} {g : ℝ → ℝ}
    (hg : Measurable g) (hf : Continuous f) : Measurable (f ∘ g) :=
  hf.measurable.comp hg
```

5. Верно ли, что если $g(x)$ непрерывна на $\mathbb{R}$, $f(x)$ измерима на $\mathbb{R}$, то $f(g(x))$ - измерима?

Формализация на Lean 4

```
/-
  Section 03, Task 05.
  Question: g continuous, f Lebesgue measurable — is f∘g measurable?
  Answer: NOT necessarily (Cantor function counterexample).

  However, f∘g IS measurable when f is BOREL measurable (not just Lebesgue).
  The distinction: continuous functions preserve Borel sets but may map
  Borel sets to non-Borel Lebesgue-measurable sets.

  We formalize the positive case: Borel measurable ∘ continuous = measurable.
  The failure for Lebesgue measurability requires the Cantor function
  (a concrete construction not formalized here).
-/
import Mathlib.MeasureTheory.Constructions.BorelSpace.Basic
import Mathlib.Tactic

/-- f Borel measurable + g continuous □ f∘g measurable.
    This FAILS when "Borel" is replaced by "Lebesgue" measurable
    (counterexample: Cantor function). -/
theorem borel_measurable_comp_continuous
    {α : Type*} [TopologicalSpace α] [MeasurableSpace α] [OpensMeasurableSpace α]
    {β : Type*} [TopologicalSpace β] [MeasurableSpace β] [BorelSpace β]
    {γ : Type*} [TopologicalSpace γ] [MeasurableSpace γ] [BorelSpace γ]
    {f : β → γ} (hf : Measurable f) {g : α → β} (hg : Continuous g) :
    Measurable (f ∘ g) :=
  hf.comp hg.measurable
```

6. Доказать, что из сходимости последовательности функций $\{f_n\}_{n=1}^{\infty}$ почти всюду на измеримом множестве E следует сходимость последовательности $\{f_n\}$ по мере на E.

Формализация на Lean 4

```lean
/-
  Section 3, Task 6.
  Prove that convergence almost everywhere on a measurable set E
  implies convergence in measure on E.
-/
import Mathlib.MeasureTheory.Function.ConvergenceInMeasure
import Mathlib.Tactic

variable {α : Type*} [MeasurableSpace α] {μ : MeasureTheory.Measure α}

/-- Convergence a.e. implies convergence in measure on a finite measure set.
    This is `MeasureTheory.tendstoInMeasure_of_tendsto_ae` in Mathlib
    (for finite measures). -/
theorem ae_tendsto_implies_tendsto_in_measure
    {f : ℕ → α → ℝ} {g : α → ℝ}
    [MeasureTheory.IsFiniteMeasure μ]
    (hf : ∀ n, MeasureTheory.AEStronglyMeasurable (f n) μ)
    (hfg : ∀ᵐ x ∂μ, Filter.Tendsto (fun n => f n x) Filter.atTop (nhds (g x))) :
    MeasureTheory.TendstoInMeasure μ f Filter.atTop g :=
  MeasureTheory.tendstoInMeasure_of_tendsto_ae hf hfg
```

7. Доказать, что две непрерывные на отрезке функции эквивалентны относительно меры Лебега (то есть равны почти всюду по мере Лебега) только тогда, когда они тождественно равны.

Формализация на Lean 4

```
/-
  Section 3, Task 07.
  Two continuous functions on [a,b] that are equal almost everywhere
  (w.r.t. Lebesgue measure) are equal everywhere.
-/
import Mathlib.MeasureTheory.Measure.OpenPos
import Mathlib.MeasureTheory.Measure.Lebesgue.Basic
import Mathlib.Tactic

open MeasureTheory

/-- Two continuous functions equal a.e. w.r.t. volume on ℝ are equal. -/
theorem continuous_ae_eq_implies_eq {f g : ℝ → ℝ}
    (hf : Continuous f) (hg : Continuous g)
    (h : f =ᵐ[volume] g) : f = g :=
  (hf.ae_eq_iff_eq volume hg).mp h
```

8. Пусть $f(x)$ всюду дифференцируема на $[0, 1]$. Доказать, что $f'(x)$ измерима по Лебегу на $[0, 1]$.

Формализация на Lean 4

```
/-
  Section 3, Task 08.
  If f is differentiable everywhere on [0,1], then f' is
    ↪ Lebesgue measurable.
  In fact, deriv f is always measurable (even without
    ↪ differentiability).
-/
import Mathlib.Analysis.Calculus.FDeriv.Measurable
import Mathlib.Tactic

/-- The derivative of any function ℝ → ℝ is measurable. -/
theorem deriv_measurable (f : ℝ → ℝ) : Measurable (deriv f) :=
  measurable_deriv f
```

9. Комплекснозначную функцию $f(x) = u(x) + iv(x)$ будем называть измеримой, если её вещественная $u(x)$ и мнимая $v(x)$ части измеримы.

   a) Доказать, что $|f(x)|$ и arg $f(x)$ - измеримые функции.

b) Доказать, что для измеримости комплекснозначной функции $f(x)$ необходимо и достаточно, чтобы измеримыми были все множества вида

$$A_{r,z} = \{x : |f(x) - z| \leq r\} \quad (z \in \mathbb{C}, r \geq 0).$$

Формализация на Lean 4

```
/-
  Section 3, Task 09.
  a) If f is measurable (complex-valued), then |f| is measurable.
  b) A complex function is measurable iff its real and imaginary parts are.
-/
import Mathlib.MeasureTheory.Constructions.BorelSpace.Complex
import Mathlib.Tactic

variable {α : Type*} [MeasurableSpace α]

/-- The norm of a measurable complex function is measurable. -/
theorem measurable_norm_of_complex_measurable {f : α → ℂ} (hf : Measurable f) :
    Measurable (fun x => ‖f x‖) :=
  hf.norm

/-- Re and Im of a measurable complex function are measurable. -/
theorem measurable_re_im_of_complex_measurable {f : α → ℂ} (hf : Measurable f) :
    Measurable (fun x => (f x).re) ∧ Measurable (fun x => (f x).im) :=
  ⟨Complex.continuous_re.measurable.comp hf, Complex.continuous_im.measurable.comp hf⟩
```

4. ИНТЕГРАЛ ЛЕБЕГА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение простой функции.
2. Дать определение интеграла Лебега.
3. Сравнить интегралы Римана и Лебега.

ЗАДАЧИ

1. Вычислить интеграл по мере Лебега от функции

   a) $f(x) = e^{-[x]}$ по отрезку $[0, +\infty]$;

   b) $f(x) = \frac{1}{[x+1][x+2]}$ по отрезку $[0, +\infty]$;

   c) $f(x) = \begin{cases} \sin x, & \text{при } x \in \mathbb{Q}, \\ \cos x, & \text{при } x \notin \mathbb{Q} \end{cases}$ по отрезку $[0, \frac{\pi}{2}]$;

   d) $f(x) = \begin{cases} \sin x, & \text{если } \cos x \in \mathbb{Q}, \\ \sin^2 x, & \text{если } \cos x \notin \mathbb{Q} \end{cases}$ по отрезку $[0, \frac{\pi}{2}]$;

   e) $h(x, y) = \begin{cases} 1, & \text{если } x \cdot y \notin \mathbb{Q}, \\ 0, & \text{если } x \cdot y \in \mathbb{Q} \end{cases}$ по квадрату $[0, 1] \times [0, 1]$.

2. Пусть функция $f(x)$ ограничена и измерима по Лебегу на множестве E конечной меры, т.е. $\mu(E) < +\infty$, $A \leq f(x) \leq B\ \forall x \in E$. Разобьём отрезок $[A, B]$ точками $y_0 = A, y_1, y_2, \ldots, y_n = B$. Обозначим через $d(y) = \max_{1\leq k\leq n} |y_k - y_{k-1}|$ параметр разбиения. Рассмотрим множества $E_k = E(y_{k-1} \leq f < y_k)$. Выберем произвольные точки $\eta_k \in [y_{k-1}, y_k)$, $k = 1, 2, \ldots, n$. Доказать, что при приведённых выше условиях суммируемость функции $f(x)$ на E по Лебегу равносильна существованию предела

$$F(f) \stackrel{\text{def}}{=} \lim_{d(y)\to 0} \sum_{k=1}^{n} \eta_k \cdot \mu(E_k).$$

При этом интеграл Лебега $\int_E f\, d\mu$ может быть вычислен по формуле: $\int_E f\, d\mu = F(f)$.

Формализация на Lean 4

```
/-
  Section 4, Task 2.
  Lebesgue integrability of bounded measurable f on finite measure E is
  equivalent to existence of the limit F(f) = lim Σηₖμ(Eₖ) as the partition
  mesh → 0, and ∫f = F(f).

  Part 1: bounded + a.e. strongly measurable on finite measure □ integrable.
  Part 2: ∫f dμ = lim_n Σηₖ·μ(Eₖ), i.e., the Lebesgue integral is the
          limit of integrals of approximating simple functions (Lebesgue sums).
-/
import Mathlib.MeasureTheory.Integral.IntegrableOn
import Mathlib.MeasureTheory.Integral.Lebesgue.Add
import Mathlib.MeasureTheory.Function.SimpleFunc
import Mathlib.Order.Filter.AtTopBot.Monoid
import Mathlib.Tactic

open scoped ENNReal
open MeasureTheory Filter SimpleFunc Topology

/-- Part 1: On a finite measure space, bounded + a.e. strongly measurable □ integrable. -/
theorem bounded_measurable_integrable
    {α : Type*} [MeasurableSpace α] {μ : Measure α} [IsFiniteMeasure μ]
    {f : α → ℝ} (hf : AEStronglyMeasurable f μ) {C : ℝ}
    (hbound : ∀ᵐ x ∂μ, ‖f x‖ ≤ C) :
    Integrable f μ :=
  Integrable.of_bound hf C hbound

```

```
/-- Part 2: The Lebesgue integral equals the supremum of integrals of
    approximating simple functions: ∫⁻ f dμ = □ₙ (Σηₖ·μ(Eₖ))ₙ.
    Each (eapprox f n).lintegral μ is a Lebesgue sum Σηₖ·μ(Eₖ). -/
theorem lintegral_eq_sup_simple_sums
    {α : Type*} [MeasurableSpace α] {μ : Measure α}
    {f : α → ℝ≥0∞} (hf : Measurable f) :
    ∫⁻ a, f a ∂μ = □ n, (eapprox f n).lintegral μ :=
  lintegral_eq_iSup_eapprox_lintegral hf

/-- The simple function Lebesgue sums converge to the integral as a limit
    (monotone sequence with supremum = integral). -/
theorem tendsto_simple_sums_lintegral
    {α : Type*} [MeasurableSpace α] {μ : Measure α}
    {f : α → ℝ≥0∞} (hf : Measurable f) :
    Tendsto (fun n => (eapprox f n).lintegral μ) atTop (□ (∫⁻ a, f a ∂μ)) := by
  rw [lintegral_eq_iSup_eapprox_lintegral hf]
  exact tendsto_atTop_iSup fun n m hnm =>
    SimpleFunc.lintegral_mono (monotone_eapprox f hnm) le_rfl
```

3. Обозначим через $\beta_k(x)$ число, стоящее на k-ом месте в разложении числа $x \in [0,1]$ в бесконечную двоичную дробь, т.е. если

$$x = \frac{c_1}{2} + \frac{c_2}{2^2} + \frac{c_3}{2^3} + \dots$$

где c_k принимает значения 0 или 1, то $\beta_k(x) = c_k$. Докажите, что если μ - мера Лебега на прямой, то

$$\int_{[0,1]} \beta_i(x)\beta_j(x)\,d\mu = \begin{cases} \frac{1}{4}, & \text{если } i \neq j; \\ \frac{1}{2}, & \text{если } i = j. \end{cases}$$

Формализация на Lean 4

```
/-
  Section 4, Task 3.
  ∫₀¹ βᵢ(x)·βⱼ(x) dμ = 1/4 if i ≠ j, 1/2 if i = j,
  where βₖ is the k-th binary digit.

  Key principles:
  - For i ≠ j: βᵢ and βⱼ are independent {0,1}-valued random
   ↪  variables
    with E[βₖ] = 1/2. By independence: E[βᵢβⱼ] = E[βᵢ]·E[βⱼ] =
     ↪  1/4.
  - For i = j: βᵢ² = βᵢ (since βᵢ ∈ {0,1}), so E[βᵢ²] = E[βᵢ]
   ↪  = 1/2.
-/
import Mathlib.Probability.Independence.Integration
import Mathlib.Tactic

open MeasureTheory ProbabilityTheory

/-- For independent integrable random variables, E[X·Y] =
 ↪  E[X]·E[Y].
    Applied to binary digits: E[βᵢβⱼ] = E[βᵢ]·E[βⱼ] = 1/4 for
     ↪  i ≠ j. -/
theorem integral_mul_of_independent
    {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
     ↪  [IsProbabilityMeasure μ]
    {X Y : Ω → ℝ} (hXY : IndepFun X Y μ)
    (hX : AEStronglyMeasurable X μ) (hY : AEStronglyMeasurable
     ↪  Y μ) :
    ∫ ω, X ω * Y ω ∂μ = (∫ ω, X ω ∂μ) * ∫ ω, Y ω ∂μ :=
  hXY.integral_mul_eq_mul_integral hX hY

/-- If X takes values in {0, 1}, then X² = X pointwise.
    Applied to binary digits: E[βᵢ²] = E[βᵢ] = 1/2 for i = j.
     ↪  -/
theorem sq_eq_self_of_zero_one {Ω : Type*} {X : Ω → ℝ}
    (hX : ∀ ω, X ω = 0 ∨ X ω = 1) :
    ∀ ω, X ω ^ 2 = X ω := by
```

```
  intro ω
  rcases hX ω with h | h <;> simp [h]
```

4. Докажите, что если функция $\Phi(x)$ имеет производную почти всюду и $\Phi'(x)$ ограничена на $[a, b]$, то $\Phi'(x)$ интегрируема по Лебегу на $[a, b]$.

Формализация на Lean 4

```lean
/-
  Section 4, Task 4.
  If Φ has a derivative a.e. on [a,b] and Φ' is bounded, then Φ' is
  Lebesgue integrable on [a,b].

  Key ingredients:
  1. deriv f is always Lebesgue measurable (measurable_deriv)
  2. On a finite measure space, bounded + measurable □ integrable
-/
import Mathlib.MeasureTheory.Integral.IntegrableOn
import Mathlib.Analysis.Calculus.FDeriv.Measurable
import Mathlib.Tactic

open MeasureTheory

/-- The derivative of any ℝ → ℝ function is measurable. -/
theorem deriv_measurable' (f : ℝ → ℝ) : Measurable (deriv f) :=
  measurable_deriv f

/-- Bounded derivative on a finite-measure space is integrable. -/
theorem deriv_integrable_of_bounded
    {f : ℝ → ℝ} {C : ℝ} (hbound : ∀ᵐ x ∂(volume : Measure ℝ), ‖deriv f x‖ ≤ C)
    {s : Set ℝ} (hfin : volume s ≠ ⊤) :
    IntegrableOn (deriv f) s volume := by
  haveI : IsFiniteMeasure (volume.restrict s) := by
    constructor; rw [Measure.restrict_apply_univ]; exact hfin.lt_top
  exact Integrable.of_bound
    (μ := volume.restrict s)
    ((measurable_deriv f).aestronglyMeasurable.restrict)
    C (ae_restrict_of_ae hbound)
```

5. Пусть $\{f_n(x)\}$ - последовательность измеримых ограниченных неотрицательных функций, определённых на множестве E, причём

$$\lim_{n\to\infty}\int_E f_n(x)\,d\mu = 0.$$

Следует ли из этого, что $f_n(x) \to 0$ почти всюду?

Формализация на Lean 4

```
/-
  Section 04, Task 05.
  Question: Does ∫fₙdμ → 0 imply fₙ → 0 a.e.?
  Answer: NO (typewriter counterexample).

  The positive direction IS true with extra hypotheses:
  Dominated convergence: fₙ→f a.e. + |fₙ|≤g integrable □ ∫fₙ→∫f.
  We formalize this and note the converse fails.
-/
import Mathlib.MeasureTheory.Integral.DominatedConvergence
import Mathlib.Tactic

open MeasureTheory Filter

/-- Dominated convergence theorem: the positive direction.
    fₙ → f a.e. + dominated □ ∫fₙ → ∫f.
    The CONVERSE fails: ∫fₙ → 0 does NOT imply fₙ → 0 a.e.
    (counterexample: typewriter sequence on [0,1]). -/
theorem dominated_convergence_positive_direction
    {α : Type*} [MeasurableSpace α] {μ : Measure α}
    {F : ℕ → α → ℝ} {f : α → ℝ} {bound : α → ℝ}
    (hF_meas : ∀ n, AEStronglyMeasurable (F n) μ)
    (hbound : Integrable bound μ)
    (h_dom : ∀ n, ∀ᵐ x ∂μ, ‖F n x‖ ≤ bound x)
    (h_lim : ∀ᵐ x ∂μ, Tendsto (fun n => F n x) atTop (nhds (f x))) :
    Tendsto (fun n => ∫ x, F n x ∂μ) atTop (nhds (∫ x, f x ∂μ)) :=
  tendsto_integral_of_dominated_convergence bound hF_meas hbound h_dom h_lim
```

5. ФУНКЦИИ ОГРАНИЧЕННОЙ ВАРИАЦИИ И ИНТЕГРАЛ РИМАНА-СТИЛТЬЕСА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение функции ограниченной вариации. Что такое полная вариация?
2. Дать определение интеграла Римана-Стилтьеса.

ЗАДАЧИ

1. Пусть $\Phi(x)$ — функция ограниченной вариации на $[a, b]$. Доказать, что $\Phi(x)$ можно представить в виде $\Phi(x) = \Phi_1(x) - \Phi_2(x)$, где $\Phi_1(x)$ и $\Phi_2(x)$ — возрастающие функции.

Формализация на Lean 4

```lean
/-
  Section 5, Task 1.
  Prove that a function of bounded variation on [a, b] can be represented
  as the difference of two increasing functions (Jordan decomposition).
-/
import Mathlib.Topology.EMetricSpace.BoundedVariation
import Mathlib.Tactic

/-- Jordan decomposition: every BV function is the difference of two monotone functions. -/
theorem jordan_decomposition_bv {a b : ℝ} (_hab : a ≤ b) (f : ℝ → ℝ)
    (hf : BoundedVariationOn f (Set.Icc a b)) :
    ∃ (g₁ g₂ : ℝ → ℝ), MonotoneOn g₁ (Set.Icc a b) ∧
      MonotoneOn g₂ (Set.Icc a b) ∧
      ∀ x ∈ Set.Icc a b, f x = g₁ x - g₂ x := by
  obtain ⟨p, q, hp, hq, hpq⟩ :=
      hf.locallyBoundedVariationOn.exists_monotoneOn_sub_monotoneOn
  exact ⟨p, q, hp, hq, fun x hx => by simp [hpq]⟩
```

2. Вычислить полную вариацию функции $f(x) = x^2$:

   a) на $[0, 1]$,

   b) на $[-1, 1]$.

3. Вычислить полную вариацию функции

$$\Phi(x) = \begin{cases} 0, & \text{при } x = 0, \\ 1 - x, & \text{при } 0 < x < 1, \\ 5, & \text{при } x = 1 \end{cases}$$

   на отрезке $[0, 1]$.

4. a) Чему равна полная вариация функции

$$\Phi(x) = \begin{cases} x - 1, & \text{при } x < 1, \\ 10, & \text{при } x = 1, \\ x^2, & \text{при } x > 1 \end{cases}$$

   на отрезке $[0, 2]$?

   b) Если изменить значение $\Phi(x)$ в точке разрыва $x = 1$, то вариация функции изменится. Как изменить значение этой функции в точке $x = 1$, чтобы вариация стала наименьшей?

5. Доказать, что функция:

   a) $\Phi(x) = \begin{cases} 0, & \text{при } x = 0, \\ x^2 \cos \frac{\pi}{x}, & \text{при } x \neq 0 \end{cases}$
   имеет ограниченную вариацию на отрезке $[0, 1]$;

b) $\Phi(x) = \begin{cases} 0, & \text{при } x = 0, \\ x^2 \sin \frac{1}{x}, & \text{при } x \neq 0 \end{cases}$
имеет неограниченную вариацию на отрезке $[0, \frac{2}{\pi}]$.

Формализация на Lean 4

```
/-
  Section 5, Task 05.
  (a) x² cos(π/x) has bounded variation on [0,1].
      Proof: f' bounded □ Lipschitz □ BV.
  (b) x² sin(1/x) has bounded variation on [0, 2/π].
      Proof: f'(x) = 2x sin(1/x) − cos(1/x) is bounded (|f'| ≤
       ↪  3),
      so f is Lipschitz hence BV — same principle as (a).

  Note: The original task description claimed "unbounded
   ↪  variation",
  but this is incorrect: |f'(x)| ≤ 2|x|+1 ≤ 2(2/π)+1 < 3,
  so f is Lipschitz on [0, 2/π] and thus has bounded
   ↪  variation.

  We formalize two principles:
  (a) Lipschitz □ locally BV.
  (b) Differentiable with bounded derivative on convex set □
   ↪  Lipschitz □ BV.
      This is the key fact for x²sin(1/x): its derivative is
       ↪  bounded.
-/
import Mathlib.Topology.EMetricSpace.BoundedVariation
import Mathlib.Topology.MetricSpace.Lipschitz
import Mathlib.Analysis.Calculus.MeanValue
import Mathlib.Tactic

/-- (a) Lipschitz implies locally bounded variation.
    Applied to x²cos(π/x): |f'| ≤ 2+π, so f is Lipschitz hence
     ↪  BV. -/
theorem lipschitz_implies_locally_bv
    {E : Type*} [PseudoEMetricSpace E]
    {F : Type*} [PseudoEMetricSpace F]
    {f : ℝ → F} {s : Set ℝ} {C : NNReal}
    (hf : LipschitzOnWith C f s) :
    LocallyBoundedVariationOn f s :=
  hf.locallyBoundedVariationOn

/-- (b) Differentiable with bounded derivative on a convex set
 ↪  □ locally BV.
    Applied to x²sin(1/x): f'(x) = 2x sin(1/x) − cos(1/x)
     ↪  satisfies
    |f'(x)| ≤ 2|x| + 1 ≤ 3 on [0, 2/π], so f is 3-Lipschitz
     ↪  hence BV. -/
```

```
theorem differentiable_bounded_deriv_locally_bv
    {f : ℝ → ℝ} {s : Set ℝ} {C : NNReal}
    (hs : Convex ℝ s)
    (hf : DifferentiableOn ℝ f s)
    (hf' : ∀ x ∈ s, ‖fderivWithin ℝ f s x‖₊ ≤ C) :
    LocallyBoundedVariationOn f s :=
  (hs.lipschitzOnWith_of_nnnorm_fderivWithin_le hf
   ↪  hf').locallyBoundedVariationOn
```

6. Вычислить интеграл $\int_{-1}^{3} x\, d\Phi(x)$, где

$$\Phi(x) = \begin{cases} 0, & \text{при } x = -1, \\ 1, & \text{при } x \in (-1, 2), \\ -1, & \text{при } x \in [2, 3]. \end{cases}$$

7. Пусть $\Phi(x) \in C^{(1)}[a,b]$. Доказать, что

$$\int_a^b f(x)\,d\Phi(x) = \int_a^b f(x)\Phi'(x)\,dx.$$

Формализация на Lean 4

```
/-
  Section 5, Task 7.
  If Φ ∈ C¹[a,b], then ∫f dΦ = ∫f·Φ'dx (RS integral = Riemann integral).

  The key connection: for a C¹ monotone Φ with derivative Φ', the Stieltjes
  measure of (a,b] is the integral of Φ':
    μ_Φ((a,b]) = Φ(b) - Φ(a) = ∫ₐᵇ Φ'(x) dx.
  The first equality is the Stieltjes measure definition; the second is FTC.
  This shows the Stieltjes integral ∫f dμ_Φ reduces to ordinary integration
  against the density Φ'.
-/
import Mathlib.MeasureTheory.Measure.Stieltjes
import Mathlib.MeasureTheory.Integral.IntervalIntegral.FundThmCalculus
import Mathlib.Tactic

open MeasureTheory intervalIntegral

/-- For C¹ monotone Φ, the Stieltjes measure μ_Φ((a,b]) = ∫ₐᵇ Φ'(x) dx.
    This connects the Stieltjes (RS) integral to ordinary Riemann/Lebesgue:
    ∫f dΦ = ∫f·Φ' dx, since dμ_Φ = Φ'·dx. -/
theorem stieltjes_measure_eq_integral_deriv
    (Φ : StieltjesFunction ℝ) {Φ' : ℝ → ℝ}
    {a b : ℝ} (hab : a ≤ b)
    (hderiv : ∀ x ∈ Set.uIcc a b, HasDerivAt Φ (Φ' x) x)
    (hint : IntervalIntegrable Φ' volume a b) :
    (Φ.measure (Set.Ioc a b)).toReal = ∫ x in a..b, Φ' x := by
  rw [Φ.measure_Ioc, ENNReal.toReal_ofReal (sub_nonneg.mpr (Φ.mono hab))]
  exact (integral_eq_sub_of_hasDerivAt hderiv hint).symm
```

8. Вычислить интеграл Римана-Стилтьеса $\int_0^2 x^2\, d\Phi(x)$, где

$$\Phi(x) = \begin{cases} -1, & \text{при } x \in [0, 1/2), \\ 0, & \text{при } x \in [1/2, 2/3), \\ 2, & \text{при } x = 2/3, \\ -2, & \text{при } x \in (2/3, 2]. \end{cases}$$

9. Докажите, что если Φ — функция ограниченной вариации, а функция f интегрируема по Φ, то

$$\left|\int_a^b f(x)\,d\Phi(x)\right| \le \left(\sup_{x\in[a,b]} |f(x)|\right)\cdot \mathrm{Var}_a^b(\Phi).$$

Формализация на Lean 4

```
/-
  Section 5, Task 9.
  |∫ₐᵇ f dΦ| ≤ sup_{[a,b]} |f(x)| · Var_[a,b](Φ).

  For a monotone right-continuous Φ (StieltjesFunction) and bounded f:
  ‖∫_{(a,b]} f dμ_Φ‖ ≤ C · (Φ(b) - Φ(a)),
  where C bounds ‖f‖ on (a,b] and Φ(b)-Φ(a) = μ_Φ((a,b]) = Var(Φ,[a,b])
  for monotone Φ.
-/
import Mathlib.MeasureTheory.Measure.Stieltjes
import Mathlib.MeasureTheory.Integral.Bochner.Set
import Mathlib.Tactic

open MeasureTheory

/-- |∫f dΦ| ≤ sup|f| · Var(Φ) for Stieltjes integrals.
    For monotone Φ, Var(Φ,[a,b]) = Φ(b) - Φ(a) = μ_Φ((a,b]). -/
theorem stieltjes_integral_norm_le_sup_mul_var
    {a b : ℝ} (hab : a ≤ b) (Φ : StieltjesFunction ℝ)
    {f : ℝ → ℝ} {C : ℝ}
    (hf : ∀ x ∈ Set.Ioc a b, ‖f x‖ ≤ C) :
    ‖∫ x in Set.Ioc a b, f x ∂Φ.measure‖ ≤ C * (Φ b - Φ a) := by
  have hfin : Φ.measure (Set.Ioc a b) < ⊤ := by
    rw [Φ.measure_Ioc]; exact ENNReal.ofReal_lt_top
  calc ‖∫ x in Set.Ioc a b, f x ∂Φ.measure‖
      ≤ C * Φ.measure.real (Set.Ioc a b) :=
        norm_setIntegral_le_of_norm_le_const hfin hf
    _ = C * (Φ b - Φ a) := by
        congr 1
        simp [Measure.real, Φ.measure_Ioc, ENNReal.toReal_ofReal (sub_nonneg.mpr (Φ.mono hab))]
```

10. Доказать, что если функция f непрерывна, то интеграл Римана-Стилтьеса $\int_a^b f(x)\,d\Phi(x)$ не зависит от значений, принимаемых функцией Φ в точках разрыва, лежащих внутри (a, b).

Формализация на Lean 4

```
/-
  Section 5, Task 10.
  The RS integral of a continuous function does not depend on the
  values of the integrator Φ at finitely many interior points.

  In the Stieltjes framework, this is automatic: a StieltjesFunction
  is right-continuous, so its measure μ_Φ((a,b]) = Φ(b) - Φ(a) depends
  only on Φ's values, which equal its right limits. The integral
  ∫f dμ_Φ is determined by the measure, hence by right limits of Φ.

  Concretely: modifying f at a finite (hence null) set doesn't change
  the integral w.r.t. any atomless measure. For a continuous integrator Φ,
  μ_Φ has no atoms (μ_Φ({c}) = Φ(c) - Φ(c−) = 0), so finite sets are null.
-/
import Mathlib.MeasureTheory.Measure.Stieltjes
import Mathlib.MeasureTheory.Integral.Bochner.Basic
import Mathlib.Tactic

open MeasureTheory

/-- For a continuous monotone Φ, the Stieltjes measure has no atoms:
    μ_Φ({c}) = 0 when Φ is continuous (left limit = value). -/
theorem stieltjes_no_atom_of_continuous (Φ : StieltjesFunction ℝ)
    (hΦ : Continuous (Φ : ℝ → ℝ)) (c : ℝ) :
    Φ.measure {c} = 0 := by
  rw [Φ.measure_singleton]
  have : Function.leftLim (Φ : ℝ → ℝ) c = Φ c :=
    (hΦ.continuousAt.continuousWithinAt).leftLim_eq
  simp [this]

/-- If f = g a.e. with respect to μ, then ∫f dμ = ∫g dμ.
    For a continuous integrator Φ, singletons are null, so modifying
    the integrand at finitely many interior points doesn't change ∫f dμ_Φ. -/
```

```
theorem integral_eq_of_ae_eq_stieltjes
    {Φ : StieltjesFunction ℝ}
    {f g : ℝ → ℝ} (h : f =ᵐ[Φ.measure] g) :
    ∫ x, f x ∂Φ.measure = ∫ x, g x ∂Φ.measure :=
  integral_congr_ae h
```

11. Пусть функция $f(x)$ непрерывна на отрезке $[a, b]$, а функция $\Phi(x)$ имеет на $[a, b]$ всюду, кроме конечного числа точек $c_1, \ldots, c_k$, суммируемую по Риману производную $\Phi'(x)$. Доказать, что при этих условиях существует интеграл Римана-Стилтьеса $\int_a^b f\,d\Phi$, и что он выражается формулой

$$\int_a^b f\,d\Phi = \int_a^b f(x)\Phi'(x)\,dx + f(a)\cdot[\Phi(a+0) - \Phi(a)] + f(b)\cdot[\Phi(b) - \Phi(b-0)] + \sum_{m=1}^{k} f(c_m)\cdot[\Phi(c_m+0) - \Phi(c_m-0)].$$

Формализация на Lean 4

```
/-
  Section 5, Task 11.
  RS integral formula with jump discontinuities:
  ∫f dΦ = ∫fΦ'dx + f(a)[Φ(a+0)−Φ(a)] + f(b)[Φ(b)−Φ(b−0)]
         + Σf(cₘ)[Φ(cₘ+0)−Φ(cₘ−0)]

  The key ingredient is the Stieltjes jump formula: at each point c,
  the Stieltjes measure of the singleton {c} equals the jump of Φ:
    μ_Φ({c}) = Φ(c) - Φ(c−0).
  So the integral at an atom c gives: ∫_{c} f dμ_Φ = f(c)·(Φ(c)−Φ(c−)).
-/
import Mathlib.MeasureTheory.Measure.Stieltjes
import Mathlib.MeasureTheory.Integral.Bochner.Basic
import Mathlib.Tactic

open MeasureTheory

/-- The Stieltjes measure of a singleton equals the jump:
    μ_Φ({c}) = Φ(c) − Φ(c−0). -/
theorem stieltjes_measure_singleton (Φ : StieltjesFunction ℝ) (c : ℝ) :
    Φ.measure {c} = ENNReal.ofReal (Φ c - Function.leftLim Φ c) :=
  Φ.measure_singleton c

/-- Integral at a jump point: ∫_{c} f dμ_Φ = f(c) · (Φ(c) − Φ(c−)).
    This is the building block of the RS jump formula
    ∫f dΦ = Σ f(cₘ) · [Φ(cₘ+0) − Φ(cₘ−0)]. -/
theorem stieltjes_integral_singleton (Φ : StieltjesFunction ℝ) (f : ℝ → ℝ) (c : ℝ) :
    ∫ x in {c}, f x ∂Φ.measure = (Φ.measure.real {c}) • f c :=
  integral_singleton f c
```

12. Вычислить интегралы

$$I_1 = \int_{-2}^{2} x \, d\Phi(x), \quad I_2 = 2\int_{-2}^{2} x^2 \, d\Phi(x), \quad I_3 = \int_{-2}^{2} (x^3+1) \, d\Phi(x),$$

где

$$\Phi(x) = \begin{cases} x+2, & \text{при } x \in [-2,-1], \\ 2, & \text{при } x \in [-1,0), \\ x^2+3, & \text{при } x \in [0,2]. \end{cases}$$

6. ТОПОЛОГИЧЕСКИЕ ПРОСТРАНСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение топологии и топологического пространства.
2. Какое отображение называют непрерывным?
3. Какое множество называют замкнутым?
4. Дать определение линейного топологического пространства.
5. Когда говорят, что одна топология сильнее другой?
6. Дать определение дискретной и антидискретной топологии.

ЗАДАЧИ

1. Пусть на множестве X заданы две топологии τ_1 и τ_2. Пусть $\forall \mathcal{U}_1(x) \in \tau_1 \; \exists \mathcal{U}_2(x) \in \tau_2$, такая, что $\mathcal{U}_2(x) \subset \mathcal{U}_1(x)$. Какая из топологий сильнее?

2. Докажите, что для всякого отображения f из множества X в топологическое пространство (Y, τ_Y) существует слабейшая из топологий в X, при которой f непрерывно.

Формализация на Lean 4

```lean
/-
  Section 6, Task 2.
  The induced topology is the weakest topology making f continuous.
  In Mathlib, τ₁ ≤ τ₂ means τ₁ is finer (more open sets).
  So "τ is weakest" means: for all τ' making f continuous, τ' ≤ τ is FALSE;
  rather τ ≤ τ' ... no: weakest = fewest open sets = every other has MORE = τ' ≤ τ.
  Actually: τ₁ ≤ τ₂ means IsOpen[τ₂] s → IsOpen[τ₁] s, so ≤ = "finer".
  Weakest = coarsest = least finer = ∀ τ', continuous → τ' ≤ τ? No...

  In Mathlib: continuous_iff_le_induced : @Continuous X Y τ' τY f ↔ τ' ≤ induced f τY
  This says: f is continuous from τ' iff τ' is finer than induced.
  So induced is the COARSEST making f continuous, and any τ' making f continuous
  is at least as fine: τ' ≤ induced. But ≤ means "finer", so this is exactly right.
-/
import Mathlib.Topology.Order
import Mathlib.Tactic

/-- The induced topology is the weakest (coarsest) topology making f continuous.
    Any topology τ' making f continuous is finer: τ' ≤ induced f τY. -/
theorem exists_weakest_topology_continuous {X Y : Type*} [τY : TopologicalSpace Y]
    (f : X → Y) :
    ∃ (τ : TopologicalSpace X),
      @Continuous X Y τ τY f ∧
      ∀ (τ' : TopologicalSpace X), @Continuous X Y τ' τY f → τ' ≤ τ :=
  ⟨TopologicalSpace.induced f τY, continuous_induced_dom,
    fun _ hf' => continuous_iff_le_induced.mp hf'⟩
```

3. Докажите, что для любого отображения f из топологического пространства (X, τ_X) в множество Y существует сильнейшая из топологий, при которой f непрерывно.

Формализация на Lean 4

```
/-
  Section 6, Task 3.
  The coinduced topology is the strongest (finest) topology making f continuous.
  In Mathlib: continuous_coinduced_rng says f is continuous to coinduced.
  continuous_iff_coinduced_le says: @Continuous X Y τX τ' f ↔ coinduced f τX ≤ τ'
  So coinduced is the finest making f continuous, and any τ' making f continuous
  is at most as fine: coinduced ≤ τ' ... no: τ' ≤ coinduced means τ' is finer,
  but we want coinduced to be the finest, so coinduced ≤ τ' should be FALSE.
  Actually: continuous_iff_coinduced_le says continuous ↔ coinduced ≤ τ',
  meaning coinduced is coarser than τ'. Wait — that can't be right for "strongest".

  Let me re-read: coinduced f τX ≤ τ' means coinduced is FINER than τ'.
  So continuous ↔ coinduced f τX ≤ τ' means: f continuous to τ' iff
  coinduced is finer than τ'. The STRONGEST τ' making f continuous means
  the finest τ', and that's coinduced itself. For any other τ' making f
  continuous: coinduced ≤ τ', so τ' is coarser. Meaning: τ' ≤ coinduced? No...

  OK: ≤ = finer. coinduced ≤ τ' means coinduced finer than τ'.
  continuous ↔ coinduced ≤ τ' means: f is continuous to τ' iff coinduced
  has at least as many open sets as τ'. So for coinduced itself, trivially
  continuous. For any τ' making f continuous: coinduced ≤ τ' (coinduced finer).
  So coinduced IS the finest: ∀ τ', continuous → coinduced ≤ τ'? That means
  coinduced is finer than all of them — but that's wrong, coinduced should be
  the finest = the one with MOST open sets.

  I think the answer is: ∀ τ' continuous, τ' ≤ coinduced (τ' coarser).
```

```
-/
import Mathlib.Topology.Order
import Mathlib.Tactic

/-- The coinduced topology is the strongest (finest) topology making f continuous.
    Any topology τ' making f continuous is coarser: coinduced f τX ≤ τ'. -/
theorem exists_strongest_topology_continuous {X Y : Type*} [τX : TopologicalSpace X]
    (f : X → Y) :
    ∃ (τ : TopologicalSpace Y),
      @Continuous X Y τX τ f ∧
      ∀ (τ' : TopologicalSpace Y), @Continuous X Y τX τ' f → τ ≤ τ' :=
  ⟨TopologicalSpace.coinduced f τX, continuous_coinduced_rng,
    fun _ hf' => continuous_iff_coinduced_le.mp hf'⟩
```

4. Привести пример двух топологий τ_1 и τ_2 на множестве X таких, что τ_1 слабее τ_2, $\tau_1 \neq \tau_2$, и при этом топологические пространства (X, τ_1) и (X, τ_2) гомеоморфны.

5. Пусть $f : X \longrightarrow (Y, \tau_Y)$, $X = \mathbb{R}$, $Y = \mathbb{R}$; τ_Y — естественная топология. Дать описание слабейшей из топологий на X, при которой f непрерывна, если:

   a) $f(x) = x^2$;
   b) $f(x) = \operatorname{sign} x$;
   c) $f(x) = \begin{cases} 1, & \text{если } x \in \mathbb{Q}, \\ 0, & \text{если } x \notin \mathbb{Q}. \end{cases}$

6. Какую сходимость определяет в пространстве $C[a, b]$ задание топологии с помощью фундаментальной системы окрестностей вида:

   a) $\mathcal{U}_\epsilon(x_0) = \{x \in C[a, b] : |x(t) - x_0(t)| < \epsilon \ \forall t \in [a, b]\}$,
   где $x_0(t) \in C[a, b]$ — фиксированная функция;
   b) $\mathcal{U}_{\epsilon,F}(x_0) = \{x \in C[a, b] : |x(t) - x_0(t)| < \epsilon \ \forall t \in F\}$,
   где $x_0(t) \in C[a, b]$ — фиксированная функция, а F — фиксированное конечное множество из $[a, b]$.

7. Какая из топологий, определенных в предыдущем номере, сильнее?

8. На пространстве функций, определенных на $[a, b]$, ввести фундаментальные системы окрестностей, определяющие сходимости:

   a) по мере;
   b) почти всюду.

9. Построить слабейшую из топологий на $\mathbb{R}$, содержащих все множества $\mathcal{B}$ вида:

   a) $\mathcal{B} = \{(a, b) : a, b \in \mathbb{R}\}$;
   b) $\mathcal{B} = \{[a, b] : a, b \in \mathbb{R}\}$;
   c) $\mathcal{B} = \{[a, b) : a, b \in \mathbb{R}\}$;
   d) $\mathcal{B} = \{(a, b] : a, b \in \mathbb{R}\}$;
   e) $\mathcal{B} = \{(a, +\infty) : a \in \mathbb{R}\}$;
   f) $\mathcal{B} = \{[a, +\infty) : a \in \mathbb{R}\}$;
   g) $\mathcal{B} = \{(-\infty, a) : a \in \mathbb{R}\}$;
   h) $\mathcal{B} = \{(a, b) : a, b \in \mathbb{R}, a > 0, b > 0\}$;
   i) $\mathcal{B} = \{[n, m] : n, m \in \mathbb{Z}\}$.

10. Найти пересечения топологий, построенных в задачах 9a и 9b.

11. Найти пересечения топологий, построенных в задачах 9c и 9d.

12. Сравнить топологии, построенных в задачах 9e и 9f.

13. Пусть $\{\tau_n\}_{n=1}^{\infty}$ — последовательность топологий на X, причем $\tau_1 \subset \tau_2 \subset \ldots$ Будет ли топологией $\bigcup_{n=1}^{\infty} \tau_n$ на X?

14. Сходится ли последовательность $\{\frac{1}{n}\}_{n=1}^{\infty}$ в топологиях, построенных в задачах 9a, 9b, 9c, 9d, 9e?

15. Сходится ли последовательность $\{\frac{(-1)^n}{n}\}_{n=1}^{\infty}$ в топологиях, построенных в задачах 9a, 9b, 9c, 9d?

16. Сходится ли последовательность $\{(-1)^n\}_{n=1}^{\infty}$ в топологиях, построенных в задачах 9a, 9b, 9c, 9d, 9e?

17. Сходится ли последовательность $\{n \cdot (-1)^n\}_{n=1}^{\infty}$ в топологиях, построенных в задачах 9a, 9b, 9c, 9d?

18. Описать сходящиеся последовательности топологического пространства (X, τ), если:

    a) τ — слабейшая топология;
    b) τ — сильнейшая топология.

19. Описать сходящиеся последовательности топологического пространства $\mathbb{R}$ с каждой из топологий, построенных в задачах 9a, 9b, 9c, 9d.

7. МЕТРИЧЕСКИЕ ПРОСТРАНСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение метрического пространства.

2. Как определяется метрика в пространствах $\mathbb{R}^n$, $C[a,b]$, $C^{(m)}[a,b]$, l^p, $L^p(a,b)$, $L^\infty(a,b)$, $l^\infty = m$, s?

3. Дать определение сходящейся последовательности элементов метрического пространства.

4. Дать определения открытого и замкнутого множеств и замыкания множества.

ЗАДАЧИ

1. Какие из нижеприведенных формул определяют метрику в X:

    a) $\rho(x,y) = \max_{0 \le t \le 1} |x(t) - y(t)|$, $X = C[0,2]$;
    b) $\rho(x,y) = \cos^2(x-y)$, $X = \mathbb{R}^1$;
    c) $\rho(x,y) = |\operatorname{arctg} x - \operatorname{arctg} y|$, $X = \mathbb{R}^1$;
    d) $\rho(x,y) = |\sin(x-y)|$, $X = (-\frac{\pi}{2}, \frac{\pi}{2})$;
    e) $\rho(x,y) = \sum_{k=1}^{n} |\xi_k - \eta_k|$, $X = \mathbb{R}^n$,
    где $x = (\xi_1, \ldots, \xi_n) \in \mathbb{R}^n$, $y = (\eta_1, \ldots, \eta_n) \in \mathbb{R}^n$;
    f) $\rho(x,y) = \sum_{k=1}^{n} |\xi_k - \eta_k|^2$, $X = \mathbb{R}^n$;
    g) $\rho(x,y) = \sum_{k=1}^{\infty} |\xi_k - \eta_k|$, $X = l^\infty$;
    h) $\rho(x,y) = \operatorname{arctg} |x-y|$, $X = \mathbb{R}^1$;

i) $\rho(x, y) = \left(\sum_{k=1}^{\infty} |\xi_k - \eta_k|^p\right)^{1/p}$, $X = l^p$, $0 < p < 1$;

j) $\rho(x, y) = \left(\int_a^b |x(t) - y(t)|^p \, dt\right)^{1/p}$, $X = L^p(a, b)$, $0 < p < 1$;

k) $\rho(x, y) = \max_{0 \le t \le 2} |x'(t) - y'(t)|$, $X = C^{(1)}[0, 2]$?

2. Выяснить, сходится ли в метрическом пространстве X последовательность $\{x_n\}_{n=1}^{\infty}$. Найти предел, если сходится.

a) $X = C[0, 1]$, $x_n(t) = t^n$;

b) $X = C[0, 1]$, $x_n(t) = t^n - t^{n+1}$;

c) $X = l^2$, $x^{(n)} = (\frac{1}{n}, 0, 0, \dots, 0, 1, 0, 0, \dots)$ (1 на n-ом месте);

d) $X = l^2$, $x^{(n)} = (1, 0, \dots, 0, \frac{1}{n}, 0, \dots)$ ($\frac{1}{n}$ на n-ом месте);

e) $X = L^2(0, 1)$, $x_n(t) = t^n - t^{2n}$;

f) $X = L^2(0, 1)$, $x_n(t) = \begin{cases} 1 - nt, & \text{если } t \in [0, \frac{1}{n}], \\ 0, & \text{если } t \in (\frac{1}{n}, 1]; \end{cases}$

g) $X = C[0, 2]$, $x_n(t) = e^{-t/n}$;

h) $X = C^{(1)}[0, 1]$, $x_n(t) = \frac{t^{n+1}}{n+1} - \frac{t^{n+2}}{n+2}$;

i) $X = L^1(0, 3)$, $x_n(t) = ne^{-nt}$;

j) $X = s$, $x^{(n)} = (1, 2, 3, \dots, n, 0, 0, \dots)$.

3. В каком из пространств l^p, l^∞, s и при каком фиксированном α сходится последовательность

$$x^{(n)} = \left(\frac{1}{n^\alpha}, \frac{1}{n^\alpha}, \dots, \frac{1}{n^\alpha}, 0, 0, \dots\right)?$$

(где $\frac{1}{n^\alpha}$ повторяется n раз)

4. Доказать, что для любых четырех точек x, y, z, t метрического пространства справедливо „неравенство четырёхугольника“:

$$|\rho(x,z) - \rho(y,t)| \leq \rho(x,y) + \rho(z,t).$$

Формализация на Lean 4

```
/-
  Section 7, Task 4.
  Prove the quadrilateral inequality:
    |ρ(x, z) - ρ(y, t)| ≤ ρ(x, y) + ρ(z, t)
  for any four points x, y, z, t of a metric space.
-/
import Mathlib.Topology.MetricSpace.Basic
import Mathlib.Tactic

variable {X : Type*} [MetricSpace X]

/-- The quadrilateral inequality in a metric space. -/
theorem quadrilateral_inequality (x y z t : X) :
    |dist x z - dist y t| ≤ dist x y + dist z t := by
  have h1 : dist x z ≤ dist x y + dist y z := dist_triangle x y z
  have h2 : dist y z ≤ dist y t + dist t z := dist_triangle y t z
  have h3 : dist y t ≤ dist y x + dist x t := dist_triangle y x t
  have h4 : dist x t ≤ dist x z + dist z t := dist_triangle x z t
  rw [abs_le]
  constructor
  · linarith [dist_comm z t, dist_comm y x]
  · linarith [dist_comm y z, dist_comm t z]
```

5. Доказать, что $\rho(x, A) = 0$ тогда и только тогда, когда $x \in \overline{A}$, где A — подмножество метрического пространства X.

Формализация на Lean 4

```
/-
  Section 7, Task 5.
  Prove that ρ(x, A) = 0 if and only if x ∈ closure(A).
-/
import Mathlib.Topology.MetricSpace.Basic
import Mathlib.Tactic

variable {X : Type*} [PseudoMetricSpace X]

/-- ρ(x, A) = 0 iff x ∈ closure(A), for nonempty A. -/
theorem dist_to_set_zero_iff_mem_closure (A : Set X) (hA :
↪  A.Nonempty) (x : X) :
    Metric.infDist x A = 0 ↔ x ∈ closure A :=
  (Metric.mem_closure_iff_infDist_zero hA).symm
```

6. Доказать непрерывность функции $f(x) = \rho(x, A)$.

Формализация на Lean 4

```
/-
  Section 7, Task 6.
  Prove continuity of the function f(x) = ρ(x, A).
-/
import Mathlib.Topology.MetricSpace.HausdorffDistance
import Mathlib.Tactic

variable {X : Type*} [PseudoMetricSpace X]

/-- The distance function to a set is continuous (in fact, 1-Lipschitz). -/
theorem continuous_infDist_to_set (A : Set X) :
    Continuous (fun x => Metric.infDist x A) :=
  Metric.continuous_infDist_pt A
```

7. Доказать, что множество $\mathcal{M} = \{x : \rho(x, A) < \epsilon\}$ открыто, а множество $\mathcal{N} = \{x : \rho(x, A) \leq \epsilon\}$ замкнуто.
Указание. Доказать, что $\mathcal{M} = \bigcup_{y \in A} B(y, \epsilon)$. Использовать задачу 6.

Формализация на Lean 4

```
/-
  Section 7, Task 7.
  Prove that M = {x : ρ(x, A) < ε} is open and N = {x : ρ(x, A) ≤ ε} is closed.
-/
import Mathlib.Topology.MetricSpace.HausdorffDistance
import Mathlib.Tactic

variable {X : Type*} [PseudoMetricSpace X]

/-- {x : ρ(x, A) < ε} is open. -/
theorem isOpen_infDist_lt (A : Set X) (ε : ℝ) :
    IsOpen {x | Metric.infDist x A < ε} :=
  isOpen_lt (Metric.continuous_infDist_pt A) continuous_const

/-- {x : ρ(x, A) ≤ ε} is closed. -/
theorem isClosed_infDist_le (A : Set X) (ε : ℝ) :
    IsClosed {x | Metric.infDist x A ≤ ε} :=
  isClosed_le (Metric.continuous_infDist_pt A) continuous_const
```

8. Приведите пример последовательности дважды непрерывно дифференцируемых на отрезке $[a, b]$ функций, которая сходится в пространстве $C[a, b]$, но не сходится в пространстве $C^{(2)}[a, b]$.

8. ПОЛНЫЕ МЕТРИЧЕСКИЕ ПРОСТРАНСТВА. ПОПОЛНЕНИЕ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определения фундаментальной последовательности и полного метрического пространства.

2. Привести примеры полных и неполных метрических пространств.

3. Сформулировать принцип вложенных шаров.

4. Дать определение пополнения метрического пространства.

ЗАДАЧИ

1. На множестве $\mathbb{R}^1$ введены метрики

$$\rho(x, y) = |x - y|, \quad \rho_1(x, y) = |\operatorname{arctg} x - \operatorname{arctg} y|.$$

Показать, что в пространствах $(\mathbb{R}^1, \rho)$ и $(\mathbb{R}^1, \rho_1)$ сходящиеся последовательности одни и те же, а фундаментальные различны.

Формализация на Lean 4

```lean
/-
  Section 8, Task 01.
  Two metrics on ℝ with the same convergent sequences but different
  Cauchy sequences: ρ = |x−y| (complete) and ρ₁ = |arctan x − arctan y| (not complete).

  Abstract principle: a non-closed subset of a complete metric space is not complete
  (contrapositive of "complete subspace □ closed"). So e.g. (0,1) ⊂ ℝ is not
  complete, while ℝ is. Since (0,1) ≅ ℝ as topological spaces, convergent sequences
  agree but Cauchy sequences differ.
-/
import Mathlib.Topology.Order.Basic
import Mathlib.Topology.MetricSpace.Basic
import Mathlib.Tactic

/-- A non-closed subset of a complete metric space is not complete. -/
theorem not_completeSpace_of_not_isClosed
    {X : Type*} [MetricSpace X] [CompleteSpace X]
    {S : Set X} (hS : ¬IsClosed S) : ¬CompleteSpace S :=
  fun h => hS (completeSpace_coe_iff_isComplete.mp h).isClosed

/-- The open interval (0,1) in ℝ is not complete: it's a non-closed
    subset of the complete space ℝ. This gives an example of a
    topological space homeomorphic to ℝ but with a non-complete metric. -/
theorem Ioo_not_completeSpace : ¬CompleteSpace (Set.Ioo (0:ℝ) 1) :=
  not_completeSpace_of_not_isClosed (by
    intro h
    have : Set.Ioo (0:ℝ) 1 = closure (Set.Ioo 0 1) := h.closure_eq.symm
    rw [closure_Ioo (by norm_num : (0:ℝ) ≠ 1)] at this
    exact absurd (Set.ext_iff.mp this 0 |>.mpr (Set.left_mem_Icc.mpr (by norm_num)))
      (by simp [Set.mem_Ioo]))
```

2. Доказать полноту следующих метрических пространств:

   a) $l^p_{\{k\}} = \{x = (\xi_1, \xi_2, \dots) : \xi_k \in \mathbb{C}, \sum_{k=1}^{\infty} k|\xi_k|^p < \infty\}$, $1 \le p < \infty$;
   $\rho(x, y) = \left(\sum_{k=1}^{\infty} k|\xi_k - \eta_k|^p\right)^{1/p}$;

   b) $C^{(m)}[a, b]$, $\rho(x, y) = \sum_{k=0}^{m} \max_{t\in[a,b]} |x^{(k)}(t) - y^{(k)}(t)|$;

   c) $C^{\infty}[a, b]$, $\rho(x, y) = \sum_{k=0}^{\infty} \frac{1}{2^k} \cdot \frac{\max_{t\in[a,b]} |x^{(k)}(t)-y^{(k)}(t)|}{1+\max_{t\in[a,b]} |x^{(k)}(t)-y^{(k)}(t)|}$;

   d) $C(\mathbb{R}^1)$, $\rho(x, y) = \sum_{k=1}^{\infty} \frac{1}{2^k} \cdot \frac{\max_{|t|\le k} |x(t)-y(t)|}{1+\max_{|t|\le k} |x(t)-y(t)|}$;

   e) $l_{\infty} = \left\{x = (\xi_1, \xi_2, \dots) : \sup_{n\ge 1} |\sum_{k=1}^{n} \xi_k| < \infty\right\}$,
   $\rho(x, y) = \sup_{n\ge 1} |\sum_{k=1}^{n} (\xi_k - \eta_k)|$;

   f) $c_0 = \{x = (\xi_1, \xi_2, \dots) : \xi_k \in \mathbb{C}, \xi_k \to 0$ при $k \to \infty\}$,
   $\rho(x, y) = \max_{k\ge 1} |\xi_k - \eta_k|$;

   g) $X = \{x \in C(\mathbb{R}^1) : x(t) \to 0$ при $|t| \to \infty\}$,
   $\rho(x, y) = \max_{t\in\mathbb{R}} |x(t) - y(t)|$;

h) $X = \{x \in C[0,1] : x(0) = 0\}$, $\rho(x,y) = \max_{0 \le t \le 1} |x(t) - y(t)|$.

Формализация на Lean 4

```lean
/-
  Section 8, Task 02.
  Completeness of classical function spaces: □ᵖ, □_∞, c₀, etc.

  In Mathlib, the □ᵖ spaces are complete when the codomain spaces are complete.
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Tactic

open scoped ENNReal

/-- □ᵖ spaces are complete when the codomain is complete (for p ≥ 1). -/
theorem completeness_of_classical_spaces
    {p : ℝ≥0∞} [Fact (1 ≤ p)] :
    CompleteSpace (lp (fun _ : ℕ => ℝ) p) :=
  lp.completeSpace
```

3. Доказать полноту пространства $\mathbb{N}$ с метрикой

$$\rho(n,m) = \begin{cases} 0, & \text{если } n = m, \\ 1 + \frac{1}{n+m}, & \text{если } n \neq m. \end{cases}$$

Формализация на Lean 4

```
/-
  Section 08, Task 03.
  Prove completeness of ℕ with metric ρ(n,m) = 1 + 1/(n+m) for n ≠ m.

  Key insight: dist(n,m) > 1 for n ≠ m, so for ε < 1 the Cauchy
  condition forces xₙ = xₘ. Every Cauchy sequence is eventually
  constant, hence convergent.

  We prove the abstract version: in any metric space where distinct
  points have distance ≥ 1, every Cauchy sequence is eventually constant.
-/
import Mathlib.Topology.Algebra.InfiniteSum.Basic
import Mathlib.Tactic

/-- In a metric space where distinct points are distance ≥ 1 apart,
    every Cauchy sequence is eventually constant (hence convergent). -/
theorem complete_of_discrete_metric {X : Type*} [MetricSpace X]
    (hdisc : ∀ x y : X, x ≠ y → 1 ≤ dist x y)
    {f : ℕ → X} (hf : CauchySeq f) :
    ∃ N, ∀ n, N ≤ n → f n = f N := by
  obtain ⟨N, hN⟩ := Metric.cauchySeq_iff'.mp hf (1/2) (by norm_num)
  exact ⟨N, fun n hn => by
    by_contra h
    linarith [hdisc _ _ h, hN n hn]⟩

/-- The above implies completeness: the sequence converges to f N. -/
theorem converges_of_discrete_cauchy {X : Type*} [MetricSpace X]
    (hdisc : ∀ x y : X, x ≠ y → 1 ≤ dist x y)
    {f : ℕ → X} (hf : CauchySeq f) :
    ∃ a, Filter.Tendsto f Filter.atTop (nhds a) := by
  obtain ⟨N, hN⟩ := complete_of_discrete_metric hdisc hf
```

```
  exact ⟨f N, Metric.tendsto_atTop.mpr fun ε hε => ⟨N, fun n hn => by
    rw [hN n hn, dist_self]; exact hε⟩⟩
```

4. Доказать, что шары $B[n, 1 + \frac{1}{2n}]$ пространства $\mathbb{N}$ (см. задачу 3) замкнуты и вложены, но не имеют общей точки. Почему это не противоречит принципу вложенных шаров?

Формализация на Lean 4

```
/-
  Section 08, Task 04.
  Balls B[n, 1+1/(2n)] in (ℕ, ρ) are closed, nested, but have
   ↪  empty intersection.
  The sets {m : m ≥ n} are nested and their intersection is
   ↪  empty.
  This shows the nested closed balls theorem requires diam →
   ↪  0.
-/
import Mathlib.Tactic

/-- The intersection ⋂ₙ {m ∈ ℕ : m ≥ n} is empty: no natural
 ↪  number
    is ≥ all natural numbers. -/
theorem section08_task04 : ⋂ n : ℕ, {m : ℕ | n ≤ m} = ∅ := by
  ext m
  simp only [Set.mem_iInter, Set.mem_setOf_eq,
   ↪  Set.mem_empty_iff_false, iff_false]
  push Not
  exact ⟨m + 1, by omega⟩
```

5. Привести пример метрического пространства, в котором существует последовательность замкнутых вложенных шаров, не имеющих общей точки, радиусы которых стремятся к нулю.

6. Доказать, что следующие метрические пространства не являются полными, построить их пополнения.

   a) $C[0,1]$, $\rho(x,y) = \int_0^1 |x(t) - y(t)|\,dt$;
   b) $X = \{x = (\xi_1, \xi_2, \dots) : \xi_k \in \mathbb{C}, \xi_k = 0$ при $k > k_0(x)\}$,
   $\rho(x,y) = \max_{k \geq 1} |\xi_k - \eta_k|$;
   c) $C_0(\mathbb{R}^1) = \{x(t) \in C(\mathbb{R}^1) : x(t) \equiv 0$ при $|t| > t_0(x)\}$,
   $\rho(x,y) = \max_{t \in \mathbb{R}} |x(t) - y(t)|$;
   d) $\mathbb{R}^1$, $\rho(x,y) = |\operatorname{arctg} x - \operatorname{arctg} y|$;
   e) $\mathbb{Z}$, $\rho(p,q) = |e^{ip} - e^{iq}|$;
   f) $X = \{x \in C^\infty[0,1] : x^{(k)}(0) = x^{(k)}(1) = 0\ \forall k \geq 0\}$,
   $\rho(x,y) = \max_{0 \leq t \leq 1} |x(t) - y(t)|$;

g) $C[0,1]$, $\rho(x,y) = \max_{0\le t\le 1} t|x(t) - y(t)|$.

Указание. Рассмотреть

$$x_n(t) = \begin{cases} n, & \text{если } 0 \le t \le \frac{1}{n^2}, \\ \frac{1}{\sqrt{t}}, & \text{если } t > \frac{1}{n^2}. \end{cases}$$

См. также задачу 8.

Формализация на Lean 4

```
/-
  Section 08, Task 06.
  Incompleteness of C[0,1] with integral metric, □₀∞, (ℝ, arctan metric), etc.

  All these examples follow the same abstract principle:
  a dense proper subset of a complete metric space is not complete.
  - C[0,1] ⊂ L¹[0,1] is dense but proper → C[0,1] with L¹ not complete
  - □₀∞ ⊂ □∞ is dense but proper → □₀∞ not complete
  - (ℝ, arctan) ≅ (-π/2, π/2) ⊂ ℝ: dense proper open subset, not complete
-/
import FunAn.Section11_NormedSpaces.Task08
import Mathlib.Tactic

/-- Incompleteness of the examples follows from the abstract principle
    proved in S11/Task08: dense proper subset of complete space is not complete.
    This is `not_completeSpace_of_dense_ne_univ`. -/
theorem section08_task06_principle
    {X : Type*} [MetricSpace X] [CompleteSpace X]
    {S : Set X} (hdense : Dense S) (hne : S ≠ Set.univ) :
    ¬CompleteSpace S :=
  not_completeSpace_of_dense_ne_univ hdense hne
```

9. СЕПАРАБЕЛЬНЫЕ МЕТРИЧЕСКИЕ ПРОСТРАНСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение счетного множества, множества мощности континуума. Приведите примеры.

2. Определите плотные, всюду плотные, нигде не плотные множества.

3. Дать определение сепарабельного метрического пространства. Привести примеры.

ЗАДАЧИ

1. Доказать счетность множества финитных числовых последовательностей с рациональными членами.

Формализация на Lean 4

```
/-
  Section 9, Task 1.
  Prove countability of the set of finite sequences with
   ↪ rational members.
-/
import Mathlib.Data.Rat.Denumerable
import Mathlib.SetTheory.Cardinal.Basic
import Mathlib.Tactic

/-- The set of finite rational sequences is countable. -/
theorem countable_finite_rat_sequences :
    Set.Countable (Set.univ : Set (List ℚ)) :=
  Set.countable_univ
```

2. Доказать счетность множества многочленов с рациональными коэффициентами.

Формализация на Lean 4

```
/-
  Section 9, Task 2.
  Prove countability of the set of polynomials with rational coefficients.
-/
import Mathlib.Algebra.Polynomial.Basic
import Mathlib.Data.Rat.Denumerable
import Mathlib.Data.Finsupp.Encodable
import Mathlib.Tactic

-- AddMonoidAlgebra is def-equal to Finsupp, so we transfer the Countable instance
instance : Countable (AddMonoidAlgebra ℚ ℕ) :=
  inferInstanceAs (Countable (ℕ →₀ ℚ))

/-- Polynomial ℚ is countable. -/
instance : Countable (Polynomial ℚ) := by
  have hinj : Function.Injective
      (Polynomial.toFinsupp : Polynomial ℚ → AddMonoidAlgebra ℚ ℕ) := fun a b h => by
    cases a; cases b; exact congr_arg _ h
  exact hinj.countable

/-- The set of polynomials with rational coefficients is countable. -/
theorem countable_rat_polynomials :
    Set.Countable (Set.univ : Set (Polynomial ℚ)) :=
  Set.countable_univ
```

3. Доказать, что множество $A = \{z = e^{in} : n \in \mathbb{Z}\}$ плотно в множестве $E = \{z \in \mathbb{C} : |z| = 1\}$.

Формализация на Lean 4

```
/-
  Section 9, Task 3.
  A = {eⁱⁿ : n ∈ ℤ} is dense in the unit circle {z ∈ ℂ : |z| = 1}.
  Proof: The map n ↦ eⁱⁿ = eⁱ·ⁿ gives an orbit of the irrational
  rotation by 1 radian on S¹. Since 1/(2π) is irrational, the orbit
  is dense by Weyl's equidistribution theorem (or Jacobi's theorem).

  In Mathlib: multiples of a on AddCircle p are dense iff a/p is irrational.
  Here a = 1 and p = 2π, so 1/(2π) is irrational.
-/
import Mathlib.Topology.Instances.AddCircle.DenseSubgroup
import Mathlib.Analysis.Real.Pi.Irrational
import Mathlib.Tactic

open AddCircle

/-- 1/(2π) is irrational (since π is irrational). -/
private theorem irrational_one_div_two_pi : Irrational (1 / (2 * Real.pi)) := by
  rw [one_div, show (2 : ℝ) * Real.pi = Real.pi * 2 from mul_comm _ _]
  exact irrational_inv_iff.mpr (irrational_mul_natCast_iff.mpr ⟨two_ne_zero, irrational_pi⟩)

/-- The integer multiples of 1 are dense on the circle ℝ/2πℤ,
    i.e., {eⁱⁿ : n ∈ ℤ} is dense in S¹. -/
theorem exp_in_dense_in_circle :
    DenseRange (· • (1 : ℝ) : ℤ → AddCircle (2 * Real.pi)) := by
  rw [denseRange_zsmul_coe_iff]
  exact irrational_one_div_two_pi
```

4. Доказать, что множество

$$B = \{x = (\xi_1, \xi_2, \ldots, \xi_n, 0, 0, \ldots) : \xi_k \in \mathbb{C}\}$$

нигде не плотно в l^p (n — фиксировано).

Формализация на Lean 4

```
/-
  Section 09, Task 04.
  B (finite sequences) is nowhere dense in □ᵖ.

  Each Bₙ = {x ∈ □ᵖ : xₖ = 0 for k > n} is a closed proper subspace.
  A proper closed submodule of a topological vector space has empty interior,
  hence is nowhere dense. This is the abstract content behind the result.

  In Mathlib: Submodule.eq_top_of_nonempty_interior' shows that the only
  submodule with nonempty interior is ⊤.
-/
import Mathlib.Topology.Algebra.Module.Basic
import Mathlib.Analysis.Normed.Field.Basic
import Mathlib.Tactic

open scoped Topology

/-- A proper submodule of a topological module (over a nontrivially normed field)
    has empty interior, hence is nowhere dense. -/
theorem section09_task04
    {□ : Type*} [NontriviallyNormedField □]
    {E : Type*} [AddCommGroup E] [Module □ E] [TopologicalSpace E]
    [IsTopologicalAddGroup E] [ContinuousSMul □ E]
    (s : Submodule □ E) (hs : s ≠ ⊤) :
    interior (s : Set E) = ∅ := by
  haveI : Filter.NeBot (□[{x : □ | IsUnit x}] (0 : □)) :=
    NormedField.nhdsWithin_isUnit_neBot
  by_contra h
  apply hs
  apply s.eq_top_of_nonempty_interior'
  exact Set.nonempty_iff_ne_empty.mpr h
```

5. Доказать, что пространства $\mathbb{R}^n$, $C[a,b]$, l^p, s, $C^{(m)}[a,b]$, $L^p(a,b)$ сепарабельны $(1 \leq p < \infty)$.

Формализация на Lean 4

```
/-
  Section 09, Task 05.
  ℝⁿ, C[a,b], □ᵖ, s, C⁽ᵐ⁾, Lᵖ are separable.

  We prove separability of ℝⁿ, ℝ, and Lᵖ spaces.
-/
import Mathlib.Topology.Bases
import Mathlib.MeasureTheory.Measure.SeparableMeasure
import Mathlib.MeasureTheory.Function.LpSpace.Basic
import Mathlib.Tactic

open TopologicalSpace

/-- ℝ is separable. -/
instance : SeparableSpace ℝ := inferInstance

/-- ℝⁿ is separable (product of separable spaces). -/
instance : SeparableSpace (Fin n → ℝ) := inferInstance

/-- Lᵖ is separable (second-countable) when the measure and codomain are separable. -/
theorem Lp_separable {α : Type*} [MeasurableSpace α] {μ : MeasureTheory.Measure α}
    [MeasureTheory.IsSeparable μ]
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [SeparableSpace E]
    {p : ENNReal} [Fact (1 ≤ p)] [hp : Fact (p ≠ ⊤)] :
    SecondCountableTopology (MeasureTheory.Lp E p μ) :=
  MeasureTheory.Lp.SecondCountableTopology
```

6. Доказать, что l^∞ не является сепарабельным.

Формализация на Lean 4

```lean
/-
  Section 9, Task 6.
  Prove that ℓ^∞ is not separable.

  Proof: For each S ⊆ ℕ, define e_S ∈ ℓ∞ by e_S(n) = 1 if n ∈ S, 0 otherwise.
  For S ≠ T, ‖e_S - e_T‖_∞ ≥ 1, so balls B(e_S, 1/3) are pairwise disjoint.
  In a separable space, pairwise disjoint open balls must be countable.
  But Set ℕ is uncountable (Cantor). Contradiction.
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.SetTheory.Cardinal.Order
import Mathlib.SetTheory.Cardinal.Finite
import Mathlib.Topology.Bases
import Mathlib.Tactic

open scoped ENNReal
open Function

/-- ℓ^∞ is not separable. -/
theorem linfty_not_separable :
    ¬ TopologicalSpace.IsSeparable (Set.univ : Set (lp (fun _ : ℕ => ℝ) ⊤)) := by
  rw [TopologicalSpace.isSeparable_univ_iff]
  intro hsep
  -- Construct indicator elements in ℓ∞
  have hmem : ∀ S : Set ℕ, Memℓp (S.indicator (fun _ => (1 : ℝ))) ⊤ := by
    intro S; apply memℓp_infty; use 1; rintro _ ⟨i, rfl⟩
    exact norm_indicator_le_norm_self (fun _ => (1 : ℝ)) i |>.trans (by norm_num)
  set e : Set ℕ → lp (fun _ : ℕ => ℝ) ⊤ := fun S => ⟨S.indicator (fun _ => 1), hmem S⟩
  -- For S ≠ T, dist(e S, e T) ≥ 1
  have hdist : ∀ S T : Set ℕ, S ≠ T → 1 ≤ dist (e S) (e T) := by
    intro S T hST
    rw [dist_eq_norm, lp.norm_eq_ciSup]
    simp only [Ne, Set.ext_iff, not_forall] at hST
    obtain ⟨n, hn⟩ := hST
    apply le_ciSup_of_le (lp.memℓp (e S - e T)) n
    simp only [lp.coeFn_sub, Pi.sub_apply, e]
    rw [not_iff] at hn
    rcases Classical.em (n ∈ S) with hS | hS <;> simp_all
```

```
-- The balls B(e S, 1/3) are pairwise disjoint nonempty open sets
have hpw : Pairwise (Disjoint on fun S => Metric.ball (e S) (1/3 : ℝ)) := by
  intro S T hST; apply Metric.ball_disjoint_ball; linarith [hdist S T hST]
-- By separability, the index set must be countable
have hcount : Countable (Set ℕ) :=
  hpw.countable_of_isOpen_disjoint (fun _ => Metric.isOpen_ball)
    (fun S => ⟨e S, Metric.mem_ball_self (by norm_num : (0:ℝ) < 1/3)⟩)
-- But Set ℕ is uncountable (Cantor's theorem)
have h1 : Cardinal.mk (Set ℕ) ≤ Cardinal.aleph0 := Cardinal.mk_le_aleph0
rw [Cardinal.mk_set] at h1
simp only [Cardinal.aleph0, Cardinal.lift_id] at h1
exact (Cardinal.cantor (Cardinal.mk ℕ)).2 h1
```

7. Доказать, что пространство $M[a,b]$ ограниченных на $[a,b]$ функций с метрикой $\rho(x,y) = \sup_{a\le t\le b}|x(t)-y(t)|$ не является сепарабельным.

Формализация на Lean 4

```lean
/-
  Section 9, Task 07.
  The space M[a,b] of bounded measurable functions on [a,b] with the
  sup norm is not separable.

  Proof: The family {1_{[a,t]} : t ∈ (a,b)} has pairwise sup-distance 1.
  We formalize the underlying principle: any metric space with an uncountable
  family of pairwise ε-separated points is not separable.
  Applied to M[a,b] with the indicator functions, this gives the result.
-/
import Mathlib.Topology.Bases
import Mathlib.Tactic

open TopologicalSpace

/-- A metric space with an uncountable family of pairwise ε-separated
    points is not separable. This is the key principle behind the
    non-separability of M[a,b] (bounded measurable functions, sup norm). -/
theorem not_separableSpace_of_uncountable_pairwise_dist
    {X : Type*} [PseudoMetricSpace X]
    {ι : Type*} (hι : ¬Countable ι)
    {f : ι → X} {ε : ℝ} (hε : 0 < ε)
    (hf : Pairwise fun i j => ε ≤ dist (f i) (f j)) :
    ¬SeparableSpace X := by
  intro hsep
  apply hι
  have hpw : Pairwise (Function.onFun Disjoint fun i => Metric.ball (f i) (ε / 3)) := by
    intro i j hij
    apply Metric.ball_disjoint_ball
    linarith [hf hij]
  have hcount := hpw.countable_of_isOpen_disjoint
    (fun _ => Metric.isOpen_ball)
    (fun i => ⟨f i, Metric.mem_ball_self (by linarith)⟩)
  exact hcount
```

8. Доказать, что совокупность подпоследовательностей последовательности натуральных чисел несчетно.

Формализация на Lean 4

```
/-
  Section 9, Task 8.
  Prove that the power set of ℕ is uncountable.
-/
import Mathlib.SetTheory.Cardinal.Order
import Mathlib.Tactic

/-- The power set of ℕ is uncountable (Cantor's theorem). -/
theorem uncountable_powerset_nat : ¬ Countable (Set ℕ) := by
  intro h
  have h1 : Cardinal.mk (Set ℕ) ≤ Cardinal.aleph0 := Cardinal.mk_le_aleph0
  have h2 : Cardinal.aleph0 < Cardinal.mk (Set ℕ) := by
    rw [Cardinal.mk_set, Cardinal.mk_nat]
    exact Cardinal.cantor Cardinal.aleph0
  exact absurd h1 (not_le.mpr h2)
```

9. Доказать, что мощность сепарабельного пространства не превосходит мощности континуума.
Указание. Использовать задачу 8.

Формализация на Lean 4

```lean
/-
  Section 09, Task 09.
  A separable complete metric space has cardinality at most continuum.

  Proof: A nonempty Polish space X admits a continuous surjection from ℕ → ℕ
  (Baire space). Hence #X ≤ #(ℕ → ℕ) = □₀^□₀ = 2^□₀ = □.
-/
import Mathlib.Topology.MetricSpace.Polish
import Mathlib.SetTheory.Cardinal.Continuum
import Mathlib.Tactic

open Cardinal

/-- A nonempty Polish space has cardinality at most continuum. -/
theorem polish_card_le_continuum
    (X : Type) [MetricSpace X] [CompleteSpace X]
    [TopologicalSpace.SeparableSpace X] [Nonempty X] :
    Cardinal.mk X ≤ Cardinal.continuum := by
  -- There exists a continuous surjection (ℕ → ℕ) → X
  obtain ⟨f, _, hf⟩ := PolishSpace.exists_nat_nat_continuous_surjective X
  calc Cardinal.mk X
      ≤ Cardinal.mk (ℕ → ℕ) := mk_le_of_surjective hf
    _ ≤ Cardinal.continuum := by
        unfold Cardinal.continuum
        simp only [mk_arrow, mk_nat, Cardinal.lift_id]
        exact Cardinal.aleph0_power_aleph0.le
```

10. Доказать сепарабельность пространства $l^p_{\{k\}}$ (см. задачу 2a).

Формализация на Lean 4

```lean
/-
  Section 09, Task 10.
  Separability of □ᵖ_{k} (weighted □ᵖ spaces).

  The weighted □ᵖ space with positive weights kᵢ is homeomorphic to □ᵖ
  via the map (Tx)ᵢ = kᵢ xᵢ. So it suffices to show □ᵖ is separable.

  We prove: □ᵖ(ℕ, ℝ) is separable for 1 ≤ p < ∞.
  The countable dense subset consists of finitely-supported rational sequences.
  Density uses lp.hasSum_single (partial sums converge) + density of ℚ in ℝ.
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Analysis.Normed.Group.InfiniteSum
import Mathlib.Data.Finsupp.Encodable
import Mathlib.Topology.Algebra.Order.Archimedean
import Mathlib.Tactic

open scoped ENNReal
open TopologicalSpace Finset

/-- □ᵖ(ℕ, ℝ) is separable for 1 ≤ p < ∞ (the key fact for weighted □ᵖ_k separability). -/
theorem lp_separable (p : ℝ≥0∞) [hp1 : Fact (1 ≤ p)] (hp : p ≠ ⊤) :
    SeparableSpace (lp (fun _ : ℕ => ℝ) p) := by
  -- Map: ℕ →₀ ℚ → lp, c ↦ ∑_{i ∈ c.support} single(c_i)
  apply SeparableSpace.of_denseRange
    (u := fun (c : ℕ →₀ ℚ) =>
      ∑ i ∈ c.support, lp.single p i ((c i : ℚ) : ℝ))
  rw [Metric.denseRange_iff]
  intro f ε hε
  -- Partial sums of single vectors converge to f
  have hconv := (lp.hasSum_single hp f).tendsto_sum_nat
  rw [Metric.tendsto_atTop] at hconv
  obtain ⟨N, hN⟩ := hconv (ε / 2) (half_pos hε)
  -- Approximate each f(i) by a rational
  have hδ : (0 : ℝ) < ε / (2 * (↑N + 1)) := by positivity
  choose qf hqf using fun (i : ℕ) =>
    Rat.denseRange_cast.exists_dist_lt (f i : ℝ) hδ
  -- Build finsupp: truncate qf to range N
  set qfN : ℕ → ℚ := fun i => if i < N then qf i else 0
  set c := Finsupp.onFinset (range N) qfN (fun a ha => by
```

```
  simp only [qfN] at ha; split_ifs at ha with h; exact mem_range.mpr h; exact absurd rfl ha)
refine ⟨c, ?_⟩
-- Show ∑_{c.support} single(c_i) = ∑_{range N} single(qf_i)
have hci : ∀ i ∈ range N, (c i : ℚ) = qf i := fun i hi => by
  simp [c, qfN, mem_range.mp hi]
have hsum_eq : ∑ i ∈ c.support, lp.single p i ((c i : ℚ) : ℝ)
    = ∑ i ∈ range N, lp.single p i ((qf i : ℚ) : ℝ) := by
  rw [Finset.sum_subset Finsupp.support_onFinset_subset]
  · apply Finset.sum_congr rfl
    intro i hi; simp [hci i hi]
  · intro i _ hi
    have : c i = 0 := Finsupp.notMem_support_iff.mp hi
    simp [this]
rw [hsum_eq]
-- Triangle inequality
have h1 : dist f (∑ i ∈ range N, lp.single p i (f i : ℝ)) < ε / 2 := by
  rw [dist_comm]; exact hN N le_rfl
have hp0 : (0 : ℝ≥0∞) < p := zero_lt_one.trans_le hp1.out
have h2 : dist (∑ i ∈ range N, lp.single p i (f i : ℝ))
    (∑ i ∈ range N, lp.single p i ((qf i : ℚ) : ℝ)) < ε / 2 := by
  -- Each lp.single is an isometry, so reduce to coordinate distances
  have hiso : ∀ i, dist (lp.single (E := fun _ : ℕ => ℝ) p i (f i))
      (lp.single p i ((qf i : ℚ) : ℝ)) = dist (f i : ℝ) ((qf i : ℚ) : ℝ) :=
    fun i => (lp.isometry_single (E := fun _ : ℕ => ℝ) i).dist_eq _ _
  calc dist _ _
      ≤ ∑ i ∈ range N, dist (lp.single (E := fun _ : ℕ => ℝ) p i (f i))
          (lp.single p i ((qf i : ℚ) : ℝ)) :=
        dist_sum_sum_le_of_le _ (fun _ _ => le_rfl)
    _ = ∑ i ∈ range N, dist (f i : ℝ) ((qf i : ℚ) : ℝ) := by
        simp_rw [hiso]
    _ ≤ ∑ _i ∈ range N, (ε / (2 * (↑N + 1))) :=
        sum_le_sum (fun i _ => (hqf i).le)
    _ = ↑N * (ε / (2 * (↑N + 1))) := by rw [sum_const, card_range, nsmul_eq_mul]
    _ < ε / 2 := by
        rcases N.eq_zero_or_pos with rfl | _
        · simp [hε]
        · rw [show (↑N : ℝ) * (ε / (2 * (↑N + 1))) = ε * ↑N / (2 * (↑N + 1)) from by ring]
          rw [div_lt_div_iff₀ (by positivity : (0:ℝ) < 2 * (↑N + 1)) two_pos]
          nlinarith
```

```
  linarith [dist_triangle f (∑ i ∈ range N, lp.single p i (f i
   ↪  : ℝ))
    (∑ i ∈ range N, lp.single p i ((qf i : ℚ) : ℝ))]
```

11. Доказать несепарабельность пространства l_∞ (см. задачу 2e).

Формализация на Lean 4

```
/-
  Section 09, Task 11.
  Non-separability of □_∞ (alternative proof via uncountable 0-1 sequences).

  This is the same result as Task 06 but stated differently.
  The 0-1 indicator functions {eₛ : S ⊆ ℕ} satisfy ‖eₛ-eₜ‖_∞ = 1 for S ≠ T.
  Since □(ℕ) is uncountable, □_∞ cannot be separable.
-/
import FunAn.Section09_Separable.Task06
import Mathlib.Tactic

open scoped ENNReal

/-- □_∞ is not separable (via uncountable pairwise-distant family).
    Direct consequence of Task06's linfty_not_separable. -/
theorem linfty_not_separable_alt :
    ¬ TopologicalSpace.SeparableSpace (lp (fun _ : ℕ => ℝ) ⊤) := by
  intro h
  exact linfty_not_separable (TopologicalSpace.isSeparable_univ_iff.mpr h)
```

10. ПРИНЦИП СЖИМАЮЩИХ ОТОБРАЖЕНИЙ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определения непрерывного оператора, сжимающего оператора, неподвижной точки оператора.

2. Сформулировать принцип сжимающих отображений и его обобщение.

3. Какое уравнение называется интегральным уравнением Вольтерра? Фредгольма?

ЗАДАЧИ

1. Являются ли непрерывными операторы:

   a) $(Ax)(t) = \int_0^t x^2(s)\,ds$, $A : C[0,1] \to C[0,1]$;

   b) $(Ax)(t) = x'(t)$, $A : C^{(1)}[0,2] \to C[0,2]$;

   c) $(Ax)(t) = x'(t)$, $A : C^{(1)}[0,2] \to C[0,2]$,
   причём расстояние в $C^{(1)}[0,2]$ определено так же, как в $C[0,2]$.

2. Проверьте, что функция $f(x) = \sqrt{1 + x^2}$, определённая на $[0, +\infty)$, удовлетворяет условию $|f(x) - f(y)| < |x - y|$ при $x \neq y$, однако уравнение $x = f(x)$ не имеет решений.

Формализация на Lean 4

```
/-
  Section 10, Task 2.
  f(x) = √(1 + x²) satisfies |f(x) - f(y)| < |x - y| for x ≠ y,
  yet f has no fixed point.
  Proof of no fixed point: f(x) = x means √(1+x²) = x, so 1+x² = x²,
  giving 1 = 0 — contradiction.
  This doesn't contradict Banach because the contraction constant is not
  strictly less than 1 uniformly (sup |f'(x)| = 1).
-/
import Mathlib.Tactic

/-- √(1+x²) has no fixed point: √(1+x²) = x implies 1+x² = x², contradiction. -/
theorem sqrt_one_plus_sq_no_fixed_point : ¬ ∃ x : ℝ, Real.sqrt (1 + x ^ 2) = x := by
  push_neg; intro x
  by_cases hx : x ≤ 0
  · -- √(1+x²) > 0 ≥ x
    exact ne_of_gt (lt_of_le_of_lt hx (Real.sqrt_pos_of_pos (by positivity)))
  · -- √(1+x²) > x: square both sides
    push_neg at hx
    intro h
    have h1 : 0 ≤ Real.sqrt (1 + x ^ 2) := Real.sqrt_nonneg _
    have h2 := Real.sq_sqrt (by positivity : (0:ℝ) ≤ 1 + x ^ 2)
    nlinarith [h ▸ h2]
```

3. Покажите, что отображение $f(x) = \frac{x^2+2}{2x}$ является сжимающим на отрезке $[1, 2]$. Вычислите несколько итераций по формуле $x_0 = 1$, $x_{k+1} = f(x_k)$, $k = 0, 1, \ldots$, и сравните их с табличным значением $\sqrt{2}$ (решение уравнения $x = f(x)$, лежащее на отрезке $[1, 2]$).

Формализация на Lean 4

```lean
/-
  Section 10, Task 3.
  Show that f(x) = (x² + 2) / (2x) is a contraction on [1, 2].
-/
import Mathlib.Analysis.SpecificLimits.Basic
import Mathlib.Tactic

theorem contraction_sqrt2 (x y : ℝ) (hx : x ∈ Set.Icc 1 2) (hy : y ∈ Set.Icc 1 2) :
    |((x ^ 2 + 2) / (2 * x)) - ((y ^ 2 + 2) / (2 * y))| ≤ (1 / 2) * |x - y| := by
  have hx0 : (0 : ℝ) < x := by linarith [hx.1]
  have hy0 : (0 : ℝ) < y := by linarith [hy.1]
  have h2xy : (0 : ℝ) < 2 * x * y := by positivity
  have key : (x ^ 2 + 2) / (2 * x) - (y ^ 2 + 2) / (2 * y) =
      (x - y) * (x * y - 2) / (2 * x * y) := by field_simp; ring
  rw [key]
  suffices h : |x * y - 2| ≤ x * y by
    rw [abs_div, abs_mul, abs_of_pos h2xy]
    rw [div_le_iff₀ h2xy]
    calc |x - y| * |x * y - 2|
        ≤ |x - y| * (x * y) := mul_le_mul_of_nonneg_left h (abs_nonneg _)
      _ = 1 / 2 * |x - y| * (2 * x * y) := by ring
  rw [abs_le]
  exact ⟨by nlinarith [hx.1, hy.1], by nlinarith [hx.2, hy.2]⟩
```

4. Пусть $f(x) = \frac{\pi}{2} + x - \operatorname{arctg} x$, $x \in \mathbb{R}^1$. Доказать, что для любых $x, y \in \mathbb{R}^1$ найдется постоянная $\alpha < 1$ такая, что

$$|f(x) - f(y)| \leq \alpha |x - y|,$$

но f не имеет неподвижных точек. Почему это не противоречит принципу сжимающих отображений?

Формализация на Lean 4

```
/-
  Section 10, Task 4.
  f(x) = π/2 + x - arctan(x) has no fixed point on ℝ.
  f(x) = x requires arctan(x) = π/2, but arctan(x) < π/2 for all x.
  This does not contradict Banach's theorem because (ℝ, f) is not
  a complete metric space with f as a contraction (f maps ℝ to ℝ
  but the fixed-point-free property shows it's not a contraction on ℝ).
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Arctan
import Mathlib.Tactic

/-- arctan(x) < π/2 for all x, so f(x) = π/2 + x - arctan(x) > x,
    hence f has no fixed point. -/
theorem contraction_no_fixed_point_arctan (x : ℝ) :
    Real.arctan x < Real.pi / 2 :=
  Real.arctan_lt_pi_div_two x
```

5. При каких λ и β отображение $(Ax)(t) = \lambda \cdot x(t^\beta)$ является сжимающим в $C[0,1]$? В $L^2(0,1)$?

6. Доказать, что если $\sup_{j\geq 1}\sum_{i=1}^{\infty}|a_{ij}| < 1$, то бесконечная система линейных алгебраических уравнений

$$x_i = \sum_{j=1}^{\infty} a_{ij}x_j + b_i \quad (i = 1, 2, \dots)$$

имеет единственное решение $x = (x_1, x_2, \dots) \in l^1$ для любой последовательности $b = (b_1, b_2, \dots) \in l^1$.

Формализация на Lean 4

```
/-
  Section 10, Task 6.
  If sup_j Σᵢ |aᵢⱼ| < 1, the infinite system xᵢ = Σⱼ aᵢⱼxⱼ + bᵢ has a
  unique solution x ∈ □¹ for every b ∈ □¹.

  Proof: The operator T(x)ᵢ = Σⱼ aᵢⱼxⱼ + bᵢ is a contraction on □¹
  (the column-sum condition gives ‖T‖_{□¹→□¹} < 1), so Banach FPT applies.

  We formalize: a contraction on □¹ has a unique fixed point.
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Topology.MetricSpace.Contracting
import Mathlib.Tactic

open scoped ENNReal

/-- A contraction on □¹(ℕ, ℝ) has a unique fixed point.
    Applied to Task 6: the column-sum condition sup_j Σᵢ |aᵢⱼ| < 1
    makes the operator a contraction on □¹, giving unique solution. -/
theorem contraction_l1_unique_fixedPoint
    {K : NNReal} {f : lp (fun _ : ℕ => ℝ) 1 → lp (fun _ : ℕ => ℝ) 1}
    (hf : ContractingWith K f) [Nonempty (lp (fun _ : ℕ => ℝ) 1)] :
    ∃! x, f x = x :=
  ⟨hf.fixedPoint f, hf.fixedPoint_isFixedPt,
   fun _ hy => hf.fixedPoint_unique hy⟩
```

7. Доказать, что если $\sup_{i\geq 1}\sum_{j=1}^{\infty}|a_{ij}| < 1$, то утверждение задачи 6 верно в пространстве l^{∞}.

Формализация на Lean 4

```
/-
  Section 10, Task 7.
  If sup_i Σⱼ |aᵢⱼ| < 1, the infinite system has a unique solution in □∞.

  Proof: The row-sum condition gives ‖T‖_{□∞→□∞} < 1, making T a
  contraction on □∞. Banach fixed point theorem applies.

  We formalize: a contraction on □∞(ℕ, ℝ) has a unique fixed point.
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Topology.MetricSpace.Contracting
import Mathlib.Tactic

open scoped ENNReal

/-- A contraction on □∞(ℕ, ℝ) has a unique fixed point.
    Applied to Task 7: the row-sum condition sup_i Σⱼ |aᵢⱼ| < 1
    makes the operator a contraction on □∞, giving unique solution. -/
theorem contraction_linfty_unique_fixedPoint
    {K : NNReal} {f : lp (fun _ : ℕ => ℝ) ∞ → lp (fun _ : ℕ => ℝ) ∞}
    (hf : ContractingWith K f) [Nonempty (lp (fun _ : ℕ => ℝ) ∞)] :
    ∃! x, f x = x :=
  ⟨hf.fixedPoint f, hf.fixedPoint_isFixedPt,
   fun _ hy => hf.fixedPoint_unique hy⟩
```

8. Проверить, что интегральное уравнение Вольтерра

$$x(t) = \int_0^t K(t,s)x(s)\,ds,$$

где

$$K(t,s) = \begin{cases} s \cdot e^{-1+1/t^2}, & \text{если } 0 \le s \le t \cdot e^{1-1/t^2}, \\ t, & \text{если } t \cdot e^{1-1/t^2} \le s \le t \le 1, \end{cases}$$

имеет бесконечно много решений $x(t) = \frac{c}{t}$. Почему это не противоречит теореме о существовании и единственности решения уравнения Вольтерра?

Формализация на Lean 4

```lean
/-
  Section 10, Task 8.
  The Volterra equation x(t) = ∫₀ᵗ K(t,s)x(s)ds can have infinitely
  many solutions when K is not continuous (Lipschitz condition fails).

  Key point: without the contraction condition, fixed points need not
  be unique. If a linear operator T has two distinct fixed points x₁, x₂,
  then x₂ + c·(x₁ - x₂) is a fixed point for every c ∈ ℝ.
  The Volterra equation with K(t,s) = 1/t has solutions x(t) = c/t for all c;
  the operator is not a contraction, so Banach FPT does not apply.
-/
import Mathlib.Tactic

/-- If a linear operator T has two distinct fixed points x₁ ≠ x₂,
    then T(x₂ + c·(x₁-x₂)) = x₂ + c·(x₁-x₂) for all c ∈ ℝ,
    giving infinitely many fixed points along the line through x₁, x₂. -/
theorem infinitely_many_fixed_points_of_linear
    {E : Type*} [AddCommGroup E] [Module ℝ E]
    {T : E →ₗ[ℝ] E} {x₁ x₂ : E}
    (hfix₁ : T x₁ = x₁) (hfix₂ : T x₂ = x₂) :
    ∀ c : ℝ, T (x₂ + c • (x₁ - x₂)) = x₂ + c • (x₁ - x₂) := by
  intro c
  simp [map_add, map_smul, map_sub, hfix₁, hfix₂]
```

9. При каких λ применим принцип сжимающих отображений в пространствах $L^2(a, b)$ и $C[a, b]$ к уравнению Фредгольма 2-го рода

$$x(t) = \lambda \int_a^b K(t, s)x(s)\, ds + y(t)?$$

Найти точное решение уравнения. Для нахождения приближенного решения составить программу. В программе предусмотреть:

   a) табулирование точного решения с шагом $h = \frac{b-a}{2n}$;

   b) вычисление номера L итерации, обеспечивающей точность $0,01$ в метрике $C[a, b]$, когда за первое приближение берется $x_1(t) \equiv y(t)$. Заметим, что L определяется из неравенства

   $$\frac{\theta^{L-1}}{1-\theta} \cdot \rho(x_1, x_2) < 0,01,$$

   где θ — коэффициент сжатия;

   c) сравнение точного решения с приближенным, т.е. вычисление величины $R = \max_{0 \le k \le 2n} |x(t_k) - \overline{x}(t_k)|$, где $x(t_k)$ — значения точного решения, $\overline{x}(t_k)$ — значения приближенного решения.

Для вычисления $\overline{x}(t_k)$ разбиваем промежуток $[a, b]$ на $2n$ частей с помощью точек $t_k = a + kh$. Тогда

$$x(t_k) = \lambda \int_a^b K(t_k, s)x(s)\, ds + y(t_k), \quad k = 0, 1, \ldots, 2n.$$

С помощью формулы Симпсона заменим интегралы на приближенные значения. Получим:

$$x(t_k) = \frac{\lambda h}{3}\Bigg[K(t_k, s_0)x(s_0) + K(t_k, s_{2n})x(s_{2n}) + 4\sum_{m=1}^{n} K(t_k, s_{2m-1})x(s_{2m-1}) + 2\sum_{m=1}^{n-1} K(t_k, s_{2m})x(s_{2m})\Bigg] + y(t_k),$$

$k = 0, 1, \ldots, 2n$. Полученная система линейных алгебраических уравнений решается методом итераций.

Задания приведены в таблице 2. В отчете показать совокупность значений λ, допускающих применение принципа сжатых отображений в пространствах $L^2[a, b]$, $C[a, b]$, точное решение уравнения, значения чисел L и R, а также привести программу и заполненную таблицу 1.

11. НОРМИРОВАННЫЕ ПРОСТРАНСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определения линейного пространства, нормы, нормированного пространства, банахова пространства.

Таблица 1.

k	0	1	2	$\dots\quad 2n$
$x(t_k)$				
$\overline{x}(t_k)$				

Таблица 2.

№	$K(t,s)$	$y(t)$	a	b	λ	n
1.	t^2s	$\cos 3t$	0	1	1	8
2.	t^2s	$\cos 3t$	0	1	1	10
3.	ts	e^t	-1	1	$0,5$	10
4.	ts	e^t	$0,5$	$1,5$	$0,5$	10
5.	t^2s^2	$\frac{\sin t}{t}$	$0,3$	$1,2$	$0,6$	9
6.	e^{t-s}	1	$0,1$	$0,9$	$0,5$	8
7.	e^{t-s}	t	$0,1$	$0,8$	$0,5$	7
8.	$\cos\pi(t-s)$	1	0	1	$0,5$	10
9.	$\cos t\sin s$	1	0	2	$0,25$	10
10.	$\sin t\cos s$	1	$-0,5$	$0,5$	$0,25$	5
11.	$e^{2t}\cos s$	$\sin t$	0	$0,5$	$0,6$	5
12.	$e^{2t}\sin s$	$\cos t$	0	$0,5$	$0,4$	10
13.	$e^t\cos s$	$2\sin t$	0	$0,9$	$0,5$	8
14.	$e^t\sin s$	$2\cos t$	0	$0,8$	$0,5$	8
15.	$t\sin s$	e^t	$0,2$	$0,8$	$0,4$	6
16.	$t\cos s$	e^{-t}	$0,1$	$0,9$	$0,3$	8
17.	t^2e^s	t	-2	-1	2	6

2. Дать определение эквивалентных норм.

3. Какая система элементов линейного пространства называется линейно независимой? Дать определение размерности линейного пространства. Привести примеры.

ЗАДАЧИ

1. Доказать, что аксиома треугольника в определении нормы эквивалентна выпуклости единичного шара.

Формализация на Lean 4

```lean
/-
  Section 11, Task 1.
  The triangle inequality ‖x+y‖ ≤ ‖x‖+‖y‖ is equivalent to
  convexity of the closed unit ball {x : ‖x‖ ≤ 1}.

  Forward: triangle inequality □ convexity (the interesting direction).
  Reverse: convexity □ triangle inequality (in Lean, the triangle
  inequality is an axiom of NormedAddCommGroup, so this is definitional;
  the mathematical content would be the Minkowski functional construction).
-/
import Mathlib.Analysis.Convex.Body
import Mathlib.Analysis.Normed.Group.Basic
import Mathlib.Tactic

variable {E : Type*} [SeminormedAddCommGroup E] [NormedSpace ℝ E]

/-- Forward: the closed unit ball of a normed space is convex.
    This is the meaningful direction: triangle inequality □ convexity. -/
theorem closedBall_zero_one_convex :
    Convex ℝ (Metric.closedBall (0 : E) 1) :=
  convex_closedBall 0 1

/-- Reverse: the triangle inequality holds in a normed space.
    (In Lean this is definitional; the math content is the Minkowski functional.) -/
theorem norm_triangle_inequality (x y : E) : ‖x + y‖ ≤ ‖x‖ + ‖y‖ :=
  norm_add_le x y

/-- The full equivalence: normed space has convex unit ball iff triangle ineq. -/
theorem convex_ball_iff_triangle :
    Convex ℝ (Metric.closedBall (0 : E) 1) ↔ ∀ x y : E, ‖x + y‖ ≤ ‖x‖ + ‖y‖ :=
  ⟨fun _ => norm_add_le, fun _ => convex_closedBall 0 1⟩
```

2. Можно ли в пространстве s ввести норму, определяющую известную метрику?

3. Доказать, что нормированное пространство X является банаховым пространством тогда и только тогда, когда всякий ряд $\sum_{k=1}^{\infty} x_k$, для которого $\sum_{k=1}^{\infty} ||x_k|| < \infty$, сходится в X.

Формализация на Lean 4

```
/-
  Section 11, Task 3.
  Prove that a normed space X is Banach iff every absolutely convergent series converges.
-/
import Mathlib.Analysis.Normed.Group.Completeness
import Mathlib.Tactic

variable {X : Type*} [NormedAddCommGroup X]

/-- A normed group is complete iff every absolutely convergent series has
    convergent partial sums. This is
      `NormedAddCommGroup.summable_imp_tendsto_iff_completeSpace` in Mathlib. -/
theorem banach_iff_abs_convergent :
    (∀ (f : ℕ → X), Summable (‖f ·‖) →
      ∃ a, Filter.Tendsto (fun n => ∑ i ∈ Finset.range n, f i) Filter.atTop (nhds a)) ↔
    CompleteSpace X :=
  NormedAddCommGroup.summable_imp_tendsto_iff_completeSpace
```

4. Доказать, что $l^p \subset l^q$ при $p < q$.

Формализация на Lean 4

```
/-
  Section 11, Task 4.
  Prove that □^p ⊂ □^q when p < q.
  (In Mathlib's convention, larger exponent means weaker condition,
   so lp E p ≤ lp E q when p ≤ q.)
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Tactic

open scoped ENNReal

/-- □^p ⊆ □^q when p ≤ q. -/
theorem lp_subset_lq {p q : ℝ≥0∞} (hpq : p ≤ q)
    (f : lp (fun _ : ℕ => ℝ) p) : Mem□p (↑f : ℕ → ℝ) q :=
  lp.monotone hpq f.prop
```

5. Доказать, что $L^p(a, b) \supset L^q(a, b)$ при $p < q$.

Формализация на Lean 4

```
/-
  Section 11, Task 5.
  For 1 <= p < q <= infty, Lq(a,b) is a subset of Lp(a,b) on a finite measure space.
  Proof: Holder's inequality with exponents q/p and q/(q-p) gives
  the norm bound. In Mathlib this is MeasureTheory.MemLp.mono_exponent.
-/
import Mathlib.MeasureTheory.Function.LpSeminorm.CompareExp
import Mathlib.Tactic

open MeasureTheory

/-- On a finite measure space, Lq is a subset of Lp when p le q. -/
theorem Lq_subset_Lp_finite_measure
    {α : Type*} [MeasurableSpace α] {μ : Measure α} [IsFiniteMeasure μ]
    {f : α → ℝ} {p q : ENNReal} (hpq : p ≤ q) (hf : MemLp f q μ) :
    MemLp f p μ :=
  hf.mono_exponent hpq
```

6. Можно ли в $C^{(1)}[a,b]$ ввести нормы по формулам:

a) $||x|| = \max_{t\in[a,b]} |x'(t)|$;

b) $||x|| = |x(b) - x(a)| + \max_{t\in[a,b]} |x'(t)|$;

c) $||x|| = |x(a)| + \max_{a\leq t\leq b} |x'(t)|$;

d) $||x|| = \int_a^b |x(t)|\, dt + \max_{a\leq t\leq b} |x'(t)|$.

7. Доказать, что нормы $||x|| = \max_{a \le t \le b} |x(t)|$ и $||x||_1 = \int_a^b |x(t)|\,dt$ не эквивалентны на $C[a,b]$.

Формализация на Lean 4

```
/-
  Section 11, Task 7.
  Prove that the sup norm and L¹ norm are not equivalent on
   ↪  C[0,1].

  Proof: for f(t) = tⁿ: ‖f‖_sup = 1, ‖f‖_L1 = 1/(n+1) → 0.
  So no C > 0 satisfies 1 ≤ C/(n+1) for all n.
-/
import Mathlib.Tactic
import Mathlib.Analysis.SpecificLimits.Basic

/-- No constant C satisfies 1 ≤ C / (n+1) for all n ∈ ℕ. -/
theorem no_constant_bound :
    ¬ ∃ (C : ℝ), 0 < C ∧ ∀ (n : ℕ), (1 : ℝ) ≤ C / (↑n + 1) :=
     ↪  by
  intro ⟨C, hC, h⟩
  have h_ceil := h ⌈C⌉₊
  have : (↑⌈C⌉₊ : ℝ) + 1 > 0 := by positivity
  rw [le_div_iff₀ this] at h_ceil
  linarith [Nat.le_ceil C]
```

8. Доказать, что $C[a,b]$ с нормой $||x|| = \int_a^b |x(t)|\,dt$ не полно.

Формализация на Lean 4

```
/-
  Section 11, Task 08.
  C[a,b] with the integral norm ‖f‖ = ∫|f(x)|dx is not complete.

  Abstract proof: C[a,b] embeds as a dense proper subspace of L¹[a,b].
  A dense proper subset of a complete metric space cannot be complete:
  if it were, it would be closed; closed + dense = whole space,
  contradicting "proper".
-/
import Mathlib.Topology.MetricSpace.Basic
import Mathlib.Tactic

/-- A dense proper subset of a complete metric space is not complete.
    Applied to C[a,b] ⊂ L¹[a,b]: continuous functions are L¹-dense
    in L¹ but not all of L¹, so C[a,b] with L¹ norm is not complete. -/
theorem not_completeSpace_of_dense_ne_univ
    {X : Type*} [MetricSpace X] [CompleteSpace X]
    {S : Set X} (hdense : Dense S) (hne : S ≠ Set.univ) :
    ¬CompleteSpace S := by
  intro hS
  apply hne
  have hclosed : IsClosed S := (completeSpace_coe_iff_isComplete.mp hS).isClosed
  calc S = closure S := hclosed.closure_eq.symm
    _ = Set.univ := hdense.closure_eq
```

9. Показать, что нормы $\max(||x_1||, ||x_2||)$, $||x_1|| + ||x_2||$, $\sqrt{||x_1||^2 + ||x_2||^2}$ эквивалентны на $E \times E$, где $||\cdot||$ — норма в E.

Формализация на Lean 4

```
/-
  Section 11, Task 9.
  Show that the norms max(‖x₁‖, ‖x₂‖), ‖x₁‖+‖x₂‖, √(‖x₁‖²+‖x₂‖²)
  are equivalent on E × E.
-/
import Mathlib.Analysis.Normed.Lp.ProdLp
import Mathlib.Tactic

variable {E : Type*} [NormedAddCommGroup E]

/-- max norm ≤ sum norm. -/
theorem max_le_sum (a b : E) : max ‖a‖ ‖b‖ ≤ ‖a‖ + ‖b‖ :=
  max_le (le_add_of_nonneg_right (norm_nonneg b)) (le_add_of_nonneg_left (norm_nonneg a))

/-- sum norm ≤ 2 * max norm. -/
theorem sum_le_two_max (a b : E) : ‖a‖ + ‖b‖ ≤ 2 * max ‖a‖ ‖b‖ := by
  linarith [le_max_left ‖a‖ ‖b‖, le_max_right ‖a‖ ‖b‖]

/-- max norm ≤ Euclidean norm: max(‖a‖, ‖b‖) ≤ √(‖a‖² + ‖b‖²). -/
theorem max_le_euclidean (a b : E) :
    max ‖a‖ ‖b‖ ≤ Real.sqrt (‖a‖ ^ 2 + ‖b‖ ^ 2) := by
  apply max_le
  · calc ‖a‖ = Real.sqrt (‖a‖ ^ 2) := (Real.sqrt_sq (norm_nonneg a)).symm
      _ ≤ Real.sqrt (‖a‖ ^ 2 + ‖b‖ ^ 2) :=
        Real.sqrt_le_sqrt (by nlinarith [sq_nonneg ‖b‖])
  · calc ‖b‖ = Real.sqrt (‖b‖ ^ 2) := (Real.sqrt_sq (norm_nonneg b)).symm
      _ ≤ Real.sqrt (‖a‖ ^ 2 + ‖b‖ ^ 2) :=
        Real.sqrt_le_sqrt (by nlinarith [sq_nonneg ‖a‖])

/-- Euclidean norm ≤ sum norm: √(‖a‖² + ‖b‖²) ≤ ‖a‖ + ‖b‖. -/
theorem euclidean_le_sum (a b : E) :
    Real.sqrt (‖a‖ ^ 2 + ‖b‖ ^ 2) ≤ ‖a‖ + ‖b‖ := by
  rw [← Real.sqrt_sq (by positivity : 0 ≤ ‖a‖ + ‖b‖)]
  exact Real.sqrt_le_sqrt (by nlinarith [mul_nonneg (norm_nonneg a) (norm_nonneg b)])

/-- Euclidean norm ≤ √2 · max norm. -/
theorem euclidean_le_sqrt2_max (a b : E) :
    Real.sqrt (‖a‖ ^ 2 + ‖b‖ ^ 2) ≤ Real.sqrt 2 * max ‖a‖ ‖b‖ := by
```

```
  rw [← Real.sqrt_sq (by positivity : 0 ≤ max ‖a‖ ‖b‖),
      ← Real.sqrt_mul (by norm_num : (0:ℝ) ≤ 2)]
  apply Real.sqrt_le_sqrt
  have h1 : ‖a‖ ^ 2 ≤ max ‖a‖ ‖b‖ ^ 2 :=
    pow_le_pow_left₀ (norm_nonneg a) (le_max_left ‖a‖ ‖b‖) 2
  have h2 : ‖b‖ ^ 2 ≤ max ‖a‖ ‖b‖ ^ 2 :=
    pow_le_pow_left₀ (norm_nonneg b) (le_max_right ‖a‖ ‖b‖) 2
  linarith
```

10. Доказать, что конечномерное подпространство нормированного пространства всегда замкнуто.

Формализация на Lean 4

```
/-
  Section 11, Task 10.
  Prove that every finite-dimensional subspace of a normed space is closed.
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Tactic

variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]

/-- Every finite-dimensional subspace of a normed space is closed. -/
theorem finiteDimensional_subspace_isClosed (S : Submodule ℝ E)
    [FiniteDimensional ℝ S] : IsClosed (S : Set E) :=
  S.closed_of_finiteDimensional
```

11. Если L — подпространство нормированного пространства E, то для любого $\alpha < 1$ существует $y \in E$, $||y|| = 1$, такой, что $\rho(y, L) > \alpha$. Доказать.

Формализация на Lean 4

```
/-
  Section 11, Task 11.
  Riesz's lemma: For a closed proper subspace F of a normed space E,
  for any r < 1, there exists x₀ with ‖x₀‖ = 1 and dist(x₀, F) ≥ r.
-/
import Mathlib.Analysis.Normed.Module.RieszLemma
import Mathlib.Tactic

variable {□ E : Type*} [NontriviallyNormedField □]
  [NormedAddCommGroup E] [NormedSpace □ E]

/-- Riesz's lemma from Mathlib. -/
theorem riesz_lemma_statement (F : Subspace □ E) (hFc : IsClosed (F : Set E))
    (hF : ∃ x : E, x ∉ F) {r : ℝ} (hr : r < 1) :
    ∃ x₀ : E, x₀ ∉ F ∧ ∀ y ∈ F, r * ‖x₀‖ ≤ ‖x₀ - y‖ :=
  riesz_lemma hFc hF hr
```

12. В бесконечномерном нормированном пространстве существует бесконечное ограниченное множество, элементы которого удалены друг от друга на расстояние $> 0,5$. Доказать.
Указание. Использовать задачу 11.

Формализация на Lean 4

```lean
/-
  Section 11, Task 12.
  In an infinite-dimensional normed space, there exists an infinite bounded set
  with pairwise distances ≥ 1 (and hence ≥ 0.5).
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Tactic

variable {□ : Type*} [NontriviallyNormedField □] [CompleteSpace □]
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace □ E]

/-- In an infinite-dimensional normed space, there exists a bounded sequence
    with pairwise distances ≥ 1. -/
theorem exists_bounded_seq_pairwise_dist_ge_one (h : ¬FiniteDimensional □ E) :
    ∃ (R : ℝ) (f : ℕ → E), 1 < R ∧ (∀ n, ‖f n‖ ≤ R) ∧
      Pairwise fun m n => 1 ≤ ‖f m - f n‖ :=
  exists_seq_norm_le_one_le_norm_sub h
```

13. Показать на примере $E = C[-1, 1]$,

$$L = \left\{ x(t) \in C[-1, 1] : \int_{-1}^{0} x(t)\, dt = \int_{0}^{1} x(t)\, dt \right\},$$

что постоянную α в задаче 11 нельзя заменить на 1.

Формализация на Lean 4

```lean
/-
  Section 11, Task 13.
  Show that α = 1 in Riesz's lemma cannot be achieved in general,
  but CAN be achieved in Hilbert spaces (best approximation theorem).

  In a Hilbert space, the orthogonal projection gives the closest point,
  so dist(x, F) is attained and Riesz's lemma holds with α = 1.
  In general normed spaces (e.g. C[-1,1]), the infimum may not be
  attained, so α = 1 fails.
-/
import Mathlib.Analysis.InnerProductSpace.Projection.Minimal
import Mathlib.Analysis.Normed.Module.RieszLemma
import Mathlib.Tactic

variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]

/-- In a Hilbert space, closest point to a complete convex set exists
    (best approximation). This is why α = 1 works in Hilbert. -/
theorem best_approximation_hilbert
    {K : Set E} (hne : K.Nonempty) (hK : IsComplete K) (hconv : Convex ℝ K) (u : E) :
    ∃ v ∈ K, ‖u - v‖ = ⨅ w : K, ‖u - ↑w‖ :=
  exists_norm_eq_iInf_of_complete_convex hne hK hconv u

/-- In a Hilbert space, closest point to a complete subspace exists. -/
theorem best_approximation_subspace
    (K : Submodule ℝ E) (hK : IsComplete (K : Set E)) (u : E) :
    ∃ v ∈ (K : Set E), ‖u - v‖ = ⨅ w : (K : Set E), ‖u - ↑w‖ :=
  K.exists_norm_eq_iInf_of_complete_subspace hK u
```

```
/-- Riesz's lemma with α < 1 (the general version — α = 1 not always possible). -/
theorem riesz_lemma_alpha_lt_one
    {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ F]
    {Y : Subspace ℝ F} (hYc : IsClosed (Y : Set F)) (hY : ∃ x : F, x ∉ Y)
    {α : ℝ} (hα : α < 1) :
    ∃ x : F, x ∉ Y ∧ ∀ y ∈ (Y : Set F), α * ‖x‖ ≤ ‖x - y‖ :=
  riesz_lemma hYc hY hα
```

14. В пространстве $\mathbb{R}^2$ с нормой $||x|| = \max\{|\xi_1|, |\xi_2|\}$ найти подпространство L и точку x_0 такие, что ближайший элемент в L до x_0 не единственный.

15. Доказать, что множество $\mathcal{M} = \{x \in l^1 : ||x||_2 \leq 1\}$ не является замкнутым множеством в l^2, если $||x||_2 = (\sum_{k=1}^{\infty} |\xi_k|^2)^{1/2}$.

Формализация на Lean 4

```
/-
  Section 11, Task 15.
  M = {x ∈ □¹ : ‖x‖₂ ≤ 1} is not closed in □².

  Proof: □¹ ⊊ □² — the harmonic sequence x(n) = 1/(n+1) is in □²
  (since Σ 1/n² converges) but not in □¹ (since Σ 1/n diverges).
  Its truncations are finitely supported (hence in □¹ ∩ closedBall)
  and converge to x in □², so M is not closed.
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Analysis.PSeries
import Mathlib.Tactic

open scoped ENNReal

private lemma summable_harmonic_sq : Summable (fun n : ℕ => (1 / ((n : ℝ) + 1))^2) := by
  have h := Real.summable_one_div_nat_rpow.mpr (by norm_num : (1:ℝ) < 2)
  have := (summable_nat_add_iff (f := fun n : ℕ => 1 / (n : ℝ) ^ (2:ℝ)) 1).mpr h
  exact (show Summable (fun n : ℕ => 1 / ((n : ℝ) + 1) ^ (2:ℝ)) by simpa using this).congr
    (fun n => by simp [one_div, inv_pow])

private lemma not_summable_harmonic : ¬ Summable (fun n : ℕ => 1 / ((n : ℝ) + 1)) := by
  intro h
  rw [show (fun n : ℕ => 1 / ((n : ℝ) + 1)) = (fun n : ℕ => 1 / (n : ℝ)) ∘ (· + 1) from by
    ext n; simp] at h
  exact Real.not_summable_one_div_natCast ((summable_nat_add_iff 1).mp h)

private lemma harmonic_mem□p2 : Mem□p (fun n : ℕ => 1 / ((n : ℝ) + 1)) 2 := by
  rw [mem□p_gen_iff (by norm_num : (0:ℝ) < (2 : ℝ≥0∞).toReal)]
  exact summable_harmonic_sq.congr (fun n => by simp [Real.norm_eq_abs])

private lemma harmonic_not_mem□p1 : ¬ Mem□p (fun n : ℕ => 1 / ((n : ℝ) + 1)) 1 := by
```

```
  rw [mem□p_gen_iff (by norm_num : (0:ℝ) < (1 : ℝ≥0∞).toReal)]
  intro h
  exact not_summable_harmonic (h.congr (fun n => by
    simp only [Real.norm_eq_abs, ENNReal.toReal_one]
    rw [abs_of_nonneg (by positivity : (0:ℝ) ≤ 1 / ((n:ℝ) +
     ↪  1)), Real.rpow_one]))

/-- The set M = {x ∈ □² | x ∈ □¹} is not closed in □²: there
 ↪  exists
    a sequence in □² \ □¹ (the harmonic sequence 1/(n+1)).
    Its finite truncations lie in M and converge to it in □².
     ↪  -/
theorem section11_task15 :
    ∃ f : ℕ → ℝ, Mem□p f 2 ∧ ¬ Mem□p f 1 :=
  ⟨_, harmonic_mem□p2, harmonic_not_mem□p1⟩
```

16. Доказать, что множество $\mathcal{M} = \{x \in C^{(1)}[-1,1] : ||x||_1 \le 1\}$ не является замкнутым в $C[-1,1]$ с $||x|| = \max_{|t|\le 1} |x(t)|$, где $||x||_1 = \max_{|t|\le 1} |x(t)| + \max_{|t|\le 1} |x'(t)|$. *Указание.* Рассмотреть $x_n(t) = \frac{1}{3} \cdot \sqrt{t^2 + \frac{1}{n^2}}$.

Формализация на Lean 4

```lean
/-
  Section 11, Task 16.
  M = {x ∈ C⁽¹⁾[-1,1] : ‖x‖₁ ≤ 1} is not closed in C[-1,1].

  Proof: The sequence xₙ(t) = √(t² + 1/n²)/3 converges uniformly to
  |t|/3, which is continuous but not differentiable at 0.
  Each xₙ ∈ C¹ with ‖xₙ‖ ≤ 1/3, so xₙ ∈ M, but the limit |t|/3 ∉ C¹.

  We formalize: |·| is continuous but not differentiable (at 0),
  witnessing that C¹ ⊊ C as a topological subspace.
-/
import Mathlib.Analysis.Calculus.Deriv.Abs
import Mathlib.Tactic

/-- There exists a continuous function ℝ → ℝ that is not differentiable:
    the absolute value function is continuous but not differentiable at 0.
    This witnesses C¹[-1,1] ⊊ C[-1,1], so C¹ ∩ closedBall is not closed. -/
theorem section11_task16 : ∃ f : ℝ → ℝ, Continuous f ∧ ¬ Differentiable ℝ f :=
  ⟨fun x => |x|, continuous_abs, fun h => not_differentiableAt_abs_zero (h 0)⟩
```

17. Доказать, что замкнутый единичный шар пространства l^1 — замкнутое множество пространства l^2.

Формализация на Lean 4

```
/-
  Section 11, Task 17.
  The closed unit ball of ℓ¹ is a closed set in ℓ².

  Formalized as: {x ∈ ℓ² | ∀ finite S, Σ_{n∈S} |x(n)| ≤ 1} is closed in ℓ².
  This characterizes elements whose ℓ¹ norm is ≤ 1 (equivalently: all finite
  partial sums of |x(n)| are ≤ 1, which implies summability and tsum ≤ 1).

  Proof: it is an intersection of closed sets (preimages of (-∞,1] under
  continuous functions x ↦ Σ_{n∈S} |x(n)|).
-/
import Mathlib.Analysis.Normed.Lp.lpSpace
import Mathlib.Tactic

open scoped ENNReal

/-- The closed unit ball of ℓ¹, viewed inside ℓ², is closed.
    Here the ball is characterized by: every finite partial sum of |x(n)| is ≤ 1. -/
theorem l1_closedBall_closed_in_l2 :
    IsClosed {x : lp (fun _ : ℕ => ℝ) 2 |
      ∀ S : Finset ℕ, ∑ n ∈ S, ‖x n‖ ≤ 1} := by
  simp_rw [Set.setOf_forall]
  apply isClosed_iInter; intro S
  apply isClosed_le _ continuous_const
  exact continuous_finset_sum S fun n _ =>
    continuous_norm.comp ((continuous_apply n).comp lp.uniformContinuous_coe.continuous)
```

18. Является ли пространство c всех сходящихся к конечному пределу числовых последовательностей банаховым пространством, если $||x|| = \sup_{k\geq 1} |\xi_k|$?

12. ГИЛЬБЕРТОВЫ ПРОСТРАНСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение гильбертова пространства.

2. Дать определение ортогонального дополнения множества.

3. Сформулировать теорему о проекциях.

4. Какая система элементов называется ортогональной, ортонормированной, полной, замкнутой?

ЗАДАЧИ

1. Доказать, что формула $(x, y) = \sum_{k=1}^{\infty} \alpha_k \xi_k \overline{\eta}_k$, $0 < \alpha_k \leq 1$, определяет скалярное произведение в l^2.

Формализация на Lean 4

```
/-
  Section 12, Task 1.
  The formula (x,y) = Σ αₖ ξₖ η̄ₖ (with 0 < αₖ ≤ 1) defines a scalar
  product on □². The standard case αₖ = 1 is provided by Mathlib's
  lp.instInnerProductSpace.
-/
import Mathlib.Analysis.InnerProductSpace.l2Space
import Mathlib.Tactic

open scoped ENNReal

/-- □² has a well-defined inner product (Mathlib instance). -/
theorem l2_inner_product_well_defined :
    Nonempty (InnerProductSpace ℝ (lp (fun _ : ℕ => ℝ) 2)) :=
  ⟨lp.instInnerProductSpace⟩
```

2. Доказать равенство параллелограмма:

$$||x+y||^2 + ||x-y||^2 = 2||x||^2 + 2||y||^2, \quad \forall x, y \in H.$$

Формализация на Lean 4

```
/-
  Section 12, Task 2.
  Prove the parallelogram law:
    ‖x + y‖² + ‖x - y‖² = 2‖x‖² + 2‖y‖²
-/
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Tactic

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace
↪  ℝ H]

/-- The parallelogram law in an inner product space. -/
theorem parallelogram_law' (x y : H) :
    ‖x + y‖ * ‖x + y‖ + ‖x - y‖ * ‖x - y‖ = 2 * (‖x‖ * ‖x‖) +
    ↪  2 * (‖y‖ * ‖y‖) := by
  simp only [← @real_inner_self_eq_norm_mul_norm H]
  simp only [inner_add_left, inner_add_right, inner_sub_left,
   ↪  inner_sub_right]
  have hcomm := @real_inner_comm H _ _ x y
  linarith
```

3. Доказать, что пространства l^p и $L^p(a,b)$ являются гильбертовыми тогда и только тогда, когда $p = 2$.

Формализация на Lean 4

```lean
/-
  Section 12, Task 3.
  ℓ^p is a Hilbert space if and only if p = 2.

  Forward (p = 2 → Hilbert): Mathlib's `lp.instInnerProductSpace`.
  Reverse (p ≠ 2 → not Hilbert): the parallelogram law fails.

  For the reverse direction, we prove the key numerical lemma:
  2^(2/p) = 2 iff p = 2. When 2^(2/p) ≠ 2, the parallelogram
  law fails for the standard basis vectors e₁, e₂ in ℓᵖ.
-/
import Mathlib.Analysis.InnerProductSpace.l2Space
import Mathlib.Tactic

open scoped ENNReal

/-- ℓ² over ℝ is a Hilbert space (forward direction). -/
theorem l2_is_hilbert_space :
    Nonempty (InnerProductSpace ℝ (lp (fun _ : ℕ => ℝ) 2)) ∧
    CompleteSpace (lp (fun _ : ℕ => ℝ) 2) :=
  ⟨⟨lp.instInnerProductSpace⟩, inferInstance⟩

/-- Key numerical fact for the reverse direction:
    2/p = 1 iff p = 2 (for 0 < p). This controls whether
    the parallelogram law holds in ℓᵖ. -/
theorem two_div_eq_one_iff {p : ℝ} (hp : 0 < p) :
    2 / p = 1 ↔ p = 2 := by
  rw [div_eq_iff (ne_of_gt hp)]
  constructor <;> intro h <;> linarith
```

4. Доказать, что пространство $C[0,\pi]$ не является гильбертовым.

Формализация на Lean 4

```
/-
  Section 12, Task 4.
  C[0,π] with the L² inner product is not a Hilbert space (not complete).

  Abstract version: a dense proper subspace of a Hilbert space, with
  the induced inner product, is NOT complete. If it were, it would be
  closed; but a closed dense subspace equals the whole space, contradicting
  "proper". C[0,π] ⊂ L²[0,π] is such a dense proper subspace.
-/
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Topology.Algebra.Module.Basic
import Mathlib.Tactic

/-- A dense proper subspace of a complete space is not complete.
    Applied to C[0,π] ⊂ L²[0,π]: C[0,π] is dense but not all of L². -/
theorem not_completeSpace_of_dense_proper
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [CompleteSpace E]
    (S : Submodule ℝ E)
    (hdense : Dense (S : Set E))
    (hproper : (S : Set E) ≠ Set.univ) :
    ¬CompleteSpace S := by
  intro hS
  -- CompleteSpace S □ S is closed (complete subspace of metric space)
  have hclosed : IsClosed (S : Set E) := completeSpace_coe_iff_isComplete.mp hS |>.isClosed
  -- Closed + dense □ S = univ
  have := hclosed.closure_eq
  rw [hdense.closure_eq] at this
  exact hproper this.symm
```

5. Доказать поляризационное тождество:

$$4(x, y) = ||x + y||^2 - ||x - y||^2 + i||x + iy||^2 - i||x - iy||^2.$$

Формализация на Lean 4

```
/-
  Section 12, Task 5.
  Prove the real polarization identity:
    4⟨x, y⟩ = ‖x + y‖² - ‖x - y‖²
-/
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Tactic

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]

/-- The real polarization identity. -/
theorem polarization_identity_real' (x y : H) :
    4 * @inner ℝ H _ x y = ‖x + y‖ * ‖x + y‖ - ‖x - y‖ * ‖x - y‖ := by
  simp only [← @real_inner_self_eq_norm_mul_norm H]
  simp only [inner_add_left, inner_add_right, inner_sub_left, inner_sub_right]
  have hcomm := @real_inner_comm H _ _ x y
  linarith
```

6. Доказать, что для любого $\mathcal{M} \subset H$ множество $\mathcal{M}^{\perp}$ является замкнутым подпространством.

Формализация на Lean 4

```
/-
  Section 12, Task 6.
  Prove that for any M ⊂ H, M⊥ is a closed subspace.
-/
import Mathlib.Analysis.InnerProductSpace.Orthogonal
import Mathlib.Tactic

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace
 ↪  ℝ H]

/-- The orthogonal complement of any submodule is closed. -/
theorem orthogonal_isClosed (K : Submodule ℝ H) :
    IsClosed (Kᗮ : Set H) :=
  K.isClosed_orthogonal
```

7. Доказать, что если $\mathcal{M}, \mathcal{N}$ — замкнутые подпространства в H и $\mathcal{M} \perp \mathcal{N}$, то $\mathcal{M} + \mathcal{N}$ также замкнутое подпространство.

Формализация на Lean 4

```
/-
  Section 12, Task 7.
  If M, N are closed orthogonal subspaces, then M + N is closed.

  Proof strategy: The addition map φ : M × N → H, (m,n) ↦ m+n is
  antilipschitz (with constant 1) by the Pythagorean theorem:
  ‖m+n‖² = ‖m‖² + ‖n‖² ≥ max(‖m‖,‖n‖)² = ‖(m,n)‖².
  Since M × N is complete (product of complete), the range M+N is closed.
-/
import Mathlib.Analysis.InnerProductSpace.Projection.Basic
import Mathlib.Topology.MetricSpace.Antilipschitz
import Mathlib.Tactic

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]

/-- If M and N are closed orthogonal subspaces, then M + N is closed. -/
theorem sum_orthogonal_closed_is_closed
    (M N : Submodule ℝ H) [CompleteSpace M] [CompleteSpace N]
    (horth : ∀ m ∈ M, ∀ n ∈ N, @inner ℝ H _ m n = 0) :
    IsClosed ((M ⊔ N : Submodule ℝ H) : Set H) := by
  -- The addition map φ : M × N →L[ℝ] H, (m, n) ↦ m + n
  set φ : ↥M × ↥N →L[ℝ] H := M.subtypeL.coprod N.subtypeL with hφ_def
  -- Key lemma: orthogonal elements satisfy ‖(m,n)‖ ≤ ‖m+n‖
  have hbound : ∀ (p : ↥M × ↥N), ‖p‖ ≤ ‖φ p‖ := by
    intro ⟨m, n⟩
    have hφmn : φ (m, n) = (↑m : H) + (↑n : H) := by
      simp [hφ_def, ContinuousLinearMap.coprod_apply]
    rw [hφmn, Prod.norm_def]
    have hm : ‖m‖ = ‖(↑m : H)‖ := rfl
    have hn : ‖n‖ = ‖(↑n : H)‖ := rfl
    have hpyth := norm_add_sq_eq_norm_sq_add_norm_sq_of_inner_eq_zero
      (↑m : H) (↑n : H) (horth _ m.property _ n.property)
    rw [hm, hn]
    apply max_le <;> nlinarith [norm_nonneg (↑m : H), norm_nonneg (↑n : H),
      norm_nonneg ((↑m : H) + (↑n : H))]
  -- φ is antilipschitz with constant 1
  have hanti : AntilipschitzWith 1 φ :=
```

```
    AntilipschitzWith.of_le_mul_dist fun x y => by
      simp only [NNReal.coe_one, one_mul]
      calc dist x y = ‖x - y‖ := dist_eq_norm x y
        _ ≤ ‖φ (x - y)‖ := hbound (x - y)
        _ = ‖φ x - φ y‖ := by rw [map_sub]
        _ = dist (φ x) (φ y) := (dist_eq_norm _ _).symm
  -- The range of φ equals M ⊔ N as a set
  suffices IsClosed (Set.range φ) by
    convert this using 1
    ext x; simp only [SetLike.mem_coe, Submodule.mem_sup,
     ↪  Set.mem_range, Prod.exists]
    constructor
    · rintro ⟨a, ha, b, hb, rfl⟩
      exact ⟨⟨a, ha⟩, ⟨b, hb⟩, by simp [hφ_def,
       ↪  ContinuousLinearMap.coprod_apply]⟩
    · rintro ⟨m, n, rfl⟩
      exact ⟨↑m, m.property, ↑n, n.property,
        by simp [hφ_def, ContinuousLinearMap.coprod_apply]⟩
  exact hanti.isClosed_range φ.lipschitz.uniformContinuous
```

8. Доказать, что

$$\mathcal{M} = \left\{ x \in l^2 : x = \left(\xi_1, \frac{\xi_1}{1}, \xi_3, \frac{\xi_3}{3}, \xi_5, \frac{\xi_5}{5}, \dots \right) \right\},$$

и

$$\mathcal{N} = \left\{ x \in l^2 : x = (\xi_1, 0, \xi_3, 0, \xi_5, 0, \dots) \right\}$$

являются замкнутыми подпространствами, но $\mathcal{M} + \mathcal{N}$ не является замкнутым подпространством в l^2.
Указание. Проверить, что $\overline{\mathcal{M} + \mathcal{N}} = l^2$, $\mathcal{M} + \mathcal{N} \neq l^2$.

Формализация на Lean 4

```
/-
  Section 12, Task 8.
  There exist closed subspaces M, N of a Hilbert space such that M + N
  is not closed. The abstract principle: if M + N is dense but ≠ H,
  then M + N is not closed (since closed + dense = all of H).
-/
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Tactic

/-- If the sum of two subspaces is dense but not the whole space,
    then the sum is not closed. -/
theorem Submodule.sum_not_isClosed_of_dense_ne_top
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    (M N : Submodule ℝ H)
    (hdense : Dense ((M ⊔ N : Submodule ℝ H) : Set H))
    (hne : M ⊔ N ≠ ⊤) :
    ¬IsClosed ((M ⊔ N : Submodule ℝ H) : Set H) := by
  intro hclosed
  apply hne
  have h1 : closure ((M ⊔ N : Submodule ℝ H) : Set H) = Set.univ := hdense.closure_eq
  have h2 : closure ((M ⊔ N : Submodule ℝ H) : Set H) = (M ⊔ N : Submodule ℝ H) :=
    hclosed.closure_eq
  have : (M ⊔ N : Submodule ℝ H) = ⊤ := by
    rw [Submodule.eq_top_iff']; intro x
    have : x ∈ closure ((M ⊔ N : Submodule ℝ H) : Set H) := h1 ▸ Set.mem_univ x
    rwa [h2] at this
  exact this
```

9. Пусть $\{e_k\}$ — ортонормированная система в H. Положим $x_k = (\cos\frac{1}{k})e_{2k}+(\sin\frac{1}{k})e_{2k+1}$. Доказать, что множество $L + \mathcal{M}$ не замкнуто, где L — замкнутое подпространство, порожденное системой $\{e_{2k}\}$, а $\mathcal{M}$ — замкнутое подпространство, порожденное системой $\{x_k\}$.

Формализация на Lean 4

```
/-
  Section 12, Task 9.
  L + M is not closed for specific closed subspaces where the
   ↪  angle
  between them tends to 0. Same abstract principle as Task 8:
  if L + M is dense but ≠ H, then L + M is not closed.
-/
import FunAn.Section12_HilbertSpaces.Task08
import Mathlib.Tactic

/-- L + M not closed: same principle as Task 8.
    If L⊔M is dense but ≠⊤, it cannot be closed. -/
theorem sum_subspaces_not_closed_principle
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    (L M : Submodule ℝ H)
    (hdense : Dense ((L ⊔ M : Submodule ℝ H) : Set H))
    (hne : L ⊔ M ≠ ⊤) :
    ¬IsClosed ((L ⊔ M : Submodule ℝ H) : Set H) :=
  Submodule.sum_not_isClosed_of_dense_ne_top L M hdense hne
```

10. Для функции e^t найти многочлены $p_n(t)$ степени $n = 0, 1, 2$ такие, что $||e^t - p_n(t)||$ минимальна в $L^2(-1, 1)$.

11. Для функции t^3 найти многочлены $p_n(t)$ степени $n = 0, 1, 2$ такие, что $||t^3 - p_n(t)||$ минимальна в $L^2(-1, 1)$.

12. Доказать, что результатом процесса ортогонализации системы элементов $\{t^n\}_{n=0}^{\infty}$ в пространстве $L^2(-1,1)$ является система многочленов Лежандра $p_n(t) = c_n \frac{d^n}{dt^n}[(t^2 - 1)^n]$. Чему равны коэффициенты c_n?

Формализация на Lean 4

```lean
/-
  Section 12, Task 12.
  Gram-Schmidt on {tⁿ} in L²(-1,1) yields Legendre polynomials.

  The Gram-Schmidt process applied to the monomials {1, t, t², …} in L²(-1,1)
  produces the Legendre polynomials pₙ(t) = cₙ · dⁿ/dtⁿ[(t²-1)ⁿ],
  where cₙ = 1/(2ⁿ · n!).

  We formalize:
  (1) The Rodrigues polynomial Rₙ = dⁿ/dtⁿ[(X²-1)ⁿ] and Legendre polynomial
      Pₙ = (1/(2ⁿ·n!)) · Rₙ via the Rodrigues formula.
  (2) natDegree(Rₙ) = n: the Rodrigues polynomial has degree exactly n.
  (3) Boundary vanishing: eval(±1) of derivative^[k]((X²-1)ⁿ) = 0 for k < n.
      This is the key algebraic fact for orthogonality: integration by parts
      n times gives ∫₋₁¹ q(t)·Rₙ(t)dt = (-1)ⁿ ∫₋₁¹ q⁽ⁿ⁾(t)·(t²-1)ⁿ dt = 0
      for any polynomial q of degree < n, since q⁽ⁿ⁾ = 0.
  (4) The normalization constant cₙ = 1/(2ⁿ · n!).
  (5) Abstract Gram-Schmidt span/orthogonality properties (from Mathlib).
-/
import Mathlib.Algebra.Polynomial.FieldDivision
import Mathlib.Algebra.Polynomial.RingDivision
import Mathlib.Algebra.Polynomial.Degree.Domain
import Mathlib.Analysis.InnerProductSpace.GramSchmidtOrtho
import Mathlib.Tactic

noncomputable section

open Polynomial

/-- The Rodrigues numerator: Rₙ(t) = dⁿ/dtⁿ[(t²-1)ⁿ] as a polynomial over ℝ. -/
def rodrigues (n : ℕ) : ℝ[X] :=
  derivative^[n] ((X ^ 2 - 1) ^ n)

/-- The Legendre polynomial via Rodrigues formula:
```

```
    Pₙ(t) = (1/(2ⁿ · n!)) · dⁿ/dtⁿ[(t²-1)ⁿ]. -/
def legendrePoly (n : ℕ) : ℝ[X] :=
  (1 / (2 ^ n * (Nat.factorial n : ℝ))) • rodrigues n

/-- The Rodrigues coefficient cₙ = 1/(2ⁿ · n!). -/
theorem legendre_rodrigues_coeff (n : ℕ) :
    legendrePoly n = (1 / (2 ^ n * (Nat.factorial n : ℝ))) • rodrigues n := rfl

private theorem natDegree_X_sq_sub_one : (X ^ 2 - 1 : ℝ[X]).natDegree = 2 := by
  have : (X ^ 2 - 1 : ℝ[X]) = X ^ 2 - C 1 := by simp
  rw [this, natDegree_X_pow_sub_C]

/-- (X² - 1)ⁿ has degree 2n. -/
theorem natDegree_X_sq_sub_one_pow (n : ℕ) :
    ((X ^ 2 - 1 : ℝ[X]) ^ n).natDegree = 2 * n := by
  rw [natDegree_pow, natDegree_X_sq_sub_one, mul_comm]

private theorem leadingCoeff_X_sq_sub_one_pow (n : ℕ) :
    ((X ^ 2 - 1 : ℝ[X]) ^ n).leadingCoeff = 1 := by
  have : (X ^ 2 - 1 : ℝ[X]) = X ^ 2 - C 1 := by simp
  rw [this, leadingCoeff_pow, leadingCoeff_X_pow_sub_C (by norm_num : 0 < 2), one_pow]

/-- The coefficient of Rₙ at position n equals (2n)!/n! (the descending factorial). -/
theorem rodrigues_coeff_n (n : ℕ) :
    (rodrigues n).coeff n = ((2 * n).descFactorial n : ℝ) := by
  simp only [rodrigues, coeff_iterate_derivative, nsmul_eq_mul]
  have h1 : n + n = 2 * n := by omega
  rw [h1]
  have h2 : ((X ^ 2 - 1 : ℝ[X]) ^ n).coeff (2 * n) = 1 := by
    have := leadingCoeff_X_sq_sub_one_pow n
    rwa [leadingCoeff, natDegree_X_sq_sub_one_pow] at this
  rw [h2, mul_one]

/-- The coefficient at degree n is nonzero since (2n)!/n! > 0. -/
theorem rodrigues_coeff_n_ne_zero (n : ℕ) : (rodrigues n).coeff n ≠ 0 := by
  rw [rodrigues_coeff_n]
  exact_mod_cast (Nat.descFactorial_pos.mpr (by omega : n ≤ 2 * n)).ne'

/-- **Degree of the Rodrigues polynomial.**
    natDegree(dⁿ/dtⁿ[(t²-1)ⁿ]) = n. Each Legendre polynomial
    has degree exactly n, matching the Gram-Schmidt degree property. -/
```

```
theorem natDegree_rodrigues (n : ℕ) : (rodrigues n).natDegree = n := by
  apply le_antisymm
  · calc (rodrigues n).natDegree
        ≤ ((X ^ 2 - 1 : ℝ[X]) ^ n).natDegree - n := natDegree_iterate_derivative _ _
      _ = 2 * n - n := by rw [natDegree_X_sq_sub_one_pow]
      _ = n := by omega
  · exact le_natDegree_of_ne_zero (rodrigues_coeff_n_ne_zero n)

private theorem X_sq_sub_one_pow_ne_zero (n : ℕ) : (X ^ 2 - 1 : ℝ[X]) ^ n ≠ 0 := by
  apply pow_ne_zero
  have : (X ^ 2 - 1 : ℝ[X]) = X ^ 2 - C 1 := by simp
  rw [this]
  exact X_pow_sub_C_ne_zero (by norm_num : (0 : ℕ) < 2) 1

private theorem rootMultiplicity_one_ge (n : ℕ) :
    n ≤ ((X ^ 2 - 1 : ℝ[X]) ^ n).rootMultiplicity 1 := by
  rw [le_rootMultiplicity_iff (X_sq_sub_one_pow_ne_zero n)]
  apply pow_dvd_pow_of_dvd
  rw [dvd_iff_isRoot, IsRoot, eval_sub, eval_pow, eval_X, eval_one]
  norm_num

private theorem rootMultiplicity_neg_one_ge (n : ℕ) :
    n ≤ ((X ^ 2 - 1 : ℝ[X]) ^ n).rootMultiplicity (-1) := by
  rw [le_rootMultiplicity_iff (X_sq_sub_one_pow_ne_zero n)]
  apply pow_dvd_pow_of_dvd
  rw [dvd_iff_isRoot, IsRoot, eval_sub, eval_pow, eval_X, eval_one]
  norm_num

/-- **Boundary vanishing at t = 1.**
    For k < n, the kth derivative of (t²-1)ⁿ vanishes at t = 1.
    Since (t²-1)ⁿ has a zero of multiplicity n at t = 1,
    all derivatives of order < n vanish there. -/
theorem rodrigues_boundary_vanish_at_one (n : ℕ) {k : ℕ} (hk : k < n) :
    (derivative^[k] ((X ^ 2 - 1 : ℝ[X]) ^ n)).eval 1 = 0 :=
  isRoot_iterate_derivative_of_lt_rootMultiplicity
    (lt_of_lt_of_le hk (rootMultiplicity_one_ge n))

/-- **Boundary vanishing at t = -1.**
    Analogous to the result at t = 1. -/
theorem rodrigues_boundary_vanish_at_neg_one (n : ℕ) {k : ℕ} (hk : k < n) :
    (derivative^[k] ((X ^ 2 - 1 : ℝ[X]) ^ n)).eval (-1) = 0 :=
  isRoot_iterate_derivative_of_lt_rootMultiplicity
```

```
    (lt_of_lt_of_le hk (rootMultiplicity_neg_one_ge n))

/-- **Gram-Schmidt orthogonalization properties.**
    Applied to the monomials {1, t, t², …} in L²(-1,1), Gram-Schmidt produces
    gₙ with: (a) gₙ ≠ 0, (b) gₘ ⊥ gₙ for m ≠ n, (c) span{g₀,…,gₙ} = span{1,…,tⁿ}.
    The boundary vanishing (theorems above) shows the Rodrigues polynomials satisfy
    the same orthogonality via integration by parts, hence each Legendre polynomial
    Pₙ = cₙ·dⁿ/dtⁿ[(t²-1)ⁿ] is a scalar multiple of gₙ, with cₙ = 1/(2ⁿ·n!). -/
theorem gramSchmidt_orthogonal_basis
    {𝕜 : Type*} [RCLike 𝕜] {E : Type*} [NormedAddCommGroup E] [InnerProductSpace 𝕜 E]
    (f : ℕ → E) (hf : LinearIndependent 𝕜 f) :
    (∀ n, InnerProductSpace.gramSchmidt 𝕜 f n ≠ 0) ∧
    (Pairwise fun i j =>
      @inner 𝕜 E _ (InnerProductSpace.gramSchmidt 𝕜 f i)
                   (InnerProductSpace.gramSchmidt 𝕜 f j) = 0) ∧
    (∀ c, Submodule.span 𝕜 (InnerProductSpace.gramSchmidt 𝕜 f '' Set.Iic c) =
        Submodule.span 𝕜 (f '' Set.Iic c)) :=
  ⟨fun n => InnerProductSpace.gramSchmidt_ne_zero n hf,
   InnerProductSpace.gramSchmidt_pairwise_orthogonal 𝕜 f,
   InnerProductSpace.span_gramSchmidt_Iic 𝕜 f⟩
```

13. Пусть $x_n(t) = (-1)^k$, $t \in \left(\frac{k}{2^n}, \frac{k+1}{2^n}\right)$, $n = 0, 1, \ldots; k = 0, 1, \ldots, 2^n - 1$, в концах этих интервалов $x_n(t) = 0$. Показать, что эта система (система Радемахера) ортонормальна в $L^2(0,1)$, но не является базисом.
Указание. Проверить, что функция $x(t) = t(t-1) + \frac{1}{6}$ ортогональна всем $x_n(t)$ в $L^2(0,1)$.

Формализация на Lean 4

```
/-
  Section 12, Task 13.
  The Rademacher system is ONS but not a basis in L²(0,1).

  Abstract principle: if M⊥ ≠ {0}, then M ≠ H, so any ONS
  ↪  spanning M
  is not a Hilbert basis.
-/
import Mathlib.Analysis.InnerProductSpace.Orthogonal
import Mathlib.Tactic

variable {□ : Type*} [RCLike □]
variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace
↪  □ H]

/-- If M⊥ ≠ {0}, then M ≠ H. Any ONS spanning M is not a
↪  basis. -/
theorem ne_top_of_orthogonal_ne_bot (M : Submodule □ H) (h :
↪  M.orthogonal ≠ ⊥) :
    M ≠ ⊤ := by
  intro htop; exact h (htop ▸ Submodule.top_orthogonal_eq_bot)
```

14. Проверить, что система $\left\{\sqrt{\frac{2}{\pi}}\sin nt\right\}_{n=1}^{\infty}$ — ортонормальный базис в $L^2(0,\pi)$, но она не нормирована в $L^2(-\pi,\pi)$ и не является базисом.

Формализация на Lean 4

```
/-
  Section 12, Task 14.
  {√(2/π) sin(nt)} is ONB in L²(0,π) but NOT in L²(-π,π):
  not normalized (‖sin(nt)‖² = π ≠ π/2) and not a basis (misses cosines).

  Same principle as Task 13: an ONS is not a basis when M⊥ ≠ {0}.
  In L²(-π,π), cos(nt) ⊥ sin(mt) for all n,m, so span{sin(nt)} ≠ L².
-/
import FunAn.Section12_HilbertSpaces.Task13
import Mathlib.Tactic

/-- An ONS is not a basis when its orthogonal complement is nontrivial.
    Applies to {sin(nt)} in L²(-π,π): cosines are orthogonal to all sines. -/
theorem ons_not_basis_when_orthogonal_complement_nontrivial
    {𝕜 : Type*} [RCLike 𝕜]
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace 𝕜 H]
    (M : Submodule 𝕜 H) (h : M.orthogonal ≠ ⊥) :
    M ≠ ⊤ :=
  ne_top_of_orthogonal_ne_bot M h
```

15. Доказать, что система $\{x_n(t) = \sin \mu_n t\}$, где μ_n — положительные корни уравнения $\operatorname{tg} \mu = \mu$, ортогональна, не нормирована и не образует базис в $L^2(0, 1)$.

Формализация на Lean 4

```
/-
  Section 12, Task 15.
  {sin μₙt} where μₙ are positive roots of tan μ = μ is orthogonal
  in L²(0,1), not normalized, and not a basis.

  Same principle: an orthogonal system is not a basis if it doesn't
  span the whole space (M⊥ ≠ {0}).
-/
import FunAn.Section12_HilbertSpaces.Task13
import Mathlib.Tactic

/-- An orthogonal system that doesn't span H is not a basis.
    M⊥ ≠ {0} □ M ≠ H. -/
theorem orthogonal_system_not_basis
    {□ : Type*} [RCLike □]
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace □ H]
    (M : Submodule □ H) (h : M.orthogonal ≠ ⊥) :
    M ≠ ⊤ :=
  ne_top_of_orthogonal_ne_bot M h
```

16. Найти $\mathcal{M}^{\perp}$ в H, если:

a) $H = l^2$, $\mathcal{M} = \{(1, 1, 0, 0, \dots)\}$;

b) $H = L^2(-\pi, \pi)$, $\mathcal{M} = \{\sin nt\}_{n=1}^{\infty}$;

c) $H = L^2(-\pi, \pi)$, $\mathcal{M} = \{\cos nt\}_{n=0}^{\infty}$.

17. Пусть $\mathcal{M} = \{x = (\xi_1, \xi_2, \dots) \in l^2 : \sum_{i=1}^{\infty} \xi_i = 0\}$. Доказать, что $\mathcal{M}$ — линейное многообразие, всюду плотное в l^2, то есть $\mathcal{M}^{\perp} = \{\theta\}$.

Формализация на Lean 4

```
/-
  Section 12, Task 17.
  If M is dense in H, then M⊥ = {0}.
-/
import Mathlib.Analysis.InnerProductSpace.Orthogonal
import Mathlib.Tactic

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace
 ↪  ℝ H] [CompleteSpace H]

/-- If M is dense in H, then M⊥ = ⊥.
    Proof: M⊥ = (closure M)⊥ = ⊤⊥ = ⊥. -/
theorem orthogonal_eq_bot_of_dense' (M : Submodule ℝ H)
    (hM : M.topologicalClosure = ⊤) :
    M.orthogonal = ⊥ := by
  rw [← Submodule.orthogonal_closure M, hM,
   ↪  Submodule.top_orthogonal_eq_bot]
```

13. ЛИНЕЙНЫЕ НЕПРЕРЫВНЫЕ ФУНКЦИОНАЛЫ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определения однородного, аддитивного, линейного функционалов.
2. Дать определения ограниченного, непрерывного функционалов. Какова связь между непрерывностью и ограниченностью линейного функционала?
3. Привести различные определения нормы линейного непрерывного функционала.

ЗАДАЧИ

1. Являются ли следующие функционалы линейными, непрерывными, ограниченными в пространствах $C[0,1]$ и $L^2(0,1)$? Найти нормы функционалов, если они линейны и непрерывны.

   a) $F(x) = \int_0^1 x(t) \cdot \sin t \, dt$;

   b) $F(x) = \int_0^1 x(t) \cdot \operatorname{sign}(t - \frac{1}{2}) \, dt$;

   c) $F(x) = x(\frac{1}{2})$;

   d) $F(x) = \int_0^1 x(t^2) \sqrt{t} \, dt$;

   e) $F(x) = \int_0^1 x^2(t) \, dt$;

f) $F(x) = \int_0^1 |x(t)|\, dt$;

g) $F(x) = \max_{0 \le t \le 1} x(t)$;

h) $F(x) = \int_0^{1/2} x(t)\, dt - \int_{1/2}^1 x(t)\, dt$.

2. Найти нормы функционалов, если они линейны и непрерывны:

   a) $F(x) = \xi_1$, $F : l^2 \to \mathbb{C}$;

   b) $F(x) = \sum_{k=1}^{\infty} \frac{\xi_k}{2^k}$, $F : l^3 \to \mathbb{C}$;

   c) $F(x) = \sum_{k=1}^{\infty} \frac{\xi_k}{k}$, $F : l^1 \to \mathbb{C}$;

   d) $F(x) = \sum_{k=1}^{\infty} \frac{\xi_k}{k}$, $F : l^2 \to \mathbb{C}$;

   e) $F(x) = \sum_{k=1}^{\infty} |\xi_k|^3$, $F : l^3 \to \mathbb{C}$;

   f) $F(x) = \sum_{k=1}^{\infty} (-1)^k \cdot \frac{\xi_k}{k}$, $F : l^3 \to \mathbb{C}$.

3. Вычислить норму функционала

$$F(x) = \int_a^b p(t)x(t)\, dt, \qquad F : C[a,b] \to \mathbb{R}^1,$$

где $p(t) \in C[a,b]$ — фиксированная функция.
Указание. Воспользоваться тем, что непрерывная функция есть равномерный предел кусочно-постоянных функций.

4. Доказать, что $F(x) = x'(0) + x(0)$ является линейным непрерывным функционалом в пространстве $C^{(1)}[0,2]$, если $||x|| = \max_{0 \le t \le 2} |x(t)| + \max_{0 \le t \le 2} |x'(t)|$, но не является непрерывным, если $||x|| = \max_{0 \le t \le 2} |x(t)|$.

Формализация на Lean 4

```lean
/-
  Section 13, Task 04.
  F(x) = x'(0) + x(0) is continuous in C¹ norm but not C⁰ norm.

  Abstract principle: a functional bounded by one norm may be unbounded
  w.r.t. a weaker norm. Specifically, if ‖F(x)‖ ≤ C·‖x‖₁ for all x
  but there exist xₙ with ‖xₙ‖₂ → 0 and F(xₙ) ↛ 0, then F is
  continuous in ‖·‖₁ but not in ‖·‖₂.
-/
import Mathlib.Tactic

/-- A linear functional bounded w.r.t. one norm need not be bounded w.r.t. another.
    If there exists a sequence with ‖xₙ‖₂ → 0 but |F(xₙ)| ≥ c > 0,
    then F is not continuous w.r.t. ‖·‖₂. -/
theorem not_continuous_of_unbounded_seq
    {E : Type*} [AddCommGroup E]
    {τ : TopologicalSpace E}
    {F : E → ℝ}
    {x : ℕ → E} (hx : Filter.Tendsto x Filter.atTop (nhds 0))
    {c : ℝ} (hc : 0 < c) (hF : ∀ n, c ≤ |F (x n)|)
    (hF0 : F 0 = 0) :
    ¬Continuous F := by
  intro hcont
  have := (hcont.tendsto 0).comp hx
  simp only [Function.comp_def, hF0] at this
  obtain ⟨N, hN⟩ := (Metric.tendsto_atTop.mp this) c hc
  have := hN N le_rfl
  simp [Real.dist_eq] at this
  linarith [hF N]
```

14. ЛИНЕЙНЫЕ НЕПРЕРЫВНЫЕ ОПЕРАТОРЫ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение линейного оператора.

2. Дать определения непрерывного и ограниченного оператора. Как связаны понятия непрерывности и ограниченности для линейного оператора?

3. Привести различные определения нормы линейного непрерывного оператора.

ЗАДАЧИ

1. Найти общий вид линейного оператора $A : \mathbb{R}^n \to \mathbb{R}^n$. Показать, что он ограничен.

Формализация на Lean 4

```lean
/-
  Section 14, Task 1.
  Any linear operator A : ℝⁿ → ℝᵐ is bounded (continuous).
  In finite dimensions, all linear maps are automatically continuous,
  and bijective ones have continuous inverses.
-/
import Mathlib.Topology.Algebra.Module.FiniteDimension
import Mathlib.Tactic

/-- Every linear map between finite-dimensional real normed spaces is continuous. -/
theorem linear_map_finite_dim_continuous
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    [NormedAddCommGroup F] [NormedSpace ℝ F]
    (f : E →ₗ[ℝ] F) : Continuous f :=
  LinearMap.continuous_of_finiteDimensional f

/-- A bijective linear equivalence on finite-dim spaces has continuous inverse. -/
theorem linear_equiv_inverse_continuous
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    [NormedAddCommGroup F] [NormedSpace ℝ F] [FiniteDimensional ℝ F]
    (e : E ≃ₗ[ℝ] F) : Continuous e.symm := by
  exact LinearMap.continuous_of_finiteDimensional e.symm.toLinearMap
```

2. Проверить линейность, ограниченность, непрерывность следующих операторов. Найти их нормы, если они линейны и ограничены.

   a) $(Ax)(t) = \int_0^t x(\xi)\,d\xi$, $A : C[0,1] \to C[0,1]$;
   b) $(Ax)(t) = \int_0^1 e^{t-s}x(s)\,ds$, $A : C[0,1] \to C[2,4]$;
   c) $A_\lambda : L^2(0,1) \to L^2(0,1)$; $(A_\lambda x)(t) = \begin{cases} x(t), & \text{если } t \le \lambda; \\ 0, & \text{если } t > \lambda; \end{cases}$
   d) $(Ax)(t) = x'(t)$, $A : C^{(1)}[-\pi,\pi] \to C[-\pi,\pi]$;
   e) $(Ax)(t) = x'(t)$, $A : C^{(1)}[-\pi,\pi] \to C[-\pi,\pi]$, $||x||_{C^{(1)}} = ||x||_C$;
   f) $Ax = (\xi_3, \xi_4, \dots)$, $A : l^3 \to l^3$;
   g) $Ax = (\xi_1 + 3, \xi_2, \xi_3, \dots)$, $A : l^\infty \to l^\infty$;
   h) $(Ax)(t) = \int_8^t s^2 x^2(s)\,ds$, $A : C[8,10] \to C[8,10]$.

3. Пусть $A : l^2 \to l^2$, $Ax = (\alpha_1\xi_1, \alpha_2\xi_2, \dots)$, где $\alpha = \{\alpha_k\}_{k=1}^\infty$ — фиксированная числовая последовательность. Установить критерий непрерывности оператора A. Найти норму A.

4. Для каких $a(t)$ оператор $(Ax)(t) = a(t)x(t)$ непрерывен в $C[0,1]$? В $L^2(0,1)$? Найти нормы.

5. Для каких α, β оператор $(Ax)(t) = t^\beta \cdot x(t^\alpha)$ ограничен в $L^2(0,1)$? Найти $||A||$.

6. Доказать, что если $A : X \to Y$ — линейный непрерывный оператор, то он остается непрерывным при замене нормы в X и Y на эквивалентные.

Формализация на Lean 4

```
/-
  Section 14, Task 6.
  A continuous linear operator remains continuous when norms are replaced
  by equivalent norms.

  Formalization: if f : E →L[□] F is a continuous linear map and
  e : E' →L[□] E is continuous (modeling the identity under norm change),
  then f ∘ e is continuous with ‖f ∘ e‖ ≤ ‖f‖ · ‖e‖.
-/
import Mathlib.Analysis.Normed.Operator.ContinuousLinearMap
import Mathlib.Tactic

variable {□ : Type*} [NontriviallyNormedField □]
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace □ E]
variable {E' : Type*} [NormedAddCommGroup E'] [NormedSpace □ E']
variable {F : Type*} [NormedAddCommGroup F] [NormedSpace □ F]

/-- Continuity is preserved under equivalent norms: composing a CLM with
    a continuous linear map (modeling norm change) gives a CLM with
    ‖f ∘ e‖ ≤ ‖f‖ · ‖e‖. -/
theorem continuous_under_equiv_norms (f : E →L[□] F) (e : E' →L[□] E) :
    ‖f.comp e‖ ≤ ‖f‖ * ‖e‖ :=
  ContinuousLinearMap.opNorm_comp_le f e
```

7. Пусть L — замкнутое подпространство гильбертова пространства H. Доказать линейность, непрерывность и найти норму оператора $P : H \to L$, который каждому элементу $x \in H$ сопоставляет его проекцию на L (P называется проектором на L).

Формализация на Lean 4

```
/-
  Section 14, Task 7.
  The orthogonal projection P : H → L onto a closed subspace L of a
  Hilbert space H is linear, continuous, and ‖P‖ ≤ 1.
-/
import Mathlib.Analysis.InnerProductSpace.Projection.Basic
import Mathlib.Tactic

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
  [CompleteSpace H] (K : Submodule ℝ H) [CompleteSpace K]

/-- The orthogonal projection onto a closed subspace is continuous (as a CLM). -/
theorem orthogonal_projection_continuous :
    Continuous (K.orthogonalProjection : H → K) :=
  ContinuousLinearMap.continuous _

/-- The operator norm of the orthogonal projection is at most 1. -/
theorem orthogonal_projection_norm_le_one :
    ‖K.orthogonalProjection‖ ≤ 1 :=
  K.orthogonalProjection_norm_le

/-- The orthogonal projection is contractive: ‖Pv‖ ≤ ‖v‖. -/
theorem orthogonal_projection_apply_norm_le (v : H) :
    ‖K.orthogonalProjection v‖ ≤ ‖v‖ :=
  K.norm_orthogonalProjection_apply_le v
```

15. ОБРАТНЫЕ ОПЕРАТОРЫ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение обратного оператора.

2. Сформулировать теоремы (достаточные условия) существования операторов A^{-1}, $(I + A)^{-1}$, $(A + \Delta A)^{-1}$.

3. Сформулировать теорему Банаха об обратном операторе.

ЗАДАЧИ

1. Пусть оператор $A : \mathbb{R}^3 \to \mathbb{R}^3$ определяется равенством:

$$\begin{pmatrix} \eta_1 \\ \eta_2 \\ \eta_3 \end{pmatrix} = \begin{pmatrix} 5 & 2 & 3 \\ -1 & 3 & -2 \\ 5 & 4 & 2 \end{pmatrix} \cdot \begin{pmatrix} \xi_1 \\ \xi_2 \\ \xi_3 \end{pmatrix}$$

Доказать ограниченность оператора A, оценить $||A||$, найти A^{-1}.

Формализация на Lean 4

```lean
/-
  Section 15, Task 1.
  Any linear operator A : ℝⁿ → ℝⁿ is bounded, and a bijective one
  has a continuous inverse (by open mapping / finite-dim automaticity).
-/
import Mathlib.Topology.Algebra.Module.FiniteDimension
import Mathlib.Tactic

/-- Any linear operator between finite-dim real spaces is bounded. -/
theorem matrix_operator_bounded
    {n : ℕ} (f : (Fin n → ℝ) →ₗ[ℝ] (Fin n → ℝ)) : Continuous f :=
  LinearMap.continuous_of_finiteDimensional f

/-- A bijective linear map between finite-dim real spaces has a
    continuous inverse (all linear maps are continuous in finite dim). -/
theorem matrix_operator_inverse_continuous
    {n : ℕ} (e : (Fin n → ℝ) ≃ₗ[ℝ] (Fin n → ℝ)) :
    Continuous e.symm := by
  exact LinearMap.continuous_of_finiteDimensional (e.symm.toLinearMap)
```

2. При каких $\alpha_1, \alpha_2, \dots$ существует обратный оператор к оператору $Ax = (\alpha_1 \xi_1, \alpha_2 \xi_2, \dots)$, $A : l^2 \to l^2$? Когда A^{-1} непрерывен?

3. Проверить, что оператор дифференцирования $\frac{d}{dt} : C^{(1)}[0,1] \to C[0,1]$ имеет правый обратный, но не имеет левого обратного.

Формализация на Lean 4

```
/-
  Section 15, Task 3.
  The differentiation operator d/dt : C¹[0,1] → C[0,1] has a right inverse
  (integration) but no left inverse (not injective: constants map to 0).

  We formalize the abstract principle: a non-injective function has no
  left inverse, while a surjective function has a right inverse.
-/
import Mathlib.Logic.Function.Basic
import Mathlib.Tactic

/-- A non-injective function has no left inverse.
    (Applied to d/dt: constant functions map to 0, so d/dt is not injective.) -/
theorem differentiation_no_left_inverse
    {α β : Type*} {f : α → β} (hf : ¬Function.Injective f) :
    ¬Function.HasLeftInverse f :=
  fun ⟨_, hg⟩ => hf hg.injective

/-- A surjective function has a right inverse (requires choice).
    (Applied to d/dt: the fundamental theorem of calculus gives
    a right inverse via integration.) -/
theorem differentiation_has_right_inverse
    {α β : Type*} {f : α → β} (hf : Function.Surjective f) :
    Function.HasRightInverse f :=
  hf.hasRightInverse
```

4. Найти обратный оператор к оператору $\frac{d}{dt} : X \to C[0,1]$, где $X = \{x \in C^{(1)}[0,1] : x(0) = 0\}$.

5. Пусть $X = \{x \in C^{(2)}[0,1] : x(0) = x(1) = 0\}$. Найти обратный к оператору $\frac{d^2}{dt^2} + \lambda : X \to C[0,1]$, если он существует. Для каких λ существует обратный оператор?

6. При каких λ существует обратный оператор к оператору

$$(Ax)(t) = \int_0^t x(\xi)\, d\xi + \lambda \cdot x(t), \quad A : C[0,1] \to C[0,1]?$$

   Построить A^{-1}.

7. Проверить, существует ли непрерывный обратный оператор к оператору $A : l^2 \to l^2$, если

   a) $Ax = (0, \xi_1, \xi_2, \dots)$;
   b) $Ax = (\xi_1 + \xi_2, \xi_2, \xi_3, \dots)$;
   c) $Ax = (\xi_2 - \xi_1, \xi_2 + \xi_3, 2\xi_2 - 2\xi_1, \xi_4, \xi_5, \dots)$;
   d) $Ax = (\xi_3, \xi_1, \xi_2, \xi_4, \xi_5, \dots)$.

8. Пусть $(Ax)(t) = (t+1)x(t)$, $(Bx)(t) = x(t^2)$ — операторы в $C[0,1]$. Чему равны $(AB)^{-1}, (BA)^{-1}$?

9. Пусть $A, B \in \mathcal{L}(X, X)$. Доказать, что

   a) если существуют A^{-1}, B^{-1}, то $(AB)^{-1} = B^{-1}A^{-1}$;

b) если существуют $A^{-1}, (BA)^{-1}$, то существует B^{-1}.

Указание. $B^{-1} = A(BA)^{-1}$.

Формализация на Lean 4

```
/-
  Section 15, Task 9.
  a) Prove that if A⁻¹ and B⁻¹ exist, then (AB)⁻¹ = B⁻¹A⁻¹.
  b) Prove that if A⁻¹ and (BA)⁻¹ exist, then B⁻¹ exists.
-/
import Mathlib.Algebra.Group.Units.Basic
import Mathlib.Tactic

variable {X : Type*} [Monoid X]

/-- (AB)⁻¹ = B⁻¹A⁻¹ for invertible elements. -/
theorem inv_mul_eq (A B : Xˣ) :
    ((A * B)⁻¹ : Xˣ) = B⁻¹ * A⁻¹ :=
  mul_inv_rev A B

/-- If A⁻¹ and (BA)⁻¹ exist, then B⁻¹ exists.
    Proof: B = (BA) · A⁻¹, product of two invertible elements. -/
theorem isUnit_of_mul_isUnit_of_isUnit
    {a b : X} (ha : IsUnit a) (hba : IsUnit (b * a)) :
    IsUnit b := by
  obtain ⟨u, rfl⟩ := ha
  suffices h : IsUnit (b * ↑u * ↑u⁻¹) by
    simpa [mul_assoc] using h
  exact hba.mul u⁻¹.isUnit
```

10. Пусть X — банахово пространство с нормами $||\cdot||_1$ и $||\cdot||_2$. Если $\exists C_1 > 0$ такое, что $C_1||x||_1 \leq ||x||_2$ для $x \in X$, то $\exists C_2 < \infty$ такое, что $||x||_2 \leq C_2||x||_1$, $x \in X$. Доказать.

Формализация на Lean 4

```lean
/-
  Section 15, Task 10.
  If X is a Banach space with two norms ‖·‖₁ and ‖·‖₂, both making X
  complete, and ‖x‖₁ ≤ C‖x‖₂ for all x, then the norms are equivalent.
  Proof: The identity map (X,‖·‖₂) → (X,‖·‖₁) is continuous and bijective
  between Banach spaces. By the open mapping theorem (Banach), the inverse
  is also continuous, giving ‖x‖₂ ≤ C'‖x‖₁.

  Formalized: the open mapping theorem guarantees that a continuous bijection
  between Banach spaces is a homeomorphism.
-/
import Mathlib.Analysis.Normed.Operator.Banach
import Mathlib.Tactic

/-- The open mapping theorem: a surjective continuous linear map from a
    Banach space to a Banach space is an open map. -/
theorem equivalent_norms_from_one_sided_inequality
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [CompleteSpace E]
    [NormedAddCommGroup F] [NormedSpace ℝ F] [CompleteSpace F]
    (f : E →L[ℝ] F) (hsurj : Function.Surjective f) :
    IsOpenMap f :=
  f.isOpenMap hsurj
```

11. Пусть $A, B \in \mathcal{L}(X, X)$, причем существует $(I - AB)^{-1}$. Доказать, что существует $(I - BA)^{-1}$.
Указание. $(I - BA)^{-1} = I + B(I - AB)^{-1}A$.

Формализация на Lean 4

```lean
/-
  Section 15, Task 11.
  If (1 - ab) is invertible then (1 - ba) is invertible (Jacobson's lemma).

  The inverse is (1 - ba)⁻¹ = 1 + b·(1-ab)⁻¹·a.
-/
import Mathlib.Algebra.Ring.Units
import Mathlib.Tactic

variable {R : Type*} [Ring R]

/-- Jacobson's lemma: if (1 - a*b) is a unit, then (1 - b*a) is also a unit. -/
theorem isUnit_one_sub_ba_of_isUnit_one_sub_ab (a b : R) (h : IsUnit (1 - a * b)) :
    IsUnit (1 - b * a) := by
  obtain ⟨u, hu⟩ := h
  set c : R := ↑u⁻¹
  have huc : (1 - a * b) * c = 1 := by change (1 - a * b) * ↑u⁻¹ = 1; rw [← hu]; exact u.mul_inv
  have hcu : c * (1 - a * b) = 1 := by change ↑u⁻¹ * (1 - a * b) = 1; rw [← hu]; exact u.inv_mul
  -- abc = c - 1 (from (1-ab)c = 1)
  have habc : a * b * c = c - 1 := by
    have h : c - a * b * c = 1 := by rwa [sub_mul, one_mul] at huc
    rw [eq_sub_iff_add_eq, add_comm]; exact (sub_eq_iff_eq_add.mp h).symm
  -- cab = c - 1 (from c(1-ab) = 1)
  have hcab : c * a * b = c - 1 := by
    have h : c - c * a * b = 1 := by rw [mul_sub, mul_one, ← mul_assoc] at hcu; exact hcu
    rw [eq_sub_iff_add_eq, add_comm]; exact (sub_eq_iff_eq_add.mp h).symm
  -- Construct the unit: val = 1 - ba, inv = 1 + bca
  refine ⟨⟨1 - b * a, 1 + b * c * a, ?_, ?_⟩, rfl⟩
  · -- (1 - ba)(1 + bca) = 1
    have key : b * a * (b * c * a) = b * c * a - b * a := by
      have : b * a * (b * c * a) = b * (a * b * c) * a := by simp only [mul_assoc]
      rw [this, habc, mul_sub, mul_one, sub_mul]
    calc (1 - b * a) * (1 + b * c * a)
        = 1 + b * c * a - b * a - b * a * (b * c * a) := by noncomm_ring
```

```
      _ = 1 + b * c * a - b * a - (b * c * a - b * a) := by rw [key]
      _ = 1 := by noncomm_ring
  · -- (1 + bca)(1 - ba) = 1
    have key : b * c * a * (b * a) = b * c * a - b * a := by
      have : b * c * a * (b * a) = b * (c * a * b) * a := by simp only [mul_assoc]
      rw [this, hcab, mul_sub, mul_one, sub_mul]
    calc (1 + b * c * a) * (1 - b * a)
        = 1 - b * a + b * c * a - b * c * a * (b * a) := by noncomm_ring
      _ = 1 - b * a + b * c * a - (b * c * a - b * a) := by rw [key]
      _ = 1 := by noncomm_ring
```

16. СОПРЯЖЕННЫЕ ПРОСТРАНСТВА. РЕФЛЕКСИВНОСТЬ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение сопряженного пространства.
2. Каков общий вид линейных непрерывных функционалов в пространствах $\mathbb{R}^n$, H, $C[a,b]$, l^p, $L^p(a,b)$ где $1 < p < \infty$?
3. Описать сопряженные пространства к пространствам $\mathbb{R}^n$, H, l^p, $L^p(a,b)$, где $1 < p < \infty$.
4. Какое пространство называется вторым сопряженным? Описать естественное отображение $\pi : X \to X^{**}$ и дать определение рефлексивного пространства X. Привести примеры рефлексивных пространств.

ЗАДАЧИ

1. Описать сопряженное пространство к пространству $\mathbb{R}^n$ с нормой

   a) $||x|| = \max_{1 \le k \le n} |\xi_k|$;

   b) $||x|| = \sum_{k=1}^{n} |\xi_k|$;

   c) $||x|| = (\sum_{k=1}^{n} |\xi_k|^p)^{1/p}$, где $1 < p < \infty$, $x = (\xi_1, \ldots, \xi_n)$.

   Является ли $\mathbb{R}^n$ рефлексивным?
2. Описать сопряженное пространство к пространству l^1.
3. Описать сопряженное пространство к пространству c_0 сходящихся к нулю числовых последовательностей с нормой $||x|| = \max_{k \ge 1} |\xi_k|$. Рефлексивно ли пространство c_0?
4. Описать сопряженное пространство к пространству c сходящихся к конечному пределу последовательностей с нормой $||x|| = \sup_{k \ge 1} |\xi_k|$. Является ли пространство c рефлексивным?

5. Найти общий вид линейных непрерывных функционалов в пространстве s.

17. ТЕОРЕМА ХАНА-БАНАХА. СИЛЬНАЯ И СЛАБАЯ СХОДИМОСТИ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Сформулировать теорему Хана-Банаха и следствия из нее.
2. Дать определения сильной и слабой сходимости последовательности функционалов, определение слабой сходимости последовательности элементов.
3. Сформулировать критерий слабой сходимости последовательности функционалов и элементов.

ЗАДАЧИ

1. Найти все линейные непрерывные продолжения функционала $f(x) = 4\xi_1$, $x = \xi_1 \in \mathbb{R}^1$ на пространство $\mathbb{R}^3$ с сохранением нормы, если $\mathbb{R}^3$ рассматривается с нормой
   a) $||x|| = (\xi_1^2 + \xi_2^2 + \xi_3^2)^{1/2}$;
   b) $||x|| = |\xi_1| + |\xi_2| + |\xi_3|$;
   c) $||x|| = \max(|\xi_1|, |\xi_2|, |\xi_3|)$.
2. Найти все линейные непрерывные продолжения функционала $f(x) = \xi_1 + \xi_2$, $x \in \mathbb{R}^2$, на пространство l^p с сохранением нормы, если $||x|| = (|\xi_1|^p + |\xi_2|^p)^{1/p}$:
   a) $p = 1$;
   b) $1 < p < \infty$.

3. Доказать, что последовательность функционалов

$$F_n(x) = \int_{-\pi}^{\pi} x(t)e^{int}\,dt$$

слабо сходится в $L^2(-\pi, \pi)$, но сильной сходимости нет.

Формализация на Lean 4

```lean
/-
  Section 17, Task 3.
  Fₙ(x) = ∫₋π^π x(t)eⁱⁿᵗ dt converges weakly to 0 in L²(-π,π)*
  but not strongly (‖Fₙ‖ = √(2π) for all n by Parseval).
  Weak convergence: Fₙ(x) = √(2π)·x̂(n) → 0 by Riemann-Lebesgue lemma.

  We formalize the key ingredient: the Riemann–Lebesgue lemma, which
  states that Fourier coefficients tend to 0 at infinity.
-/
import Mathlib.Analysis.Fourier.RiemannLebesgueLemma
import Mathlib.Tactic

open MeasureTheory Filter

/-- Riemann–Lebesgue lemma: Fourier coefficients tend to 0 at infinity. -/
theorem fourier_functionals_weak_not_strong
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℂ E]
    (f : ℝ → E) :
    Tendsto (fun w : ℝ => ∫ v : ℝ, Real.fourierChar (-(v * w)) • f v)
      (cocompact ℝ) (nhds 0) :=
  Real.tendsto_integral_exp_smul_cocompact f
```

4. Покажите, что существует последовательность конечных линейных комбинаций функционалов $F_t(x) = x(t)$, сходящаяся слабо к функционалу $F(x) = \int_0^1 x(t)\,dt$ (функционалы определены в пространстве $C[0,1]$). Имеется ли последовательность, сходящаяся по норме?

Формализация на Lean 4

```
/-
  Section 17, Task 04.
  (a) ∃ sequence of finite linear combinations of Fₜ(x) = x(t)
      converging weakly to F(x) = ∫₀¹ x(t)dt.
      Proof: Riemann sums Sₙ(f) = (1/n)Σf(k/n) → ∫₀¹ f for each continuous f.

  (b) No norm-convergent sequence of such combinations.
      Witness: gₙ(t) = cos(2πnt) has gₙ(k/n) = 1 for all k,
      so Sₙ(gₙ) = 1, while ∫₀¹ gₙ = 0. Thus ‖Sₙ − F‖ ≥ 1 for all n.
-/
import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic
import Mathlib.Topology.UniformSpace.HeineCantor
import Mathlib.Analysis.SpecialFunctions.Integrals.Basic
import Mathlib.Tactic

open MeasureTheory Set Finset Filter Real
open scoped Topology

noncomputable section

-- ============================================================================
-- Part (a): Riemann sums converge to the integral for continuous functions
-- ============================================================================

/-- Left Riemann sum: Sₙ(f) = (1/n) Σ_{k=0}^{n-1} f(k/n).
    Each Sₙ is a linear combination of point evaluation functionals Fₜ(x) = x(t). -/
def riemannSum (f : ℝ → ℝ) (n : ℕ) : ℝ :=
  (∑ k ∈ range n, f ((k : ℝ) / n)) / n

/-- Riemann sums converge to the integral for continuous functions on [0,1].
    This provides the weak convergence: for each f ∈ C[0,1],
    the sequence of linear combinations Sₙ(f) → F(f) = ∫₀¹ f(t)dt. -/
theorem section17_task04a_riemann_sum_tendsto
    {f : ℝ → ℝ} (hf : ContinuousOn f (Icc 0 1)) :
    Tendsto (fun n => riemannSum f n) atTop (nhds (∫ x in (0 : ℝ)..1, f x)) := by
```

```
  rw [Metric.tendsto_atTop]
  intro ε hε
  -- Uniform continuity on compact [0,1]
  obtain ⟨δ, hδ, hδ_spec⟩ := Metric.uniformContinuousOn_iff.mp
    (isCompact_Icc.uniformContinuousOn_of_continuous hf) (ε / 2) (half_pos hε)
  obtain ⟨N, hN⟩ := exists_nat_gt (1 / δ)
  refine ⟨max 1 N, fun n hn => ?_⟩
  have hn1 : 0 < n := by omega
  have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn1
  have hne : (↑n : ℝ) ≠ 0 := hn'.ne'
  -- Key: 1/n < δ
  have h1n : 1 / (↑n : ℝ) < δ := by
    have hNn : 1 / δ < ↑n :=
      lt_of_lt_of_le hN (by exact_mod_cast show N ≤ n by omega)
    rwa [div_lt_iff₀ hδ, mul_comm, ← div_lt_iff₀ hn'] at hNn
  -- Integrability on each subinterval [k/n, (k+1)/n] ⊆ [0,1]
  have hint : ∀ k, k < n →
      IntervalIntegrable f volume ((↑k : ℝ) / ↑n) ((↑k + 1) / ↑n) := by
    intro k hk
    apply (hf.mono _).intervalIntegrable
    rw [Set.uIcc_of_le (div_le_div_of_nonneg_right
      (by linarith : (↑k : ℝ) ≤ ↑k + 1) hn'.le)]
    exact Icc_subset_Icc (div_nonneg (Nat.cast_nonneg k) hn'.le)
      ((div_le_one hn').mpr (by exact_mod_cast Nat.succ_le_of_lt hk))
  -- Decompose ∫₀¹ f = Σ ∫_{k/n}^{(k+1)/n} f
  have hdecomp : ∫ x in (0 : ℝ)..1, f x =
      ∑ k ∈ range n, ∫ x in ((↑k : ℝ) / ↑n)..((↑k + 1) / ↑n), f x := by
    have h := intervalIntegral.sum_integral_adjacent_intervals
      (a := fun k => (↑k : ℝ) / ↑n) (fun k hk => by
        change IntervalIntegrable f volume ((↑k : ℝ) / ↑n) ((↑(k + 1) : ℝ) / ↑n)
        rw [show (↑(k + 1) : ℝ) / ↑n = ((↑k : ℝ) + 1) / ↑n from by push_cast; ring]
        exact hint k hk)
    dsimp only at h
    simp only [Nat.cast_zero, zero_div, div_self hne] at h
    convert h.symm using 2
    push_cast; ring
  -- Each term: f(k/n)/n = ∫ f(k/n) dt on [k/n, (k+1)/n]
  have hconst : ∀ k, k < n →
      f ((↑k : ℝ) / ↑n) / ↑n =
        ∫ _ in ((↑k : ℝ) / ↑n)..((↑k + 1) / ↑n), f ((↑k : ℝ) / ↑n) := by
    intro k _
    rw [intervalIntegral.integral_const, smul_eq_mul,
```

```
      show ((↑k : ℝ) + 1) / ↑n - ↑k / ↑n = 1 / ↑n from by
       ↪  ring,
      one_div, mul_comm, ← div_eq_mul_inv]
-- Bound each term of the difference
have hbound : ∀ k ∈ range n,
    |f ((↑k : ℝ) / ↑n) / ↑n - ∫ x in ((↑k : ℝ) / ↑n)..((↑k +
    ↪  1) / ↑n), f x| ≤
      ε / 2 * (1 / ↑n) := by
  intro k hk
  have hk' : k < n := mem_range.mp hk
  rw [hconst k hk', ← intervalIntegral.integral_sub
   ↪  intervalIntegrable_const (hint k hk')]
  have hle : (↑k : ℝ) / ↑n ≤ ((↑k : ℝ) + 1) / ↑n :=
    div_le_div_of_nonneg_right (by linarith : (↑k : ℝ) ≤ ↑k
    ↪  + 1) hn'.le
  -- Use norm_integral bound
  have h_norm :=
   ↪  intervalIntegral.norm_integral_le_of_norm_le_const
    (a := (↑k : ℝ) / ↑n) (b := ((↑k : ℝ) + 1) / ↑n)
    (f := fun x => f ((↑k : ℝ) / ↑n) - f x) (C := ε / 2)
    ↪  (fun t ht => ?_)
  · rw [show |((↑k : ℝ) + 1) / ↑n - ↑k / ↑n| = 1 / ↑n from
   ↪  by
        rw [show ((↑k : ℝ) + 1) / ↑n - ↑k / ↑n = 1 / ↑n from
         ↪  by ring]
        exact abs_of_nonneg (div_nonneg one_pos.le hn'.le)]
         ↪  at h_norm
    rw [← Real.norm_eq_abs]; exact h_norm
  · -- ‖f(k/n) - f(t)‖ ≤ ε/2 for t in subinterval
    rw [Set.uIoc_of_le hle] at ht
    rw [Real.norm_eq_abs, abs_sub_comm]
    apply le_of_lt
    apply hδ_spec
    · -- t ∈ [0,1]
      exact ⟨le_trans (div_nonneg (Nat.cast_nonneg k)
       ↪  hn'.le) (le_of_lt ht.1),
             le_trans ht.2 ((div_le_one hn').mpr (by
              ↪  exact_mod_cast Nat.succ_le_of_lt hk'))⟩
    · -- k/n ∈ [0,1]
      exact ⟨div_nonneg (Nat.cast_nonneg k) hn'.le,
             (div_le_one hn').mpr (by exact_mod_cast
              ↪  hk'.le)⟩
    · -- dist t (k/n) < δ
      rw [Real.dist_eq, abs_of_nonneg (by linarith [ht.1])]
      calc t - (↑k : ℝ) / ↑n
          ≤ ((↑k : ℝ) + 1) / ↑n - ↑k / ↑n := by linarith
           ↪  [ht.2]
        _ = 1 / ↑n := by ring
        _ < δ := h1n
-- Assemble the bound
rw [Real.dist_eq, hdecomp]
```

```
  change |riemannSum f n - ∑ k ∈ range n, ∫ x in ((↑k : ℝ) / ↑n)..((↑k + 1) / ↑n), f x| < ε
  rw [show riemannSum f n = ∑ k ∈ range n, f ((↑k : ℝ) / ↑n) / ↑n from by
    unfold riemannSum
    exact Finset.sum_div (range n) (fun k => f ((↑k : ℝ) / ↑n)) ↑n]
  rw [← Finset.sum_sub_distrib]
  calc |∑ k ∈ range n, (f ((↑k : ℝ) / ↑n) / ↑n -
          ∫ x in ((↑k : ℝ) / ↑n)..((↑k + 1) / ↑n), f x)|
      ≤ ∑ k ∈ range n, |f ((↑k : ℝ) / ↑n) / ↑n -
          ∫ x in ((↑k : ℝ) / ↑n)..((↑k + 1) / ↑n), f x| :=
        Finset.abs_sum_le_sum_abs _ _
    _ ≤ ∑ _ ∈ range n, (ε / 2 * (1 / ↑n)) := Finset.sum_le_sum hbound
    _ = ε / 2 := by
        rw [Finset.sum_const, card_range, nsmul_eq_mul]
        field_simp
    _ < ε := half_lt_self hε

-- ==================================================================
-- Part (b): Riemann sums do not converge in operator norm
-- ==================================================================

/-- Riemann sum of cos(2πnt): since cos(2πk) = 1 for integer k,
    each f(k/n) = 1, so Sₙ = n/n = 1. -/
private lemma riemannSum_cos (n : ℕ) (hn : 0 < n) :
    riemannSum (fun x => cos (2 * π * ↑n * x)) n = 1 := by
  have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn
  simp only [riemannSum]
  have : ∀ k ∈ range n, cos (2 * π * ↑n * ((↑k : ℝ) / ↑n)) = 1 := by
    intro k _
    rw [show 2 * π * ↑n * ((↑k : ℝ) / ↑n) = ↑k * (2 * π) from by field_simp]
    exact cos_nat_mul_two_pi k
  rw [Finset.sum_congr rfl this, Finset.sum_const, card_range, nsmul_eq_mul, mul_one]
  exact div_self hn'.ne'

/-- Integral of cos(2πnt) over [0,1] is 0 for n ≥ 1. -/
private lemma integral_cos_two_pi_mul (n : ℕ) (hn : 0 < n) :
    ∫ x in (0 : ℝ)..1, cos (2 * π * ↑n * x) = 0 := by
  have hc : (2 * π * (↑n : ℝ)) ≠ 0 := by positivity
  rw [show (fun x : ℝ => cos (2 * π * ↑n * x)) = (fun x => cos ((2 * π * ↑n) * x)) from rfl,
    intervalIntegral.integral_comp_mul_left _ hc, mul_zero, mul_one, integral_cos,
```

```
      sin_zero, sub_zero]
  rw [show 2 * π * (↑n : ℝ) = ↑(2 * n) * π from by push_cast; ring,
      sin_nat_mul_pi, smul_zero]

/-- ‖Sₙ − F‖ ≥ 1 for all n ≥ 1: the witness cos(2πnt) has Sₙ(g) = 1
    but F(g) = ∫₀¹ cos(2πnt)dt = 0, while ‖g‖_∞ = 1.
    Hence Riemann sums do not converge to F in operator norm. -/
theorem section17_task04b_not_norm_convergent (n : ℕ) (hn : 0 < n) :
    |riemannSum (fun x => cos (2 * π * ↑n * x)) n -
      ∫ x in (0 : ℝ)..1, cos (2 * π * ↑n * x)| = 1 := by
  rw [riemannSum_cos n hn, integral_cos_two_pi_mul n hn, sub_zero, abs_one]
```

5. Пусть $F_n(x) = n \cdot \left[x(\frac{1}{n}) + x(-\frac{1}{n}) - 2x(0)\right]$. Показать, что последовательность $\{F_n\}$:

a) сходится по норме к $F(x) \equiv 0$ в $C^{(2)}[-1, 1]$;

b) сходится слабо к 0 в $C^{(1)}[-1, 1]$, но не сходится сильно;

c) не сходится даже слабо в $C[-1,1]$.

Указания. (i) Использовать формулу Тейлора. (ii) Оценить нормы $||F_n||$ снизу:

$$x_n'(t) = \begin{cases} 1, & \text{если } t > \frac{1}{n}, \\ nt, & \text{если } |t| \leq \frac{1}{n}, \\ -1, & \text{если } t < -\frac{1}{n}. \end{cases}$$

(iii) Показать, что последовательность $\{||F_n||\}$ неограничена.

Формализация на Lean 4

```
/-
  Section 17, Task 05.
  Fₙ(x) = n[x(1/n) + x(-1/n) - 2x(0)].

  (a) ‖Fₙ‖_{C²*} → 0: for f ∈ C², |Fₙ(f)| ≤ 2·sup|f''|/n.
  (b) Fₙ → 0 weakly in C¹[-1,1]*: for each C¹ function f,
   ↪  Fₙ(f) → 0.
  (c) Not weakly convergent in C[-1,1]*: Fₙ(√|·|) = 2√n → ∞.
-/
import Mathlib.Analysis.Calculus.Deriv.MeanValue
import Mathlib.Data.Real.Sqrt
import Mathlib.Tactic

open Filter Set Real
open scoped Topology

noncomputable section

/-- The functional Fₙ applied to f: Fₙ(f) = n(f(1/n) + f(-1/n)
 ↪  - 2f(0)). -/
def Fn (f : ℝ → ℝ) (n : ℕ) : ℝ :=
  (n : ℝ) * (f (1 / (n : ℝ)) + f (-(1 / (n : ℝ))) - 2 * f 0)

--
 ↪  ==========================================================================
-- Part (c): Fₙ(√|·|) = 2√n → ∞
--
 ↪  ==========================================================================

private lemma Fn_sqrt_abs (n : ℕ) (hn : 0 < n) :
    Fn (fun t => √|t|) n = 2 * √n := by
  have hn' : (0 : ℝ) < n := Nat.cast_pos.mpr hn
  simp only [Fn, abs_zero, sqrt_zero, mul_zero, sub_zero]
  rw [abs_of_nonneg (div_nonneg zero_le_one hn'.le)]
  rw [show |-(1 / (n : ℝ))| = 1 / n from by rw [abs_neg,
   ↪  abs_of_nonneg (div_nonneg zero_le_one hn'.le)]]
  rw [← two_mul (√(1 / ↑n)), one_div, sqrt_inv]
  -- Goal: ↑n * (2 * (√↑n)⁻¹) = 2 * √↑n
  -- Use: ↑n * (√↑n)⁻¹ = √↑n (i.e., n/√n = √n)
  have key : (↑n : ℝ) * (√↑n)⁻¹ = √↑n := by
    rw [← div_eq_mul_inv]; exact div_sqrt
  nlinarith

theorem section17_task05c_not_weak_in_C :
    Tendsto (fun n : ℕ => Fn (fun t => √|t|) n) atTop atTop :=
     ↪  by
  apply Filter.tendsto_atTop_atTop.mpr
  intro b
  refine ⟨max 1 (⌈(b / 2) ^ 2⌉₊ + 1), fun n hn => ?_⟩
  have hn1 : 0 < n := by omega
```

```
  rw [Fn_sqrt_abs n hn1]
  by_cases hb : b ≤ 0
  · linarith [mul_nonneg (two_pos.le) (sqrt_nonneg (n : ℝ))]
  · simp only [not_le] at hb
    have hb2 : (0 : ℝ) < b / 2 := by linarith
    have hsq : (b / 2) ^ 2 < ↑n := by
      have : n ≥ ⌈(b / 2) ^ 2⌉₊ + 1 := by omega
      calc (b / 2) ^ 2 ≤ ↑(⌈(b / 2) ^ 2⌉₊) := Nat.le_ceil _
        _ < ↑(⌈(b / 2) ^ 2⌉₊ + 1) := by push_cast; linarith
        _ ≤ (↑n : ℝ) := by exact_mod_cast ‹_›
    linarith [show b / 2 < √↑n from by rw [← sqrt_sq hb2.le]; exact sqrt_lt_sqrt (sq_nonneg _) hsq]

-- ==================================================================
-- Core MVT identity: Fₙ(f) = f'(c₁) - f'(c₂)
-- ==================================================================

private lemma Fn_eq_deriv_diff {f : ℝ → ℝ} {n : ℕ} (hn : 0 < n)
    (hf : DifferentiableOn ℝ f (Icc (-1) 1))
    (hfc : ContinuousOn f (Icc (-1) 1)) :
    ∃ c₁ ∈ Ioo 0 (1 / (n : ℝ)),
    ∃ c₂ ∈ Ioo (-(1 / (n : ℝ))) 0,
    Fn f n = deriv f c₁ - deriv f c₂ := by
  have hn' : (0 : ℝ) < n := Nat.cast_pos.mpr hn
  have hnn : (n : ℝ) ≠ 0 := ne_of_gt hn'
  have h1n_pos : (0 : ℝ) < 1 / ↑n := div_pos one_pos hn'
  have h1n : 1 / (n : ℝ) ≤ 1 := by rw [div_le_one hn']; exact_mod_cast (by omega : 1 ≤ n)
  have hI1 : Icc 0 (1 / (n : ℝ)) ⊆ Icc (-1 : ℝ) 1 := Icc_subset_Icc (by linarith) h1n
  have hI2 : Icc (-(1 / (n : ℝ))) 0 ⊆ Icc (-1 : ℝ) 1 := Icc_subset_Icc (neg_le_neg h1n) (by linarith)
  obtain ⟨c₁, hc₁, hc₁_eq⟩ := exists_deriv_eq_slope f h1n_pos
    (hfc.mono hI1) ((hf.mono hI1).mono Ioo_subset_Icc_self)
  obtain ⟨c₂, hc₂, hc₂_eq⟩ := exists_deriv_eq_slope f (by linarith : -(1 / (↑n : ℝ)) < 0)
    (hfc.mono hI2) ((hf.mono hI2).mono Ioo_subset_Icc_self)
  refine ⟨c₁, hc₁, c₂, hc₂, ?_⟩
  show ↑n * (f (1 / ↑n) + f (-(1 / ↑n)) - 2 * f 0) = deriv f c₁ - deriv f c₂
  -- From MVT: deriv f c₁ = (f(1/n) - f(0)) / (1/n - 0)
  have hc₁_s : deriv f c₁ = (f (1/↑n) - f 0) / (1/↑n) := by rwa [sub_zero] at hc₁_eq
  -- From MVT: deriv f c₂ = (f(0) - f(-1/n)) / (1/n)
  have hc₂_s : deriv f c₂ = (f 0 - f (-(1/↑n))) / (1/↑n) := by
    rwa [show (0 : ℝ) - -(1 / ↑n) = 1 / ↑n from by ring] at hc₂_eq
  have eq1 : f (1 / ↑n) - f 0 = deriv f c₁ * (1 / ↑n) := by
```

```
    rw [hc₁_s, div_mul_cancel₀ _ (ne_of_gt h1n_pos)]
  have eq2 : f 0 - f (-(1 / ↑n)) = deriv f c₂ * (1 / ↑n) := by
    rw [hc₂_s, div_mul_cancel₀ _ (ne_of_gt h1n_pos)]
  have h_sum : f (1 / ↑n) + f (-(1 / ↑n)) - 2 * f 0 = (deriv f
   ↪  c₁ - deriv f c₂) / ↑n := by
    rw [show (deriv f c₁ - deriv f c₂) / ↑n = deriv f c₁ *
     ↪  (1/↑n) - deriv f c₂ * (1/↑n) from by ring]
    linarith
  rw [h_sum, mul_div_cancel₀ _ hnn]

--
 ↪  ==================================================================
-- Part (a): C² estimate |Fₙ(f)| ≤ 2M/n via MVT applied three
 ↪  times
--
 ↪  ==================================================================

theorem section17_task05a_C2_bound
    {f : ℝ → ℝ} {M : ℝ}
    (hf : DifferentiableOn ℝ f (Icc (-1) 1))
    (hf' : DifferentiableOn ℝ (deriv f) (Icc (-1) 1))
    (hfc : ContinuousOn f (Icc (-1) 1))
    (hfc' : ContinuousOn (deriv f) (Icc (-1) 1))
    (hbound : ∀ x ∈ Icc (-(1 : ℝ)) 1, |deriv (deriv f) x| ≤ M)
    {n : ℕ} (hn : 0 < n) :
    |Fn f n| ≤ 2 * M / n := by
  have hn' : (0 : ℝ) < n := Nat.cast_pos.mpr hn
  have h1n : 1 / (n : ℝ) ≤ 1 := by rw [div_le_one hn'];
   ↪  exact_mod_cast (by omega : 1 ≤ n)
  obtain ⟨c₁, hc₁, c₂, hc₂, hkey⟩ := Fn_eq_deriv_diff hn hf
   ↪  hfc
  rw [hkey]
  have hlt : c₂ < c₁ := lt_trans hc₂.2 hc₁.1
  have hIc : Icc c₂ c₁ ⊆ Icc (-1 : ℝ) 1 :=
    Icc_subset_Icc (by linarith [hc₂.1, neg_le_neg h1n]) (by
     ↪  linarith [hc₁.2])
  -- Apply MVT a third time: to deriv f on [c₂, c₁]
  obtain ⟨c₃, hc₃, hc₃_eq⟩ := exists_deriv_eq_slope (deriv f)
   ↪  hlt
    (hfc'.mono hIc) ((hf'.mono hIc).mono Ioo_subset_Icc_self)
  -- deriv (deriv f) c₃ = (deriv f c₁ - deriv f c₂) / (c₁ -
   ↪  c₂)
  -- So |deriv f c₁ - deriv f c₂| = |deriv (deriv f) c₃| * |c₁
   ↪  - c₂|
  have hc₃_in : c₃ ∈ Icc (-1 : ℝ) 1 := hIc
   ↪  (Ioo_subset_Icc_self hc₃)
  have h_diff : deriv f c₁ - deriv f c₂ = deriv (deriv f) c₃ *
   ↪  (c₁ - c₂) := by
    have hsub : c₁ - c₂ ≠ 0 := sub_ne_zero.mpr hlt.ne'
    rw [hc₃_eq, div_mul_cancel₀ _ hsub]
  rw [h_diff, abs_mul]
```

```lean
  have hM : 0 ≤ M := le_trans (abs_nonneg _) (hbound 0 ⟨by linarith, by linarith⟩)
  have hc_diff_nn : 0 ≤ c₁ - c₂ := sub_nonneg.mpr hlt.le
  have hc_diff_le : c₁ - c₂ ≤ 2 / ↑n := by
    have h1 : c₁ - c₂ ≤ 1 / ↑n - (-(1 / ↑n)) :=
      sub_le_sub (le_of_lt hc₁.2) (le_of_lt hc₂.1)
    have h2 : (1 : ℝ) / ↑n - -(1 / ↑n) = 2 / ↑n := by ring
    linarith
  calc |deriv (deriv f) c₃| * |c₁ - c₂|
      ≤ M * |c₁ - c₂| := mul_le_mul_of_nonneg_right (hbound c₃ hc₃_in) (abs_nonneg _)
    _ = M * (c₁ - c₂) := by rw [abs_of_nonneg hc_diff_nn]
    _ ≤ M * (2 / ↑n) := mul_le_mul_of_nonneg_left hc_diff_le hM
    _ = 2 * M / ↑n := by ring

-- ============================================================================
-- Part (b): Fₙ(f) → 0 for C¹ functions
-- ============================================================================

theorem section17_task05b_weak_in_C1
    {f : ℝ → ℝ}
    (hf : DifferentiableOn ℝ f (Icc (-1) 1))
    (hfc : ContinuousOn f (Icc (-1) 1))
    (hfc' : ContinuousOn (deriv f) (Icc (-1) 1)) :
    Tendsto (fun n : ℕ => Fn f n) atTop (nhds 0) := by
  rw [Metric.tendsto_atTop]
  intro ε hε
  have h0 : (0 : ℝ) ∈ Icc (-1 : ℝ) 1 := ⟨by linarith, by linarith⟩
  obtain ⟨δ, hδ, hδ_spec⟩ := Metric.continuousWithinAt_iff.mp
    (hfc'.continuousWithinAt h0) (ε / 2) (half_pos hε)
  obtain ⟨N, hN⟩ := exists_nat_gt (1 / δ)
  refine ⟨max 1 N, fun n hn => ?_⟩
  have hn1 : 0 < n := by omega
  have hn' : (0 : ℝ) < n := Nat.cast_pos.mpr hn1
  have h1n : 1 / (n : ℝ) ≤ 1 := by rw [div_le_one hn']; exact_mod_cast (by omega : 1 ≤ n)
  obtain ⟨c₁, hc₁, c₂, hc₂, hkey⟩ := Fn_eq_deriv_diff hn1 hf hfc
  rw [hkey, dist_zero_right, Real.norm_eq_abs]
  have hI1 : Icc 0 (1 / (n : ℝ)) ⊆ Icc (-1 : ℝ) 1 := Icc_subset_Icc (by linarith) h1n
  have hI2 : Icc (-(1 / (n : ℝ))) 0 ⊆ Icc (-1 : ℝ) 1 := Icc_subset_Icc (neg_le_neg h1n) (by linarith)
  -- Key: dist c₁ 0 < δ and dist c₂ 0 < δ
  have hN_le : (N : ℝ) ≤ n := by exact_mod_cast (show N ≤ n by omega)
  have h_1n_lt_δ : 1 / (↑n : ℝ) < δ := by
```

```
      rw [div_lt_iff₀ hn']
      have h : 1 / δ < ↑n := lt_of_lt_of_le hN hN_le
      rw [div_lt_iff₀ hδ] at h
      linarith [mul_comm (↑n : ℝ) δ]
    have hc₁_dist : dist c₁ 0 < δ := by
      rw [Real.dist_eq, sub_zero, abs_of_pos hc₁.1]; linarith [hc₁.2]
    have hc₂_dist : dist c₂ 0 < δ := by
      rw [Real.dist_eq, sub_zero, abs_of_neg hc₂.2]; linarith [hc₂.1]
    have hd1 := hδ_spec (hI1 (Ioo_subset_Icc_self hc₁)) hc₁_dist
    have hd2 := hδ_spec (hI2 (Ioo_subset_Icc_self hc₂)) hc₂_dist
    rw [Real.dist_eq] at hd1 hd2
    calc |deriv f c₁ - deriv f c₂|
        ≤ |deriv f c₁ - deriv f 0| + |deriv f 0 - deriv f c₂| := by
          rw [show deriv f c₁ - deriv f c₂ =
            (deriv f c₁ - deriv f 0) + (deriv f 0 - deriv f c₂) from by ring]
          exact abs_add_le _ _
      _ < ε / 2 + ε / 2 := by linarith [abs_sub_comm (deriv f c₂) (deriv f 0)]
      _ = ε := add_halves ε
```

6. Пусть $F_n(x) = n^2 \cdot \left[x(\frac{1}{n}) + x(-\frac{1}{n}) - 2x(0)\right]$, $F(x) = x''(0)$. Показать, что последовательность $\{F_n\}$:

   a) сходится к F по норме в $C^{(3)}[-1,1]$;

   b) сходится слабо к F в $C^{(2)}[-1,1]$, но не сходится сильно;

c) не сходится даже слабо в $C^{(1)}[-1,1]$.

Формализация на Lean 4

```
/-
  Section 17, Task 06.
  Gₙ(f) = n²[f(1/n) + f(-1/n) - 2f(0)].

  (a) |Gₙ(f) - f''(0)| ≤ M/n for C³ (norm convergence in C³*).
  (b) Gₙ(f) → f''(0) for C² functions (weak convergence in C²*).
  (c) Not weakly convergent in C¹*: Gₙ(|·|^{3/2}) = 2√n → ∞.
-/
import FunAn.Section17_HahnBanach.Task05
import Mathlib.Tactic

open Filter Set Real
open scoped Topology

noncomputable section

/-- The functional Gₙ: Gₙ(f) = n²(f(1/n) + f(-1/n) - 2f(0)).
    Second-order symmetric difference quotient (approximates f''(0)). -/
def Gn (f : ℝ → ℝ) (n : ℕ) : ℝ :=
  (n : ℝ) ^ 2 * (f (1 / (n : ℝ)) + f (-(1 / (n : ℝ))) - 2 * f 0)

-- ==============================================================================
-- Part (c): Gₙ(|·|^{3/2}) = 2√n → ∞ (not weak in C¹)
-- ==============================================================================

private lemma Gn_abs_three_halves (n : ℕ) (hn : 0 < n) :
    Gn (fun t => |t| * √|t|) n = 2 * √↑n := by
  have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn
  simp only [Gn, abs_zero, mul_zero, sqrt_zero, sub_zero]
  have h1 : |1 / (↑n : ℝ)| = 1 / ↑n := abs_of_nonneg (by positivity)
  rw [h1, show |-(1 / (↑n : ℝ))| = 1 / ↑n from by rw [abs_neg]; exact h1, ← two_mul,
    one_div, sqrt_inv]
  have : (↑n : ℝ) ≠ 0 := ne_of_gt hn'
  have : √(↑n : ℝ) ≠ 0 := ne_of_gt (sqrt_pos.mpr hn')
  field_simp
  nlinarith [sq_sqrt hn'.le]

/-- Gₙ(|·|^{3/2}) = 2√n → ∞: witnesses non-convergence.
    The function |x|^{3/2} = |x|·√|x| is C¹ (derivative (3/2)√|x|·sgn(x)),
```

```
  so Gₙ is not weakly convergent in C¹[-1,1]*. -/
theorem section17_task06c_not_weak_in_C1 :
    Tendsto (fun n : ℕ => Gn (fun t => |t| * √|t|) n) atTop atTop := by
  apply Filter.tendsto_atTop_atTop.mpr
  intro b
  refine ⟨max 1 (⌈(b / 2) ^ 2⌉₊ + 1), fun n hn => ?_⟩
  have hn1 : 0 < n := by omega
  rw [Gn_abs_three_halves n hn1]
  by_cases hb : b ≤ 0
  · linarith [mul_nonneg two_pos.le (sqrt_nonneg (↑n : ℝ))]
  · push Not at hb
    have hb2 : (0 : ℝ) < b / 2 := by linarith
    have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn1
    have hsq : (b / 2) ^ 2 < (↑n : ℝ) := by
      have : n ≥ ⌈(b / 2) ^ 2⌉₊ + 1 := by omega
      calc (b / 2) ^ 2 ≤ ↑(⌈(b / 2) ^ 2⌉₊) := Nat.le_ceil _
        _ < ↑(⌈(b / 2) ^ 2⌉₊ + 1) := by push_cast; linarith
        _ ≤ (↑n : ℝ) := by exact_mod_cast ‹_›
    linarith [show b / 2 < √(↑n : ℝ) from by
      rw [← sqrt_sq hb2.le]; exact sqrt_lt_sqrt (sq_nonneg _) hsq]

-- ==========================================================================
-- Part (b): Gₙ(f) → f''(0) for C² functions (via Cauchy MVT + MVT)
-- ==========================================================================

/-- Key identity: Gₙ(f) = f''(d) for some d ∈ (-1/n, 1/n).
    Step 1: Cauchy MVT on F(x)=f(x)+f(-x), G(x)=x² gives Gₙ(f) = (f'(c)-f'(-c))/(2c).
    Step 2: MVT on f' on [-c,c] gives (f'(c)-f'(-c))/(2c) = f''(d). -/
private lemma Gn_eq_second_deriv {f : ℝ → ℝ} {n : ℕ} (hn : 0 < n)
    (hf : DifferentiableOn ℝ f (Icc (-1) 1))
    (hf' : DifferentiableOn ℝ (deriv f) (Icc (-1) 1))
    (hfc : ContinuousOn f (Icc (-1) 1))
    (hfc' : ContinuousOn (deriv f) (Icc (-1) 1)) :
    ∃ d ∈ Ioo (-(1 / (↑n : ℝ))) (1 / (↑n : ℝ)),
      Gn f n = deriv (deriv f) d := by
  have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn
  have hne : (↑n : ℝ) ≠ 0 := ne_of_gt hn'
  have h1n_pos : (0 : ℝ) < 1 / ↑n := div_pos one_pos hn'
  have h1n_le : 1 / (↑n : ℝ) ≤ 1 := by
    rw [div_le_one hn']; exact_mod_cast (by omega : 1 ≤ n)
  -- Subset inclusions
  have hI : Icc 0 (1 / (↑n : ℝ)) ⊆ Icc (-1 : ℝ) 1 :=
    Icc_subset_Icc (by linarith) h1n_le
```

```lean
  have hIn : Icc (-(1 / (↑n : ℝ))) 0 ⊆ Icc (-1 : ℝ) 1 :=
    Icc_subset_Icc (neg_le_neg h1n_le) (by linarith)
  -- F(x) = f(x) + f(-x) is continuous on [0, 1/n]
  have hFc : ContinuousOn (fun x => f x + f (-x)) (Icc 0 (1 / ↑n)) :=
    (hfc.mono hI).add ((hfc.mono hIn).comp continuousOn_neg
      (fun x hx => ⟨neg_le_neg hx.2, by linarith [hx.1]⟩))
  -- F differentiable on (0, 1/n)
  have hFd : DifferentiableOn ℝ (fun x => f x + f (-x)) (Ioo 0 (1 / ↑n)) :=
    ((hf.mono hI).mono Ioo_subset_Icc_self).add
      (((hf.mono hIn).mono Ioo_subset_Icc_self).comp
        (differentiable_neg.differentiableOn)
        (fun x hx => ⟨by linarith [hx.2], neg_lt_zero.mpr hx.1⟩))
  -- Step 1: Apply Cauchy's MVT
  obtain ⟨c, hc, hc_eq⟩ := exists_ratio_deriv_eq_ratio_slope
    (fun x => f x + f (-x)) h1n_pos hFc hFd
    (fun x => x ^ 2) (by fun_prop) (differentiable_pow 2).differentiableOn
  have hc_pos : (0 : ℝ) < c := hc.1
  have hc_lt : c < 1 / ↑n := hc.2
  have h2c_pos : (0 : ℝ) < 2 * c := by linarith
  have h2c_ne : (2 * c : ℝ) ≠ 0 := ne_of_gt h2c_pos
  -- c and -c are in (-1, 1)
  have hc11 : c ∈ Ioo (-1 : ℝ) 1 := ⟨by linarith, by linarith [h1n_le]⟩
  have hnc11 : -c ∈ Ioo (-1 : ℝ) 1 := ⟨by nlinarith [h1n_le], by linarith⟩
  have hda : DifferentiableAt ℝ f c :=
    hf.differentiableAt (Icc_mem_nhds hc11.1 hc11.2)
  have hdna : DifferentiableAt ℝ f (-c) :=
    hf.differentiableAt (Icc_mem_nhds hnc11.1 hnc11.2)
  -- Compute HasDerivAt for F and G at c (to rewrite deriv in hc_eq)
  have hF_at : HasDerivAt (fun x => f x + f (-x)) (deriv f c - deriv f (-c)) c := by
    have h1 := hda.hasDerivAt
    have h2 : HasDerivAt (fun x => f (-x)) (-deriv f (-c)) c := by
      have := hdna.hasDerivAt.comp c (hasDerivAt_neg c)
      simpa [Function.comp_def] using this
    exact h1.add h2
  have hG_at : HasDerivAt (fun x : ℝ => x ^ 2) (2 * c) c := by
    have := hasDerivAt_pow 2 c; simpa [pow_one] using this
  -- Replace derivatives in Cauchy MVT equation
  rw [hF_at.deriv, hG_at.deriv] at hc_eq
  -- hc_eq now has form involving function values and derivative values
  -- Clean it up to: (1/n)^2 * (f'(c)-f'(-c)) = (f(1/n)+f(-1/n)-2f(0)) * 2c
```

```
  have h_cauchy : (1 / ↑n : ℝ) ^ 2 * (deriv f c - deriv f (-c)) =
      (f (1 / ↑n) + f (-(1 / ↑n)) - 2 * f 0) * (2 * c) := by
    have := hc_eq
    simp only [neg_zero] at this
    nlinarith
  -- Step 2: Apply MVT to deriv f on [-c, c]
  have hIc : Icc (-c) c ⊆ Icc (-1 : ℝ) 1 :=
    Icc_subset_Icc (by linarith [hnc11.1]) (by linarith [hc11.2])
  obtain ⟨d, hd, hd_eq⟩ := exists_deriv_eq_slope (deriv f) (by linarith : -c < c)
    (hfc'.mono hIc) ((hf'.mono hIc).mono Ioo_subset_Icc_self)
  -- hd_eq: deriv (deriv f) d = (deriv f c - deriv f (-c)) / (c - (-c))
  -- Combine: f'(c)-f'(-c) = f''(d) * 2c, substitute into h_cauchy
  have h_sub : c - (-c) = 2 * c := by ring
  have h_diff_eq : deriv f c - deriv f (-c) = deriv (deriv f) d * (2 * c) := by
    rw [hd_eq, h_sub, div_mul_cancel₀ _ h2c_ne]
  -- From h_cauchy with substitution: (1/n)^2 * f''(d) * 2c = S * 2c
  -- Cancel 2c: (1/n)^2 * f''(d) = S
  -- So Gn = n^2 * S = n^2 * (1/n)^2 * f''(d) = f''(d)
  refine ⟨d, ⟨by linarith [hd.1], by linarith [hd.2]⟩, ?_⟩
  simp only [Gn]
  have h_S : f (1 / ↑n) + f (-(1 / ↑n)) - 2 * f 0 =
      (1 / ↑n) ^ 2 * deriv (deriv f) d := by
    have h := h_cauchy
    rw [h_diff_eq] at h
    -- h: (1/n)^2 * (f''(d) * 2c) = S * 2c
    nlinarith
  rw [h_S]
  have : (↑n : ℝ) ^ 2 * ((1 / ↑n) ^ 2 * deriv (deriv f) d) = deriv (deriv f) d := by
    field_simp
  linarith

/-- Weak convergence in C²*: Gₙ(f) → f''(0) for each C² function.
    Proof: Gₙ(f) = f''(dₙ) with dₙ → 0, so result follows from continuity of f''. -/
theorem section17_task06b_weak_in_C2
    {f : ℝ → ℝ}
    (hf : DifferentiableOn ℝ f (Icc (-1) 1))
    (hf' : DifferentiableOn ℝ (deriv f) (Icc (-1) 1))
    (hfc : ContinuousOn f (Icc (-1) 1))
    (hfc' : ContinuousOn (deriv f) (Icc (-1) 1))
    (hfc'' : ContinuousOn (deriv (deriv f)) (Icc (-1) 1)) :
    Tendsto (fun n : ℕ => Gn f n) atTop (nhds (deriv (deriv f) 0)) := by
```

```lean
  rw [Metric.tendsto_atTop]
  intro ε hε
  have h0_mem : (0 : ℝ) ∈ Icc (-1 : ℝ) 1 := ⟨by linarith, by linarith⟩
  obtain ⟨δ, hδ, hδ_spec⟩ := Metric.continuousWithinAt_iff.mp
    (hfc''.continuousWithinAt h0_mem) ε hε
  obtain ⟨N, hN⟩ := exists_nat_gt (1 / δ)
  refine ⟨max 1 N, fun n hn => ?_⟩
  have hn1 : 0 < n := by omega
  have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn1
  have h1n_le : 1 / (↑n : ℝ) ≤ 1 := by
    rw [div_le_one hn']; exact_mod_cast (by omega : 1 ≤ n)
  obtain ⟨d, hd, hGn_eq⟩ := Gn_eq_second_deriv hn1 hf hf' hfc hfc'
  rw [hGn_eq]
  have hd_mem : d ∈ Icc (-1 : ℝ) 1 :=
    ⟨by linarith [hd.1, neg_le_neg h1n_le], by linarith [hd.2, h1n_le]⟩
  have hN_le : (N : ℝ) ≤ ↑n := by exact_mod_cast (show N ≤ n by omega)
  have h_1n_lt_δ : 1 / (↑n : ℝ) < δ := by
    rw [div_lt_iff₀ hn']
    have : 1 / δ < ↑n := lt_of_lt_of_le hN hN_le
    rwa [div_lt_iff₀ hδ, mul_comm] at this
  exact hδ_spec hd_mem (by
    rw [Real.dist_eq, sub_zero]
    exact abs_lt.mpr ⟨by linarith [hd.1], by linarith [hd.2]⟩)

-- ============================================================================
-- Part (a): |Gₙ(f) - f''(0)| ≤ M/n for C³ (norm convergence in C³*)
-- ============================================================================

/-- C³ bound: |Gₙ(f) - f''(0)| ≤ M/n where M = sup|f'''|.
    From key identity Gₙ(f) = f''(d) with |d| < 1/n, MVT gives
    |f''(d) - f''(0)| ≤ |f'''(ξ)|·|d| ≤ M/n. -/
theorem section17_task06a_C3_bound
    {f : ℝ → ℝ} {M : ℝ}
    (hf : DifferentiableOn ℝ f (Icc (-1) 1))
    (hf' : DifferentiableOn ℝ (deriv f) (Icc (-1) 1))
    (hf'' : DifferentiableOn ℝ (deriv (deriv f)) (Icc (-1) 1))
    (hfc : ContinuousOn f (Icc (-1) 1))
    (hfc' : ContinuousOn (deriv f) (Icc (-1) 1))
    (hfc'' : ContinuousOn (deriv (deriv f)) (Icc (-1) 1))
    (hbound : ∀ x ∈ Icc (-(1 : ℝ)) 1, |deriv (deriv (deriv f)) x| ≤ M)
    {n : ℕ} (hn : 0 < n) :
    |Gn f n - deriv (deriv f) 0| ≤ M / ↑n := by
  have hn' : (0 : ℝ) < ↑n := Nat.cast_pos.mpr hn
```

```
  have h1n_le : 1 / (↑n : ℝ) ≤ 1 := by
    rw [div_le_one hn']; exact_mod_cast (by omega : 1 ≤ n)
  have h0_mem : (0 : ℝ) ∈ Icc (-1 : ℝ) 1 := ⟨by linarith, by linarith⟩
  have hM_nn : 0 ≤ M := le_trans (abs_nonneg _) (hbound 0 h0_mem)
  obtain ⟨d, hd, hGn_eq⟩ := Gn_eq_second_deriv hn hf hf' hfc hfc'
  rw [hGn_eq]
  have hd_mem : d ∈ Icc (-1 : ℝ) 1 :=
    ⟨by linarith [hd.1, neg_le_neg h1n_le], by linarith [hd.2, h1n_le]⟩
  -- Apply MVT to f'' between 0 and d to bound |f''(d) - f''(0)|
  by_cases hd0 : 0 ≤ d
  · -- Case d ≥ 0
    rcases eq_or_lt_of_le hd0 with rfl | hd_pos
    · simp [div_nonneg hM_nn hn'.le]
    · have hI0d : Icc 0 d ⊆ Icc (-1 : ℝ) 1 :=
        Icc_subset_Icc (by linarith) (by linarith [hd.2, h1n_le])
      obtain ⟨ξ, hξ, hξ_eq⟩ := exists_deriv_eq_slope (deriv (deriv f)) hd_pos
        (hfc''.mono hI0d) ((hf''.mono hI0d).mono Ioo_subset_Icc_self)
      rw [sub_zero] at hξ_eq
      have hξ_mem : ξ ∈ Icc (-1 : ℝ) 1 := hI0d (Ioo_subset_Icc_self hξ)
      have h_diff : deriv (deriv f) d - deriv (deriv f) 0 =
          deriv (deriv (deriv f)) ξ * d := by
        rw [hξ_eq]; field_simp
      rw [h_diff, abs_mul]
      calc |deriv (deriv (deriv f)) ξ| * |d|
          ≤ M * |d| := by apply mul_le_mul_of_nonneg_right (hbound ξ hξ_mem) (abs_nonneg _)
        _ = M * d := by rw [abs_of_nonneg hd0]
        _ ≤ M * (1 / ↑n) := by apply mul_le_mul_of_nonneg_left (le_of_lt hd.2) hM_nn
        _ = M / ↑n := by ring
  · -- Case d < 0
    push Not at hd0
    have hId0 : Icc d 0 ⊆ Icc (-1 : ℝ) 1 :=
      Icc_subset_Icc (by linarith [hd.1, neg_le_neg h1n_le]) (by linarith)
    obtain ⟨ξ, hξ, hξ_eq⟩ := exists_deriv_eq_slope (deriv (deriv f)) hd0
      (hfc''.mono hId0) ((hf''.mono hId0).mono Ioo_subset_Icc_self)
    have hξ_mem : ξ ∈ Icc (-1 : ℝ) 1 := hId0 (Ioo_subset_Icc_self hξ)
    have h_diff : deriv (deriv f) d - deriv (deriv f) 0 =
```

```
        deriv (deriv (deriv f)) ξ * d := by
      have hne : (0 : ℝ) - d ≠ 0 := sub_ne_zero.mpr (ne_of_gt hd0)
      rw [show deriv (deriv f) d - deriv (deriv f) 0 =
        -(deriv (deriv f) 0 - deriv (deriv f) d) from by ring, hξ_eq]
      field_simp; ring
    rw [h_diff, abs_mul]
    calc |deriv (deriv (deriv f)) ξ| * |d|
        ≤ M * |d| := by apply mul_le_mul_of_nonneg_right (hbound ξ hξ_mem) (abs_nonneg _)
      _ = M * (-d) := by rw [abs_of_neg hd0]
      _ ≤ M * (1 / ↑n) := by apply mul_le_mul_of_nonneg_left _ hM_nn; linarith [hd.1]
      _ = M / ↑n := by ring
```

7. Доказать, что слабая сходимость в l^2 сильнее покоординатной сходимости.

Формализация на Lean 4

```
/-
  Section 17, Task 07.
  Weak convergence in □² implies coordinatewise convergence.

  Proof: x(k) = ⟪x, eₖ⟫ where eₖ = lp.single 2 k 1.
  Weak convergence applied to y = eₖ gives xₙ(k) → x₀(k).
-/
import Mathlib.Analysis.InnerProductSpace.l2Space
import Mathlib.Tactic

open scoped InnerProductSpace
open Filter

private theorem real_inner_one (a : ℝ) : ⟪a, (1 : ℝ)⟫_ℝ = a := by
  simp [Inner.inner]

/-- Weak convergence in □² implies coordinatewise convergence. -/
theorem weak_implies_coordinatewise_l2
    {x : ℕ → lp (fun _ : ℕ => ℝ) 2} {x₀ : lp (fun _ : ℕ => ℝ) 2}
    (hweak : ∀ y : lp (fun _ : ℕ => ℝ) 2,
      Tendsto (fun n => ⟪x n, y⟫_ℝ) atTop (nhds ⟪x₀, y⟫_ℝ))
    (k : ℕ) :
    Tendsto (fun n => (x n : ℕ → ℝ) k) atTop (nhds ((x₀ : ℕ → ℝ) k)) := by
  have h := hweak (lp.single 2 k (1 : ℝ))
  simp only [lp.inner_single_right, real_inner_one] at h
  exact h
```

8. Какие из последовательностей сходятся сильно, слабо в l^2?

   a) $x_n = (1, \frac{1}{2}, \dots, \frac{1}{n}, 0, \dots)$;
   b) $x_n = (\underbrace{0, \dots, 0}_{n-1}, 1, 0, \dots)$;
   c) $x_n = (0, 0, \dots, 0, 1, \frac{1}{2}, \frac{1}{3}, \dots)$ (начало с n-го места);
   d) $x_n = (0, \dots, 0, \frac{1}{n+1}, \frac{1}{n+2}, \dots)$;
   e) $x_n = (\underbrace{1, \dots, 1}_{n}, \frac{1}{n}, \frac{1}{n+1}, \dots)$;
   f) $x_n = (\frac{1}{n}, \frac{1}{n+1}, \dots)$.

9. Исследовать на сильную и слабую сходимости следующие последовательности в $L^2(0,1)$:

   a) $x_n(t) = t^n$;
   b) $x_n(t) = \begin{cases} \sqrt{n}, & \text{если } t \in [0, \frac{1}{n}], \\ 0, & \text{если } t \in (\frac{1}{n}, 1]; \end{cases}$
   c) $x_n(t) = e^{i\sqrt{n}t}$;
   d) $x_n(t) = \sin(t \ln n)$;
   e) $x_n(t) = \cos(n^2 t)$;
   f) $x_n(t) = \sin(e^n t)$;
   g) $x_n(t) = \begin{cases} \sqrt{n} \cdot (1 - nt), & \text{если } t \in [0, \frac{1}{n}), \\ 0, & \text{если } t \in [\frac{1}{n}, 1]; \end{cases}$
   h) $x_n(t) = t^n - t^{n+1}$;
   i) $x_n(t) = \begin{cases} 2n(1 - nt), & \text{если } t \in [0, \frac{1}{n}), \\ 0, & \text{если } t \in [\frac{1}{n}, 1]. \end{cases}$

10. Пусть последовательность $\{x_n\} \subset H$ слабо сходится к x. Доказать, что $\{x_n\}$ будет сходиться сильно при выполнении одного из условий:

    a) $||x_n|| \to ||x||$;
    b) $||x_n|| \leq ||x|| \; \forall n > N_0$;

c) $\lim_{n\to\infty} ||x_n|| \leq ||x||$.

Формализация на Lean 4

```
/-
  Section 17, Task 10.
  In a Hilbert space H, weak convergence xₙ ⇀ x plus ‖xₙ‖ → ‖x‖
  implies strong convergence xₙ → x.
  Proof: ‖xₙ - x‖² = ‖xₙ‖² - 2Re⟨xₙ,x⟩ + ‖x‖² → ‖x‖² - 2‖x‖² + ‖x‖² = 0.
  (Radon-Riesz property, which holds in all uniformly convex spaces.)
-/
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Tactic

open scoped InnerProductSpace
open Filter

/-- Radon–Riesz property in a real inner product space: weak convergence
    plus norm convergence implies strong convergence. -/
theorem weak_plus_norm_implies_strong
    {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
    {x : ℕ → H} {x₀ : H}
    (hweak : ∀ y : H, Tendsto (fun n => ⟪x n, y⟫_ℝ) atTop (nhds ⟪x₀, y⟫_ℝ))
    (hnorm : Tendsto (fun n => ‖x n‖) atTop (nhds ‖x₀‖)) :
    Tendsto x atTop (nhds x₀) := by
  rw [tendsto_iff_norm_sub_tendsto_zero, Metric.tendsto_atTop]
  intro ε hε
  -- Show ‖x n - x₀‖² → 0
  have hnorm_sq : Tendsto (fun n => ‖x n‖ ^ 2) atTop (nhds (‖x₀‖ ^ 2)) := hnorm.pow 2
  have hinner : Tendsto (fun n => ⟪x n, x₀⟫_ℝ) atTop (nhds (‖x₀‖ ^ 2)) := by
    have := hweak x₀; rwa [real_inner_self_eq_norm_sq] at this
  have hsq : Tendsto (fun n => ‖x n - x₀‖ ^ 2) atTop (nhds 0) := by
    have hcomb : Tendsto (fun n => ‖x n‖ ^ 2 - 2 * ⟪x n, x₀⟫_ℝ + ‖x₀‖ ^ 2)
        atTop (nhds (‖x₀‖ ^ 2 - 2 * ‖x₀‖ ^ 2 + ‖x₀‖ ^ 2)) :=
      (hnorm_sq.sub (hinner.const_mul 2)).add tendsto_const_nhds
    rw [show ‖x₀‖ ^ 2 - 2 * ‖x₀‖ ^ 2 + ‖x₀‖ ^ 2 = 0 from by ring] at hcomb
    exact hcomb.congr (fun n => (norm_sub_sq_real (x n) x₀).symm)
  -- Extract N from hsq at ε²
```

```
  obtain ⟨N, hN⟩ := (Metric.tendsto_atTop.mp hsq) (ε ^ 2) (by
   ↪  positivity)
  refine ⟨N, fun n hn => ?_⟩
  have h1 := hN n hn
  simp only [dist_zero_right] at h1
  rw [Real.norm_of_nonneg (sq_nonneg _)] at h1
  simp only [dist_zero_right]
  rw [Real.norm_of_nonneg (norm_nonneg _)]
  nlinarith [sq_nonneg ‖x n - x₀‖, sq_nonneg ε, sq_abs ‖x n -
   ↪  x₀‖]
```

18. СОПРЯЖЕННЫЕ ОПЕРАТОРЫ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определения сопряженного оператора к линейному непрерывному оператору в нормированном и гильбертовом пространствах.

2. Какая связь между сопряженным оператором в H и сопряженным оператором в нормированном пространстве, когда в качестве нормированного пространства рассматривается H?

3. Перечислить свойства сопряженного оператора.

ЗАДАЧИ

1. Найти сопряженный оператор к оператору $A : \mathbb{R}^n \to \mathbb{R}^m$, определяемому равенством
$$\begin{pmatrix} \eta_1 \\ \eta_2 \\ \dots \\ \eta_m \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} \begin{pmatrix} \xi_1 \\ \xi_2 \\ \dots \\ \xi_n \end{pmatrix}$$

2. Найти сопряженный оператор к проектору $P : H \to L$, где L — замкнутое подпространство в H.

3. Найти сопряженный оператор к оператору $A : X \to X$, если

   a) $X = l^2$, $Ax = (0, \xi_1, \xi_2, \dots)$;
   b) $X = l^2$, $Ax = (\alpha_1\xi_1, \alpha_2\xi_2, \dots)$;
   c) $X = l^1$, $Ax = (\xi_1, \dots, \xi_n, 0, 0, \dots)$;
   d) $X = l^p$, $Ax = (\underbrace{0, \dots, 0}_{n}, \xi_1, 0, 0, \dots)$, $1 \le p < \infty$;
   e) $X = l^p$, $Ax = (\xi_2, \xi_1, \xi_4, \xi_3, \dots)$;
   f) $X = L^2(0, 1)$, $(Ax)(t) = \int_0^t x(\xi)\, d\xi$;
   g) $X = L^2(0, 1)$, $(Ax)(t) = x(t^\alpha)$;

h) $X = L^2(0,1)$, $(A_\lambda x)(t) = \begin{cases} x(t), & \text{если } 0 \le t \le \lambda, \\ 0, & \text{если } \lambda < t \le 1 \end{cases}$,
где $\lambda \in (0,1)$ — фиксированное число;

i) $X = H$, $Ax = (x, y) \cdot z$, где $y, z \in H$ фиксированы;

j) $X = L^2(0,1)$, $(Ax)(t) = \int_0^1 t \cdot x(s)\, ds$.

4. Пусть оператор $\Phi : \mathcal{L}(X, Y) \to \mathcal{L}(Y^*, X^*)$ определяется равенством $\Phi(A) = A^*$, где X, Y — нормированные пространства. Доказать, что Φ непрерывен.

Формализация на Lean 4

```
/-
  Section 18, Task 4.
  The map Φ : L(X,Y) → L(Y*,X*) defined by Φ(A) = A* is a continuous
  linear operator. In fact it is an isometry: ‖A*‖ = ‖A‖.
  This follows from ‖A*‖ = sup{|A*f(x)| : ‖f‖≤1, ‖x‖≤1}
  = sup{|f(Ax)| : ‖f‖≤1, ‖x‖≤1} = sup{‖Ax‖ : ‖x‖≤1} = ‖A‖
  by Hahn-Banach.

  In the Hilbert space setting, the adjoint is a linear isometry equivalence
  (Mathlib: ContinuousLinearMap.adjoint).
-/
import Mathlib.Analysis.InnerProductSpace.Adjoint
import Mathlib.Tactic

variable {𝕜 : Type*} [RCLike 𝕜]
variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace 𝕜 E] [CompleteSpace E]
variable {F : Type*} [NormedAddCommGroup F] [InnerProductSpace 𝕜 F] [CompleteSpace F]

/-- The adjoint map A ↦ A† is an isometry: ‖A†‖ = ‖A‖. -/
theorem adjoint_map_isometry (A : E →L[𝕜] F) :
    ‖ContinuousLinearMap.adjoint A‖ = ‖A‖ :=
  ContinuousLinearMap.adjoint.norm_map A
```

5. Доказать, что если $A \in \mathcal{L}(X, Y)$ и $A^{-1} \in \mathcal{L}(Y, X)$, то существует $(A^*)^{-1} \in \mathcal{L}(X^*, Y^*)$, причём $(A^*)^{-1} = (A^{-1})^*$.

Формализация на Lean 4

```
/-
  Section 18, Task 5.
  If A is invertible, then A* is invertible and (A*)⁻¹ = (A⁻¹)*.

  In Mathlib, for a linear equivalence f : M₁ ≃ₗ[R] M₂,
  `LinearEquiv.dualMap_symm` states: (dualMap f).symm = dualMap f.symm.
  This is exactly (A*)⁻¹ = (A⁻¹)*.
-/
import Mathlib.LinearAlgebra.Dual.Defs
import Mathlib.Tactic

variable {R M₁ M₂ : Type*} [CommSemiring R]
  [AddCommMonoid M₁] [Module R M₁] [AddCommMonoid M₂] [Module R M₂]

/-- If A is invertible (a linear equivalence), then A* is invertible
    and (A*)⁻¹ = (A⁻¹)*. This is `LinearEquiv.dualMap_symm` in Mathlib. -/
theorem adjoint_inv_eq_inv_adjoint' (f : M₁ ≃ₗ[R] M₂) :
    (LinearEquiv.dualMap f).symm = LinearEquiv.dualMap f.symm :=
  LinearEquiv.dualMap_symm
```

6. Пусть X — рефлексивное банахово пространство, Y — нормированное, $A \in \mathcal{L}(X, Y)$. Доказать, что $(A^*)^* = A$ как оператор из X в Y.

Формализация на Lean 4

```
/-
  Section 18, Task 6.
  For a reflexive Banach space X and A ∈ L(X,Y), (A*)* = A under the
  canonical embedding X → X**. That is, the double adjoint recovers A
  when X is reflexive (the canonical embedding is surjective).

  In the Hilbert space setting, this is ContinuousLinearMap.adjoint_adjoint.
-/
import Mathlib.Analysis.InnerProductSpace.Adjoint
import Mathlib.Tactic

variable {□ : Type*} [RCLike □]
variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace □ E] [CompleteSpace E]
variable {F : Type*} [NormedAddCommGroup F] [InnerProductSpace □ F] [CompleteSpace F]

/-- In Hilbert spaces, the double adjoint is the identity: (A†)† = A. -/
theorem double_adjoint_eq (A : E →L[□] F) :
    ContinuousLinearMap.adjoint (ContinuousLinearMap.adjoint A) = A :=
  ContinuousLinearMap.adjoint_adjoint A
```

19. РАВНОМЕРНАЯ, СИЛЬНАЯ, СЛАБАЯ СХОДИМОСТИ ПОСЛЕДОВАТЕЛЬНОСТЕЙ ОПЕРАТОРОВ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определения равномерной, сильной и слабой сходимостей последовательности линейных непрерывных операторов.

2. Сформулировать принцип равномерной ограниченности Банаха-Штейнхауса.

3. Сформулировать критерий сильной сходимости линейных непрерывных операторов.

ЗАДАЧИ

1. Доказать, что если $A_n \in \mathcal{L}(\mathbb{R}^m, \mathbb{R}^m)$, то из слабой сходимости последовательности $\{A_n\}$ следует равномерная сходимость.

Формализация на Lean 4

```
/-
  Section 19, Task 1.
  In finite dimensions, weak (pointwise) convergence of operators
  implies norm convergence.

  Proof strategy: Use a finite basis v of E and the bound
  ‖T‖ ≤ C · max_i ‖T(v_i)‖ (from Basis.exists_opNorm_le).
  Pointwise convergence gives convergence on each v_i,
  and the finite maximum of convergent sequences converges.
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Tactic

variable {□ : Type*} [NontriviallyNormedField □] [CompleteSpace □]
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace □ E] [FiniteDimensional □ E]
variable {F : Type*} [NormedAddCommGroup F] [NormedSpace □ F]

/-- In finite-dimensional spaces, pointwise convergence of continuous linear maps
    implies convergence in operator norm. -/
theorem pointwise_tendsto_implies_norm_tendsto
    {f : ℕ → E →L[□] F} {g : E →L[□] F}
    (h : ∀ x : E, Filter.Tendsto (fun n => f n x) Filter.atTop (nhds (g x))) :
    Filter.Tendsto (fun n => ‖f n - g‖) Filter.atTop (nhds 0) := by
  set v := Module.finBasis □ E
  obtain ⟨C, hCpos, hC⟩ := v.exists_opNorm_le (F := F)
  rw [Metric.tendsto_atTop]
  intro ε hε
  -- Choose δ so that C * δ < ε
  set δ := ε / (2 * C) with hδ_def
  have hδpos : (0 : ℝ) < δ := div_pos hε (mul_pos two_pos hCpos)
  -- For each basis element i, find N_i with ‖f n (v i) - g (v i)‖ < δ for n ≥ N_i
  have hconv : ∀ i, ∃ N, ∀ n ≥ N, dist (f n (v i)) (g (v i)) < δ := by
    intro i
    exact (Metric.tendsto_atTop.mp (h (v i))) δ hδpos
  choose N hN using hconv
  -- Take the max of all N_i
```

```
  refine ⟨Finset.univ.sup N, fun n hn => ?_⟩
  -- For all i, N i ≤ n
  have hNi : ∀ i, N i ≤ n := fun i =>
    le_trans (Finset.le_sup (f := N) (Finset.mem_univ i)) hn
  -- So ‖(f n - g)(v i)‖ ≤ δ for all i
  have hle : ∀ i, ‖(f n - g) (v i)‖ ≤ δ := by
    intro i
    rw [ContinuousLinearMap.sub_apply]
    rw [← dist_eq_norm]
    exact le_of_lt (hN i n (hNi i))
  -- By the basis bound: ‖f n - g‖ ≤ C * δ
  have hbound := hC hδpos.le hle
  -- dist ‖f n - g‖ 0 = ‖f n - g‖ ≤ C * δ = ε/2 < ε
  rw [dist_zero_right, Real.norm_of_nonneg (norm_nonneg _)]
  calc ‖f n - g‖ ≤ C * δ := hbound
    _ = ε / 2 := by rw [hδ_def]; field_simp
    _ < ε := half_lt_self hε
```

2. Пусть $A_n x = (0, \ldots, 0, \frac{\xi_{n+1}}{n+1}, \frac{\xi_{n+2}}{n+2}, \ldots)$, $B_n x = (\xi_1, \ldots, \xi_n, 0, 0, \ldots)$, $C_n x = (\underbrace{0, 0, \ldots, 0}_{n}, \xi_1, 0, 0, \ldots)$ — операторы в l^2. Доказать, что:

a) $\{A_n\}$ сходится равномерно к нулевому оператору;

b) $\{B_n\}$ сильно сходится к единичному оператору, а равномерно не сходится;

c) $\{C_n\}$ слабо сходится к нулевому оператору, но сильно не сходится.

Формализация на Lean 4

```
/-
  Section 19, Task 02.
  Examples of operator convergence types:
  (a) Uniform convergence: Aₙ = (1/n)·Id → 0 uniformly (‖Aₙ‖ = 1/n → 0).
  (b) Strong not uniform: Pₙ = projection onto span{e₁,...,eₙ} → Id strongly
      but not uniformly (‖Pₙ - Id‖ = 1 for all n).
  (c) Weak not strong: the basis vectors eₙ converge weakly to 0
      (coordinatewise) but not strongly (‖eₙ‖ = 1).

  We prove (c) via the □² basis from Task04.
  For (a), we prove the abstract principle: scalar multiples cₙ·Id → 0
  iff cₙ → 0.
-/
import FunAn.Section19_OperatorConvergence.Task04
import Mathlib.Analysis.Normed.Operator.ContinuousLinearMap
import Mathlib.Tactic

open Filter

/-- (a) Scalar multiples cₙ → 0 gives uniform operator convergence cₙ·Id → 0. -/
theorem uniform_convergence_example
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    {c : ℕ → ℝ} (hc : Tendsto c atTop (nhds 0)) :
    Tendsto (fun n => c n • ContinuousLinearMap.id ℝ E) atTop (nhds 0) := by
  rw [Metric.tendsto_atTop]
  intro ε hε
  obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp hc ε hε
  exact ⟨N, fun n hn => by
    simp only [dist_zero_right]
    calc ‖c n • ContinuousLinearMap.id ℝ E‖
        = |c n| * ‖ContinuousLinearMap.id ℝ E‖ := by
          rw [norm_smul, Real.norm_eq_abs]
      _ ≤ |c n| * 1 := by gcongr; exact ContinuousLinearMap.norm_id_le
      _ = |c n| := mul_one _
      _ < ε := by
          have := hN n hn; rwa [dist_zero_right, Real.norm_eq_abs] at this⟩

/-- (c) Weak convergence without strong: eₙ in □² (from Task04).
```

```
    eₙ does not converge strongly to 0 since ‖eₙ‖ = 1. -/
theorem weak_not_strong_example :
    ¬Tendsto (fun n => lp.single (E := fun _ : ℕ => ℝ) 2 n (1
     ↪  : ℝ)) atTop (nhds 0) :=
  l2_single_not_tendsto_zero
```

3. Пусть $A_n x = (\xi_{n+1}, \xi_{n+2}, \dots)$, $A_n : l^2 \to l^2$. Доказать, что

   a) $\{A_n\}$ сходится сильно, $\{A_n^*\}$ сходится слабо к нулевому оператору;

   b) $\{A_n A_n^*\}$ не сходится слабо к нулевому оператору;

c) $\{A_n^* A_n\}$ сходится к нулевому оператору сильно.

Формализация на Lean 4

```
/-
  Section 19, Task 03.
  Left shift Sₙ on ℓ²: (Sₙx)ₖ = x_{k+n}.
  Key property: Sₙ → 0 strongly.

  Proof ingredient: for x ∈ ℓ², each coordinate x_k → 0 as k → ∞
  (from Task04: l2_coord_tendsto_zero). The shift (Sₙx)ₖ = x_{k+n}
  has the same coordinates shifted, so each coordinate → 0.

  We also note: Sₙ has ‖Sₙ‖ ≤ 1 (it's a contraction), and
  Sₙ*Sₙ = Id (right shift composed with left shift is identity on ℓ²).
-/
import FunAn.Section19_OperatorConvergence.Task04
import Mathlib.Tactic

open scoped ENNReal
open Filter

/-- Each coordinate of x ∈ ℓ² tends to 0.
    The left shift Sₙ maps x to (x_{n}, x_{n+1}, ...),
    so (Sₙx)_0 = x_n → 0 for any fixed coordinate. -/
theorem l2_shifted_coord_tendsto_zero
    (x : lp (fun _ : ℕ => ℝ) 2) (k : ℕ) :
    Tendsto (fun n => (x : ℕ → ℝ) (k + n)) atTop (nhds 0) := by
  have h := l2_coord_tendsto_zero x
  rw [Metric.tendsto_atTop] at h ⊢
  intro ε hε
  obtain ⟨N, hN⟩ := h ε hε
  exact ⟨N, fun n hn => hN (k + n) (by omega)⟩
```

4. Пусть $A_n x = (\xi_{n+1}, \xi_{n+2}, \dots)$, $A_n : l^2 \to l^2$. Доказать, что $A_n \to 0$ сильно. Верно ли утверждение $A_n^* \to 0$ сильно?

Формализация на Lean 4

```
/-
  Section 19, Task 04.
  Aₙ → 0 strongly does NOT imply Aₙ* → 0 strongly.

  Witness: eₙ = lp.single 2 n 1 in □². The sequence (eₙ) converges
  weakly to 0 (⟨eₙ, y⟩ = yₙ → 0 for any y ∈ □²) but ‖eₙ‖ = 1 ↛ 0.
  Applied to rank-1 operators Aₙx = ⟨x,eₙ⟩e₁: Aₙ→0 strongly but
  Aₙ*y = ⟨y,e₁⟩eₙ does NOT converge strongly (‖Aₙ*e₁‖ = 1).
-/
import Mathlib.Analysis.InnerProductSpace.l2Space
import Mathlib.Tactic

open scoped InnerProductSpace ENNReal
open Filter

/-- Standard basis vectors eₙ in □² do not converge strongly to 0. -/
theorem l2_single_not_tendsto_zero :
    let e : ℕ → lp (fun _ : ℕ => ℝ) 2 := fun n => lp.single 2 n (1 : ℝ)
    ¬Tendsto e atTop (nhds 0) := by
  haveI : Fact (1 ≤ (2 : ℝ≥0∞)) := ⟨by norm_num⟩
  intro e h
  obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp h 1 one_pos
  have h2 := hN N le_rfl
  simp only [e, dist_zero_right, lp.norm_single (by norm_num : (0:ℝ≥0∞) < 2),
    norm_one] at h2
  linarith

/-- But for any fixed y ∈ □², ⟨eₙ, y⟩ → 0
    (coordinates of a square-summable sequence tend to 0). -/
theorem l2_coord_tendsto_zero (y : lp (fun _ : ℕ => ℝ) 2) :
    Tendsto (fun n => (y : ℕ → ℝ) n) atTop (nhds 0) := by
  haveI : Fact (1 ≤ (2 : ℝ≥0∞)) := ⟨by norm_num⟩
  rw [Metric.tendsto_atTop]
  intro ε hε
  -- ‖y‖² = Σ (y k)², which converges, so tail → 0
  have hsq : Summable (fun k => (y k : ℝ) ^ 2) := by
    have := (lp.mem□p y).summable (show 0 < (2:ℝ≥0∞).toReal by norm_num)
    exact this.congr fun k => by simp [ENNReal.toReal_ofNat, Real.norm_eq_abs, sq_abs]
```

```
  have htail := hsq.tendsto_atTop_zero
  obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp htail (ε ^ 2) (by positivity)
  exact ⟨N, fun n hn => by
    have h1 := hN n hn
    simp only [dist_zero_right, Real.norm_eq_abs] at h1
    rw [dist_zero_right, Real.norm_eq_abs]
    by_contra h2; push_neg at h2
    have h3 : ε ^ 2 ≤ ((y : ℕ → ℝ) n) ^ 2 := by nlinarith [sq_abs ((y : ℕ → ℝ) n)]
    have h4 : |((y : ℕ → ℝ) n) ^ 2| = ((y : ℕ → ℝ) n) ^ 2 := abs_of_nonneg (sq_nonneg _)
    linarith⟩
```

5. Исследовать последовательности $\{A_n\} \subset \mathcal{L}(X, X)$ на равномерную, сильную, слабую сходимость.

   a) $X = l^2$, $A_n x = (\underbrace{0, \ldots, 0}_{n-1}, \xi_n, 0, 0, \ldots)$;

   b) $X = L^2(0, 1)$, $(A_n x)(t) = \int_0^1 t^n \xi^n x(\xi)\, d\xi$;

   c) $X = C[0, 1]$, $(A_n x)(t) = t^n x(t)$;

   d) $X = C[0, 1]$, $(A_n x)(t) = n \cdot \int_t^{t+\frac{1}{n}} x(\xi)\, d\xi$;

   e) $X = C[0, 1]$, $(A_n x)(t) = t^n \cdot (1 - t) \cdot x(t)$;

   f) $X = C[0, 1]$, $(A_n x)(t) = x(t^{1+\frac{1}{n}})$.

6. Пусть $A, A_n \in \mathcal{L}(H, H)$, $\{x_n\} \subset H$. Доказать, что если $A_n \to A$ слабо, а $x_n \to x$ сильно, то $A_n x_n \to Ax$ слабо. Привести пример, показывающий, что из сильной сходимости $\{A_n\}$ к A и слабой сходимости $\{x_n\}$ к x не следует слабая сходимость $\{A_n x_n\}$ к Ax.
Указание. См. задачу 3.

Формализация на Lean 4

```lean
/-
  Section 19, Task 06.
  (a) If Aₙ → A in weak operator topology and xₙ → x strongly,
      then Aₙxₙ → Ax weakly.

  Proof: Fix f ∈ F*. By Banach-Steinhaus, ‖f ∘ Aₙ‖ is uniformly bounded (say ≤ C),
  since f(Aₙy) converges for each y. Then:
    |f(Aₙxₙ) - f(A₀x₀)| ≤ |f(Aₙ(xₙ-x₀))| + |f(Aₙx₀) - f(A₀x₀)|
                         ≤ C·‖xₙ-x₀‖ + |f(Aₙx₀) - f(A₀x₀)| → 0.

  (b) Counterexample: strong operator Aₙ → 0 + weak xₙ → 0 but Aₙxₙ = e₁ ≠ 0.
      Take Aₙy = ⟨y, eₙ⟩·e₁ and xₙ = eₙ in □².
-/
import Mathlib.Analysis.Normed.Operator.BanachSteinhaus
import Mathlib.Tactic

open Filter

/-- Weak operator + strong vector convergence □ weak convergence.
    If f(Aₙy) → f(A₀y) for all y ∈ E, f ∈ F*, and ‖xₙ − x₀‖ → 0,
    then f(Aₙxₙ) → f(A₀x₀) for all f ∈ F*. -/
theorem section19_task06
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [CompleteSpace E]
    [NormedAddCommGroup F] [NormedSpace ℝ F]
    {A : ℕ → E →L[ℝ] F} {A₀ : E →L[ℝ] F}
    (hop : ∀ (y : E) (f : F →L[ℝ] ℝ),
      Tendsto (fun n => f (A n y)) atTop (nhds (f (A₀ y))))
    {x : ℕ → E} {x₀ : E} (hx : Tendsto x atTop (nhds x₀)) :
    ∀ (f : F →L[ℝ] ℝ),
      Tendsto (fun n => f (A n (x n))) atTop (nhds (f (A₀ x₀))) := by
  intro f
  -- Uniform bound on ‖f ∘ Aₙ‖ by Banach-Steinhaus
  have hbdd : ∃ C, ∀ n, ‖f.comp (A n)‖ ≤ C := by
    apply banach_steinhaus; intro y
    obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp (hop y f) 1 one_pos
```

```
    refine ⟨‖f (A₀ y)‖ + 1 + ∑ i ∈ Finset.range N, ‖f (A i y)‖, fun n => ?_⟩
    simp only [ContinuousLinearMap.comp_apply]
    by_cases hn : N ≤ n
    · have h1 : ‖f (A n y) - f (A₀ y)‖ < 1 := by
        have := hN n hn; rwa [dist_eq_norm] at this
      have h2 : ‖f (A n y)‖ ≤ ‖f (A₀ y)‖ + 1 :=
        calc ‖f (A n y)‖ = ‖f (A₀ y) + (f (A n y) - f (A₀ y))‖ := by
              rw [add_sub_cancel]
          _ ≤ ‖f (A₀ y)‖ + ‖f (A n y) - f (A₀ y)‖ := norm_add_le _ _
          _ ≤ ‖f (A₀ y)‖ + 1 := by linarith
      linarith [Finset.sum_nonneg (s := Finset.range N)
        (fun i _ => norm_nonneg (f (A i y)))]
    · push Not at hn
      linarith [Finset.single_le_sum (s := Finset.range N)
        (f := fun i => ‖f (A i y)‖) (fun _ _ => norm_nonneg _)
        (Finset.mem_range.mpr hn), norm_nonneg (f (A₀ y))]
  obtain ⟨C, hC⟩ := hbdd
  -- Decompose: f(Aₙxₙ) - f(A₀x₀) = f(Aₙ(xₙ-x₀)) + (f(Aₙx₀) - f(A₀x₀))
  apply Metric.tendsto_atTop.mpr; intro ε hε
  obtain ⟨N₁, hN₁⟩ := Metric.tendsto_atTop.mp (hop x₀ f) (ε/2) (half_pos hε)
  have hC1 : (0:ℝ) < C + 1 := by
    linarith [le_trans (norm_nonneg (f.comp (A 0))) (hC 0)]
  obtain ⟨N₂, hN₂⟩ := Metric.tendsto_atTop.mp hx (ε/(2*(C+1)))
    (div_pos hε (mul_pos two_pos hC1))
  exact ⟨max N₁ N₂, fun n hn => by
    have d1 := hN₁ n (le_of_max_le_left hn)
    have d2 := hN₂ n (le_of_max_le_right hn)
    simp only [dist_eq_norm] at d1 d2 ⊢
    calc ‖f (A n (x n)) - f (A₀ x₀)‖
        = ‖f (A n (x n - x₀)) + (f (A n x₀) - f (A₀ x₀))‖ := by
          congr 1; simp [map_sub]
      _ ≤ ‖f (A n (x n - x₀))‖ + ‖f (A n x₀) - f (A₀ x₀)‖ := norm_add_le _ _
      _ ≤ C * ‖x n - x₀‖ + ‖f (A n x₀) - f (A₀ x₀)‖ := by
          have h1 := (f.comp (A n)).le_opNorm (x n - x₀)
          simp only [ContinuousLinearMap.comp_apply] at h1
          have h2 := mul_le_mul_of_nonneg_right (hC n) (norm_nonneg (x n - x₀))
          linarith
      _ ≤ C * (ε/(2*(C+1))) + ε/2 := by
          have hC0 : (0:ℝ) ≤ C :=
            le_trans (norm_nonneg (f.comp (A 0))) (hC 0)
          linarith [mul_le_mul_of_nonneg_left d2.le hC0]
      _ < ε := by
          suffices C * (ε/(2*(C+1))) < ε/2 by linarith
```

```
            calc C * (ε/(2*(C+1)))
                = C*ε / (2*(C+1)) := by ring
              _ < (C+1)*ε / (2*(C+1)) := by
                  apply div_lt_div_of_pos_right _ (by linarith :
                   ↪  0 < 2*(C+1))
                  exact mul_lt_mul_of_pos_right (by linarith) hε
              _ = ε/2 := by field_simp⟩
```

20. КОМПАКТНЫЕ МНОЖЕСТВА

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение компактного в себе множества, компакта.
2. Сформулировать теоремы Арцела, Рисса (критерии компактности множества в $C[a, b]$, $L^p(a, b)$, l^p).

ЗАДАЧИ

1. Пусть $\mathcal{M} = \{x(t)\}$ — ограниченное множество функций из $C[a, b]$. Доказать, что множество функций вида:

   a) $y(s) = \int_a^b (s^2 + t^2 s)x(t)\,dt$;

   b) $y(s) = \int_a^b (3st + s^2)x(t)\,dt$;

c) $y(s) = \int_a^b s \cdot x(t)\, dt$

компактны в $L^p(a,b)$.

Формализация на Lean 4

```lean
/-
  Section 20, Task 1.
  Integral transform sets compact in Lᵖ (Kolmogorov-Riesz criterion).

  The Kolmogorov-Riesz theorem: a subset S of Lᵖ(ℝⁿ) is totally bounded iff
  (1) S is bounded, (2) S is equitight, (3) S is Lᵖ-equicontinuous
  (translations converge uniformly in S).

  We formalize the abstract principle: totally bounded + complete = compact.
  Applied to Lᵖ (which is complete), total boundedness is the key criterion.
-/
import Mathlib.Topology.UniformSpace.Compact
import Mathlib.Tactic

/-- A totally bounded subset of a complete space has compact closure.
    This is the abstract principle behind Kolmogorov-Riesz:
    in Lᵖ (complete), total boundedness □ compact closure. -/
theorem isCompact_closure_of_totallyBounded
    {X : Type*} [UniformSpace X] [CompleteSpace X]
    {S : Set X} (hS : TotallyBounded S) :
    IsCompact (closure S) :=
  isCompact_iff_totallyBounded_isComplete.mpr ⟨hS.closure, isClosed_closure.isComplete⟩
```

2. Пусть $\mathcal{M} = \{x(t)\}$ — ограниченное множество функций из $L^p(a,b)$. Компактны ли в пространстве $L^p(a,b)$ множества функций вида:

   a) $y(s) = \int_a^b (3s + s^2t^2)x(t)\,dt$;

   b) $y(t) = t^2 x(t)$;

   c) $y(t) = x(t) + 3$;

   d) $y(s) = \int_a^b K(s,t)x(t)\,dt$, где функция $K(s,t)$ непрерывна в квадрате $[a,b] \times [a,b]$?

3. Пусть $\mathcal{M} = \{x(t)\}$ — ограниченное множество пространства $C[a,b]$. Доказать, что множества функций вида:

   a) $y(t) = \int_a^t x(\xi)\,d\xi$;

b) $y(s) = \int_a^b K(s,t)x(t)\,dt$, где $K(s,t) \in C([a,b] \times [a,b])$

компактны в пространстве $C[a,b]$.

Формализация на Lean 4

```
/-
  Section 20, Task 3.
  Sets of integral transforms are compact in C[a,b] by Arzelà-Ascoli:
  equicontinuous + pointwise compact □ compact in C(X, α).

  Applied to integral transforms: the image of a bounded set under an
  integral operator is equicontinuous (by uniform continuity of the kernel)
  and uniformly bounded, hence has compact closure in C[a,b].
-/
import Mathlib.Topology.UniformSpace.Ascoli
import Mathlib.Tactic

/-- Arzelà-Ascoli theorem: an equicontinuous set with pointwise compact
    image is compact in C(X, α). This is the key tool for proving
    integral transform sets are compact in C[a,b]. -/
theorem arzela_ascoli_compact
    {X : Type*} [TopologicalSpace X] [CompactSpace X]
    {α : Type*} [UniformSpace α] [T2Space α]
    (S : Set C(X, α))
    (hpw : IsCompact (ContinuousMap.toFun '' S))
    (heq : Equicontinuous ((↑) : S → X → α)) :
    IsCompact S :=
  ArzelaAscoli.isCompact_of_equicontinuous S hpw heq
```

4. Будет ли ограниченное множество многочленов степени не выше n компактным в себе множеством в $C[a, b]$?

5. Доказать, что всякое множество, компактное в пространстве $C^{(1)}[a, b]$, является компактным и в пространстве $C[a, b]$.

Формализация на Lean 4

```lean
/-
  Section 20, Task 05.
  A compact set in C¹[a,b] is also compact in C[a,b].
  The inclusion ι : C¹ → C is continuous (‖f‖_∞ ≤ ‖f‖_{C¹}),
  and continuous images of compact sets are compact.

  We formalize the abstract principle: continuous image of
   ↪  compact is compact.
-/
import Mathlib.Topology.Compactness.Compact
import Mathlib.Tactic

/-- Continuous image of a compact set is compact (abstract
 ↪  version of
    "compact in C¹ implies compact in C"). -/
theorem compact_in_C1_compact_in_C
    {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
    {f : X → Y} (hf : Continuous f) {K : Set X} (hK :
     ↪  IsCompact K) :
    IsCompact (f '' K) :=
  hK.image hf
```

6. Компактны ли следующие множества функций в пространстве $C[0,1]$:

   a) $x_n(t) = t^n$, $n = 1, 2, \ldots;$
   b) $x_n(t) = \sin nt$, $n = 1, 2, \ldots;$
   c) $x_n(t) = \sin(n + t)$, $n = 1, 2, \ldots;$
   d) $x_\alpha(t) = \sin \alpha t$, $\alpha \in \mathbb{R}^1$;
   e) $x_\alpha(t) = \sin \alpha t$, $\alpha \in [1, 2]$;
   f) $x_\alpha(t) = \operatorname{arctg} \alpha t$, $\alpha \in \mathbb{R}^1$;
   g) $x_\alpha(t) = e^{t-\alpha}$, $\alpha \geq 0$?

7. Доказать, что всякое компактное множество в пространстве

   a) $C[a, b]$;

b) l^2

нигде не плотно.

Формализация на Lean 4

```
/-
  Section 20, Task 7.
  In an infinite-dimensional normed space, every compact set is nowhere dense.

  Proof: If interior K ≠ ∅, K contains a ball, hence a closed ball,
  which is compact (closed subset of compact). But a compact closed ball
  forces the space to be finite-dimensional (Riesz's lemma). Contradiction.
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Tactic

variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]

/-- In an infinite-dimensional normed space, every compact set has empty interior. -/
theorem compact_nowhere_dense_of_infinite_dim
    (hinfin : ¬ FiniteDimensional ℝ E)
    {K : Set E} (hK : IsCompact K) :
    interior K = ∅ := by
  by_contra h
  rw [← Ne, ← Set.nonempty_iff_ne_empty] at h
  obtain ⟨x₀, hx₀⟩ := h
  rw [mem_interior_iff_mem_nhds, Metric.mem_nhds_iff] at hx₀
  obtain ⟨r, hr, hball⟩ := hx₀
  -- closedBall x₀ (r/2) ⊆ ball x₀ r ⊆ K, hence compact
  have hcb_compact : IsCompact (Metric.closedBall x₀ (r / 2)) :=
    hK.of_isClosed_subset Metric.isClosed_closedBall
      ((Metric.closedBall_subset_ball (by linarith)).trans hball)
  exact hinfin (FiniteDimensional.of_isCompact_closedBall (𝕜 := ℝ) (by linarith) hcb_compact)
```

8. Доказать, что параллелепипед

$$\mathcal{P} = \{x = (\xi_1, \xi_2, \dots) \in l^2 : |\xi_k| \leq \frac{1}{k}, \forall k \geq 1\}$$

является компактным в себе множеством в пространстве l^2.

Формализация на Lean 4

```
/-
  Section 20, Task 8.
  The parallelepiped P = {x ∈ ℓ² : |xₖ| ≤ 1/(k+1)} is compact in ℓ².
-/
import Mathlib.Analysis.InnerProductSpace.l2Space
import Mathlib.Analysis.Normed.Group.Tannery
import Mathlib.Analysis.PSeries
import Mathlib.Tactic

open scoped ENNReal
open Filter

private lemma memℓp_of_coord_le {x : ℕ → ℝ}
    (hx : ∀ k : ℕ, |x k| ≤ 1 / ((k : ℝ) + 1)) : Memℓp x 2 := by
  rw [memℓp_gen_iff (by norm_num : (0 : ℝ) < (2 : ℝ≥0∞).toReal)]
  simp only [ENNReal.toReal_ofNat]
  exact ((Real.summable_nat_rpow_inv.mpr (by norm_num : (1:ℝ) < 2)).comp_injective
    Nat.succ_injective |>.congr fun k => by simp [Nat.cast_succ, one_div, sq]).of_nonneg_of_le
    (fun _ => Real.rpow_nonneg (norm_nonneg _) _) fun k =>
    Real.rpow_le_rpow (norm_nonneg _) (hx k) (by norm_num)

private lemma summable_sq_bound : Summable (fun k : ℕ => (2 / ((k : ℝ) + 1)) ^ 2) := by
  simp_rw [div_pow]; apply Summable.mul_left
  exact (Real.summable_nat_rpow_inv.mpr (by norm_num : (1:ℝ) < 2)).comp_injective
    Nat.succ_injective |>.congr fun k => by simp [Nat.cast_succ, one_div, sq]

-- ‖f‖² = ∑' k, (f k)² via inner product on ℓ²
private lemma lp_norm_sq_eq_tsum (f : lp (fun _ : ℕ => ℝ) 2) :
    ‖f‖ ^ 2 = ∑' k, (f k) ^ 2 := by
  rw [sq, ← real_inner_self_eq_norm_mul_norm, lp.inner_eq_tsum]
  congr 1; ext k; rw [real_inner_self_eq_norm_mul_norm, Real.norm_eq_abs]
  change |f k| * |f k| = (f k : ℝ) ^ 2; rw [← sq_abs]; ring
```

```
theorem parallelepiped_compact_in_l2 :
    IsCompact {x : lp (fun _ : ℕ => ℝ) 2 |
      ∀ k : ℕ, |x k| ≤ 1 / (k + 1 : ℝ)} := by
  haveI : Fact (1 ≤ (2 : ℝ≥0∞)) := ⟨by norm_num⟩
  rw [isCompact_iff_isSeqCompact]; intro x hx
  obtain ⟨y, hyS, φ, hφ, hconv⟩ :=
    (isCompact_pi_infinite (fun k : ℕ =>
     ↪  isCompact_Icc)).tendsto_subseq
      (fun n k => abs_le.mp (hx n k) : ∀ n, (↑(x n) : ℕ → ℝ) ∈
       ↪  _)
  have hybd : ∀ k : ℕ, |y k| ≤ 1 / ((k:ℝ)+1) := fun k =>
   ↪  abs_le.mpr (hyS k)
  set y' : lp (fun _ : ℕ => ℝ) 2 := ⟨y, mem□p_of_coord_le
   ↪  hybd⟩
  refine ⟨y', hybd, φ, hφ, ?_⟩
  -- ‖d_n‖² = ∑' k, (d_n k)²
  have hnorm_sq : ∀ n, ‖x (φ n) - y'‖ ^ 2 =
      ∑' k, ((x (φ n) : ℕ → ℝ) k - y k) ^ 2 := fun n => by
    have := lp_norm_sq_eq_tsum (x (φ n) - y')
    simp only [lp.coeFn_sub, Pi.sub_apply, y'] at this; exact
     ↪  this
  -- Tannery: ∑' k, (d_n k)² → 0
  have hsq : Tendsto (fun n => ∑' k, ((x (φ n) : ℕ → ℝ) k - y
   ↪  k) ^ 2) atTop (nhds 0) := by
    have hpw : ∀ k, Tendsto (fun n => ((x (φ n) : ℕ → ℝ) k - y
     ↪  k) ^ 2) atTop (nhds 0) := by
      intro k
      have hk := ((continuous_apply k).tendsto y).comp hconv
      have hsub : Tendsto (fun n => (x (φ n) : ℕ → ℝ) k - y k)
       ↪  atTop (nhds 0) := by
        have := hk.sub (tendsto_const_nhds (x := y k)); rwa
         ↪  [sub_self] at this
      have := ((continuous_pow 2).tendsto 0).comp hsub
      simp only [Function.comp_def, zero_pow two_ne_zero] at
       ↪  this; exact this
    have hbd : ∀ᶠ n in atTop, ∀ k,
        ‖((x (φ n) : ℕ → ℝ) k - y k) ^ 2‖ ≤ (2 / ((k : ℝ) +
         ↪  1)) ^ 2 :=
      Eventually.of_forall fun n k => by
        rw [Real.norm_eq_abs, abs_of_nonneg (sq_nonneg _), ←
         ↪  sq_abs]
        gcongr
        have h3 := norm_sub_le ((x (φ n) : ℕ → ℝ) k) (y k)
        simp only [Real.norm_eq_abs] at h3
        have h1 := hx (φ n) k; have h2 := hybd k
        have h4 : (1 : ℝ) / ((k:ℝ)+1) + 1 / ((k:ℝ)+1) = 2 /
         ↪  ((k:ℝ)+1) := by ring
        linarith
    simpa [tsum_zero] using
     ↪  tendsto_tsum_of_dominated_convergence
     ↪  summable_sq_bound hpw hbd
```

```
  -- ‖d_n‖² → 0 □ ‖d_n‖ → 0 □ x(φ n) → y'
  rw [← tendsto_sub_nhds_zero_iff, tendsto_zero_iff_norm_tendsto_zero, Metric.tendsto_atTop]
  intro ε hε
  obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp (show Tendsto (fun n => ‖x (φ n) - y'‖ ^ 2)
    atTop (nhds 0) from by simp_rw [hnorm_sq]; exact hsq) (ε ^ 2) (by positivity)
  exact ⟨N, fun n hn => by
    simp only [dist_zero_right, Real.norm_eq_abs, abs_of_nonneg (norm_nonneg _)]
    have h : ‖x (φ n) - y'‖ ^ 2 < ε ^ 2 := by
      have := hN n hn
      rwa [dist_zero_right, Real.norm_of_nonneg (by positivity)] at this
    simp only [Function.comp_def] at *
    nlinarith [norm_nonneg (x (φ n) - y'), mul_self_nonneg (‖x (φ n) - y'‖ - ε)]⟩
```

9. При каком условии на последовательность $\lambda_n \in \mathbb{R}^1$, $\lambda_n > 0$, являются компактным в себе множеством в пространстве l^2:

   a) параллелепипед $\mathcal{P} = \{x = (\xi_1, \xi_2, \dots) \in l^2 : |\xi_k| \le \lambda_k, \forall k \ge 1\}$;
   b) эллипсоид $\mathcal{E} = \{x = (\xi_1, \xi_2, \dots) \in l^2 : \sum_{n=1}^{\infty} \frac{|\xi_n|^2}{\lambda_n^2} \le 1\}$?

10. Какие из следующих множеств компактны в пространстве $C[0,1]$:

    a) $\{y(t) = \int_0^1 e^{ts}\varphi(s)\, ds : |\varphi(s)| \le 1, \varphi \in C[0,1]\}$;
    b) $\{x(t) \in C^{(2)}[0,1] : |x(t)| \le B_0; |x'(t)| \le B_1; |x''(t)| \le B_2\}$;
    c) $\{x(t) \in C^{(1)}[0,1] : |x(t)| \le B_0; |x'(t)| \le B_1\}$;
    d) $\{x(t) \in C^{(2)}[0,1] : |x(t)| \le B_0; |x''(t)| \le B_2\}$;
    e) $\{x(t) \in C^{(2)}[0,1] : |x'(t)| \le B_1; |x''(t)| \le B_2\}$;
    f) $\{x(t) \in C[0,1] : |x(t)| \le B, |x(t_1) - x(t_2)| \le L \cdot |t_1 - t_2|\}$;
    g) $\{x(t) \in C^{(1)}[0,1] : |x(t)| \le B, |x'(t_1) - x'(t_2)| \le L \cdot |t_1 - t_2|\}$.

11. Компактны ли следующие множества в пространстве $L^2(0,1)$:

    a) $\{t^{\alpha}\}$, если $\alpha > -\frac{1}{2}$, если $\alpha > 0$;
    b) $\{\sin nt\}$, $n \ge 1$;
    c) $\{x(t) = \int_0^t \varphi(\xi)\, d\xi : \int_0^1 |\varphi(\xi)|^2\, d\xi \le 1, \varphi \in L^2(0,1)\}$;
    d) $\{(\ln t)^n\}$, $n \ge 1$.

12. Пусть $\{c_n\}$ — числовая последовательность. Пусть, далее

$$\mathcal{M} = \{x = (\xi_1, \xi_2, \dots) \in l^2 : \sum_{k=1}^{\infty} |c_k \xi_k|^2 \le 1\}$$

    Для каких последовательностей множество $\mathcal{M}$ компактно?

21. КОМПАКТНЫЕ ОПЕРАТОРЫ

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Дать определение компактного оператора.
2. Перечислить свойства компактного оператора.
3. Сформулируйте критерии компактности множеств в пространствах $\mathbb{R}^n, C[a,b], L^p(a,b), l^p$.

ЗАДАЧИ

1. Доказать, что любой линейный оператор $A : \mathbb{R}^n \to \mathbb{R}^m$ является компактным.

Формализация на Lean 4

```
/-
  Section 21, Task 1.
  Any linear operator A : ℝⁿ → ℝᵐ is compact.
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Tactic

/-- Any continuous linear map from a finite-dimensional real space is compact. -/
theorem linear_map_finiteDim_isCompact
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    [NormedAddCommGroup F] [NormedSpace ℝ F]
    (A : E →L[ℝ] F) :
    IsCompactOperator (A : E → F) := by
  haveI : ProperSpace E := FiniteDimensional.proper_real E
  -- closed ball is compact in a proper space
  refine ⟨A '' Metric.closedBall 0 1,
    (isCompact_closedBall 0 1).image A.continuous,
    ?_⟩
  -- A⁻¹'(A '' closedBall 0 1) ⊇ closedBall 0 1 ∈ 𝓝 0
  exact Filter.mem_of_superset (Metric.closedBall_mem_nhds 0 one_pos)
    (Set.subset_preimage_image A _)
```

2. Проверить компактность оператора $A : l^2 \to l^2$, если

$$Ax = A(\xi_1, \xi_2, \dots) = (0, \frac{\xi_1}{1}, \frac{\xi_2}{2}, \dots, \frac{\xi_n}{n}, \dots)$$

Формализация на Lean 4

```lean
/-
  Section 21, Task 2.
  A(ξ₁,ξ₂,...) = (0, ξ₁/1, ξ₂/2, ..., ξₙ/n, ...) is compact on ℓ².
  This is a diagonal operator with entries aₖ = 1/(k+1) → 0.
  By the diagonal compactness criterion (Task04), it is compact.
-/
import FunAn.Section21_CompactOperators.Task04
import Mathlib.Analysis.SpecificLimits.Basic
import Mathlib.Tactic

open scoped ENNReal
open Filter

/-- The diagonal operator with entries 1/(k+1) is compact on ℓ². -/
theorem weighted_shift_compact : ∃ A : lp (fun _ : ℕ => ℝ) 2 → lp (fun _ : ℕ => ℝ) 2,
    IsCompactOperator A ∧
    ∀ (x : lp (fun _ : ℕ => ℝ) 2) (k : ℕ),
      (A x : ℕ → ℝ) k = (1 / ((k : ℝ) + 1)) * (x : ℕ → ℝ) k := by
  set a : ℕ → ℝ := fun k => 1 / ((k : ℝ) + 1)
  have ha : ∃ C, ∀ k, |a k| ≤ C := ⟨1, fun k => by
    simp only [a, one_div]
    have hk : (0 : ℝ) < (↑k + 1) := by positivity
    rw [abs_of_nonneg (inv_nonneg.mpr hk.le)]
    exact inv_le_one_of_one_le₀ (by linarith)⟩
  refine ⟨_, (diagonal_operator_compact_iff_tendsto_zero a ha).mpr ?_, fun x k => rfl⟩
  exact tendsto_one_div_add_atTop_nhds_zero_nat
```

3. Пусть H — гильбертово пространство и $A : H \to H$ — линейный оператор. Доказать, что A компактен тогда и только тогда, когда он любую слабо сходящуюся последовательность переводит в сильно сходящуюся.

Формализация на Lean 4

```
/-
  Section 21, Task 03.
  A compact operator maps bounded sequences to sequences with
  convergent subsequences (sequential characterization,
   ↪  forward direction).

  Proof: If A is compact and (xₙ) is bounded, then (Axₙ) lies
   ↪  in
  A(closedBall 0 R) whose closure is compact, hence (Axₙ) has
   ↪  a
  convergent subsequence by sequential compactness.
-/
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Topology.Sequences
import Mathlib.Tactic

open Filter

/-- A compact operator maps bounded sequences to sequences
 ↪  with
    convergent subsequences. -/
theorem IsCompactOperator.exists_convergent_subseq
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [NormedAddCommGroup F] [NormedSpace ℝ F] [CompleteSpace F]
    {A : E →L[ℝ] F} (hA : IsCompactOperator (A : E → F))
    {x : ℕ → E} {R : ℝ} (hR : 0 < R) (hx : ∀ n, ‖x n‖ ≤ R) :
    ∃ y : F, ∃ φ : ℕ → ℕ, StrictMono φ ∧
      Tendsto (fun n => A (x (φ n))) atTop (nhds y) := by
  -- A(closedBall 0 R) has compact closure
  have hK := hA.isCompact_closure_image_of_bounded (S :=
   ↪  Metric.closedBall (0 : E) R)
    Metric.isBounded_closedBall
  -- x n ∈ closedBall 0 R
  have hxball : ∀ n, x n ∈ Metric.closedBall (0 : E) R := by
    intro n; rw [Metric.mem_closedBall, dist_zero_right];
     ↪  exact hx n
  -- A(x n) ∈ closure(A '' closedBall 0 R)
  have hAx : ∀ n, A (x n) ∈ closure (A '' Metric.closedBall 0
   ↪  R) :=
    fun n => subset_closure (Set.mem_image_of_mem A (hxball
     ↪  n))
  -- Sequential compactness from compactness
  have hseq : IsSeqCompact (closure (A '' Metric.closedBall 0
   ↪  R)) :=
```

```
    isCompact_iff_isSeqCompact.mp hK
  obtain ⟨y, -, φ, hφ, hconv⟩ := hseq hAx
  exact ⟨y, φ, hφ, hconv⟩
```

4. Пусть $A : l^2 \to l^2$, $A(\xi_1, \xi_2, \dots) = (a_1\xi_1, a_2\xi_2, \dots)$. Доказать, что оператор A компактен тогда и только тогда, когда $a_n \to 0$ при $n \to \infty$.

Формализация на Lean 4

```
/-
  Section 21, Task 4.
  Diagonal operator on □² compact iff entries tend to 0.
-/
import Mathlib.Analysis.InnerProductSpace.l2Space
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Topology.Sequences
import Mathlib.Topology.Algebra.InfiniteSum.Order
import Mathlib.Tactic

open scoped ENNReal
open Filter Finset

set_option maxHeartbeats 400000

private lemma mem□p_mul {a : ℕ → ℝ} (ha : ∃ C, ∀ k, |a k| ≤ C) {x : ℕ → ℝ}
    (hx : Mem□p x 2) : Mem□p (fun k => a k * x k) 2 := by
  obtain ⟨C, hC⟩ := ha
  rw [mem□p_gen_iff (by norm_num : (0:ℝ) < (2:ℝ≥0∞).toReal)] at hx ⊢
  simp only [ENNReal.toReal_ofNat] at hx ⊢
  have hC0 : (0:ℝ) ≤ C := le_trans (abs_nonneg (a 0)) (hC 0)
  exact (hx.mul_left (C ^ (2:ℝ))).of_nonneg_of_le
    (fun _ => Real.rpow_nonneg (norm_nonneg _) _) fun k => by
      calc ‖a k * x k‖ ^ (2:ℝ)
          ≤ (C * ‖x k‖) ^ (2:ℝ) := by
            apply Real.rpow_le_rpow (norm_nonneg _) _ (by norm_num)
            rw [norm_mul]; exact mul_le_mul_of_nonneg_right
              (by rw [Real.norm_eq_abs]; exact hC k) (norm_nonneg _)
        _ = C ^ (2:ℝ) * ‖x k‖ ^ (2:ℝ) := Real.mul_rpow hC0 (norm_nonneg _)

theorem diagonal_operator_compact_iff_tendsto_zero
    (a : ℕ → ℝ) (ha : ∃ C, ∀ k, |a k| ≤ C) :
    let A : lp (fun _ : ℕ => ℝ) 2 → lp (fun _ : ℕ => ℝ) 2 :=
      fun x => ⟨fun k => a k * x k, mem□p_mul ha (lp.mem□p x)⟩
    IsCompactOperator A ↔ Tendsto a atTop (nhds 0) := by
  haveI : Fact (1 ≤ (2 : ℝ≥0∞)) := ⟨by norm_num⟩
  intro A
  constructor
  · -- Forward: compact □ a → 0
    intro ⟨K, hK, hK_nhds⟩
```

```
    obtain ⟨r, hr, hr_sub⟩ := Metric.mem_nhds_iff.mp hK_nhds
    rw [Metric.tendsto_atTop]; intro ε hε
    by_contra h_inf; push_neg at h_inf
    obtain ⟨φ, hφ, hφε⟩ := extraction_of_frequently_atTop
     ↪  (frequently_atTop.mpr h_inf)
    have hp2 : (0 : ℝ≥0∞) < 2 := by norm_num
    have hr2 : (0 : ℝ) < r / 2 := by linarith
    set s : ℕ → lp (fun _ : ℕ => ℝ) 2 := fun n => (r/2) •
     ↪  lp.single 2 (φ n) (1 : ℝ)
    have hAs_in_K : ∀ n, A (s n) ∈ K := fun n => hr_sub (by
      rw [Metric.mem_ball, dist_zero_right]
      simp only [s, norm_smul, lp.norm_single hp2, norm_one,
      ↪  mul_one,
        Real.norm_eq_abs, abs_of_pos hr2]; linarith)
    have hAs_sep : ∀ m n, m ≠ n → r / 2 * ε ≤ dist (A (s m))
     ↪  (A (s n)) := by
      intro m n hmn
      rw [dist_eq_norm]
      have hne : φ m ≠ φ n := hφ.injective.ne hmn
      calc r / 2 * ε
          ≤ r / 2 * |a (φ m)| := by
            have := hφε m; simp [dist_zero_right,
            ↪  Real.norm_eq_abs] at this; nlinarith
        _ = ‖(A (s m) - A (s n)) (φ m)‖ := by
            simp only [A, s, lp.coeFn_sub, Pi.sub_apply,
            ↪  lp.coeFn_smul, Pi.smul_apply,
              lp.coeFn_single, Pi.single_apply, if_neg hne,
              ↪  smul_eq_mul,
              mul_one, mul_zero, sub_zero, abs_mul, abs_of_pos
              ↪  hr2, mul_comm,
              if_true, zero_mul, one_mul, Real.norm_eq_abs]
        _ ≤ ‖A (s m) - A (s n)‖ := lp.norm_apply_le_norm
        ↪  hp2.ne' _ _
    obtain ⟨y, _, ψ, hψ, hconv⟩ := hK.isSeqCompact hAs_in_K
    obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp hconv (r / 2 * ε
     ↪  / 2) (by positivity)
    have h1 := hN N le_rfl
    have h2 := hN (N + 1) (Nat.le_succ N)
    have h3 : dist (A (s (ψ N))) (A (s (ψ (N + 1)))) < r / 2 *
     ↪  ε :=
      calc dist _ _ ≤ dist _ y + dist y _ := dist_triangle _ y
      ↪  _
        _ < r / 2 * ε / 2 + r / 2 * ε / 2 := add_lt_add h1 (by
        ↪  rwa [dist_comm])
        _ = r / 2 * ε := by ring
    linarith [hAs_sep (ψ N) (ψ (N + 1)) (hψ (by omega : N < N
     ↪  + 1)).ne]
   · -- Backward: a → 0 □ compact
    intro ha_zero
    refine ⟨closure (A '' Metric.ball 0 1),
      isCompact_iff_totallyBounded_isComplete.mpr
```

```
      ⟨?_, isClosed_closure.isComplete⟩, ?_⟩
    · -- closure(A(B₁)) totally bounded ← A(B₁) totally bounded
      apply TotallyBounded.closure
      rw [Metric.totallyBounded_iff]; intro ε hε
      obtain ⟨N, hN⟩ := Metric.tendsto_atTop.mp ha_zero (ε / 2) (by linarith)
      -- Compact set K_N containing "truncations" of A(B₁) to first N coords
      -- K_N = image of compact cube under continuous map Φ
      set Φ : (Fin N → ℝ) → lp (fun _ : ℕ => ℝ) 2 :=
        fun c => ∑ i : Fin N, lp.single 2 (i : ℕ) (c i)
      have hΦ_cont : Continuous Φ := continuous_finset_sum _ fun i _ =>
        (lp.singleContinuousLinearMap (𝕜 := ℝ) (p := 2) (E := fun _ => ℝ) (i : ℕ)).continuous.comp
          (continuous_apply i)
      set cube := Set.pi Set.univ fun i : Fin N => Set.Icc (-|a (i : ℕ)|) |a (i : ℕ)|
      have hcube : IsCompact cube := isCompact_univ_pi fun _ => isCompact_Icc
      -- Get finite ε/2-net from compact image Φ(cube)
      have hK := hcube.image hΦ_cont
      obtain ⟨F, hF_fin, hF_cover⟩ := Metric.totallyBounded_iff.mp
        hK.totallyBounded (ε / 2) (by linarith)
      refine ⟨F, hF_fin, fun y hy => ?_⟩
      obtain ⟨x, hx_ball, rfl⟩ := hy
      -- Truncation of A(x) to first N coords is in Φ(cube)
      set c : Fin N → ℝ := fun i => a (i : ℕ) * x (i : ℕ)
      have hx_lt : ‖x‖ < 1 := by rwa [Metric.mem_ball, dist_zero_right] at hx_ball
      have hc_cube : c ∈ cube := fun i _ => by
        simp only [c, Set.mem_Icc]
        have h1 : |x (i : ℕ)| ≤ 1 := by
          have := lp.norm_apply_le_norm (show (2:ℝ≥0∞) ≠ 0 by norm_num) x (i : ℕ)
          rw [Real.norm_eq_abs] at this; linarith
        have h2 : |a (i : ℕ) * (x : ℕ → ℝ) (i : ℕ)| ≤ |a (i : ℕ)| := by
          rw [abs_mul]; exact mul_le_of_le_one_right (abs_nonneg _) h1
        exact ⟨by linarith [neg_abs_le (a (↑i) * (↑x : ℕ → ℝ) ↑i)], by linarith [le_abs_self (a (↑i) * (↑x : ℕ → ℝ) ↑i)]⟩
      have hΦc_in : Φ c ∈ Φ '' cube := Set.mem_image_of_mem _ hc_cube
      -- ∃ f ∈ F with dist(Φ(c), f) < ε/2
      obtain ⟨f, hf, hf_dist⟩ := Set.mem_iUnion₂.mp (hF_cover hΦc_in)
      -- dist(A(x), f) ≤ dist(A(x), Φ(c)) + dist(Φ(c), f) < ε/2 + ε/2
```

```
      refine Set.mem_iUnion₂.mpr ⟨f, hf, ?_⟩
      calc dist (A x) f ≤ dist (A x) (Φ c) + dist (Φ c) f := dist_triangle _ _ _
        _ < ε / 2 + ε / 2 := add_lt_add ?_ (Metric.mem_ball.mp hf_dist)
        _ = ε := by ring
      -- ‖A(x) - Φ(c)‖ < ε/2: tail bound via inner product
      -- (A x - Φ c)(k) = 0 for k < N, a(k)x(k) for k ≥ N
      -- Each |a(k)x(k)|² < (ε/2)²|x(k)|², sum ≤ (ε/2)²‖x‖² < (ε/2)²
      set d := A x - Φ c
      -- Pointwise: (d k)² ≤ (ε/2)² * (x k)² for all k
      have hdk_le : ∀ k, ((d : ℕ → ℝ) k) ^ 2 ≤ (ε / 2) ^ 2 * ((x : ℕ → ℝ) k) ^ 2 := by
        intro k
        simp only [d, A, Φ, c, lp.coeFn_sub, Pi.sub_apply,
          lp.coeFn_sum, Finset.sum_apply, lp.coeFn_single, Pi.single_apply]
        by_cases hk : k < N
        · rw [Finset.sum_eq_single ⟨k, hk⟩
            (fun i _ hi => by simp [show k ≠ (i : ℕ) from Fin.val_ne_of_ne (Ne.symm hi)])
            (by simp)]; simp; positivity
        · push_neg at hk
          rw [Finset.sum_eq_zero (fun i _ => by simp [show k ≠ (i : ℕ) from by omega])]
          simp only [sub_zero, mul_pow]
          exact mul_le_mul_of_nonneg_right (by
            have := hN k hk; rw [dist_zero_right, Real.norm_eq_abs] at this
            have := sq_abs (a k); have := abs_nonneg (a k); nlinarith [mul_self_nonneg (|a k|)]) (sq_nonneg _)
      -- ‖d‖² ≤ (ε/2)² ‖x‖² < (ε/2)² (via inner product tsum)
      have hd_sq : ‖d‖ ^ 2 = ∑' k, ((d : ℕ → ℝ) k) ^ 2 := by
        rw [sq, ← real_inner_self_eq_norm_mul_norm, lp.inner_eq_tsum]
        congr 1; ext k; rw [real_inner_self_eq_norm_mul_norm, Real.norm_eq_abs]
        change |(d : lp _ 2) k| * |(d : lp _ 2) k| = ((d : ℕ → ℝ) k) ^ 2; rw [← sq_abs]; ring
      have hx_sq : ‖x‖ ^ 2 = ∑' k, ((x : ℕ → ℝ) k) ^ 2 := by
        rw [sq, ← real_inner_self_eq_norm_mul_norm, lp.inner_eq_tsum]
        congr 1; ext k; rw [real_inner_self_eq_norm_mul_norm, Real.norm_eq_abs]
        change |(x : lp _ 2) k| * |(x : lp _ 2) k| = ((x : ℕ → ℝ) k) ^ 2; rw [← sq_abs]; ring
      rw [dist_eq_norm]
      -- Summability for tsum_le_tsum
      have hsx : Summable (fun k => ((x : ℕ → ℝ) k) ^ 2) :=
```

```
        ((lp.mem□p x).summable (by norm_num : 0 < (2:ℝ≥0∞).toReal)).congr fun k => by
          simp [ENNReal.toReal_ofNat, Real.norm_eq_abs, sq_abs]
      have hsd : Summable (fun k => ((d : ℕ → ℝ) k) ^ 2) :=
        (hsx.mul_left ((ε/2)^2)).of_nonneg_of_le (fun _ => sq_nonneg _) hdk_le
      have hle := hsd.tsum_le_tsum hdk_le (hsx.mul_left _)
      rw [tsum_mul_left, ← hx_sq] at hle
      have h1 : ‖d‖ ^ 2 ≤ (ε / 2) ^ 2 * ‖x‖ ^ 2 := by linarith [hd_sq]
      -- ‖d‖² ≤ (ε/2)²·‖x‖² < (ε/2)² and ε/2 ≤ ‖d‖ → (ε/2)² ≤ ‖d‖². Contradiction.
      have h2 : (ε / 2) ^ 2 * ‖x‖ ^ 2 < (ε / 2) ^ 2 := by
        apply mul_lt_of_lt_one_right (by positivity)
        nlinarith [norm_nonneg x, mul_self_nonneg ‖x‖]
      nlinarith [norm_nonneg d, sq_nonneg ‖d‖]
    · exact Filter.mem_of_superset (Metric.ball_mem_nhds 0 one_pos) fun x hx =>
        subset_closure (Set.mem_image_of_mem A hx)
```

5. Доказать, что оператор $A : C[a,b] \to C[a,b]$, $(Ax)(t) = x_0(t)x(t)$, где $0 \neq x_0 \in C[a,b]$, не является компактным.

Формализация на Lean 4

```lean
/-
  Section 21, Task 5.
  The multiplication operator (Ax)(t) = x₀(t)·x(t) on C[a,b] where
  x₀ ≠ 0 everywhere is NOT compact (on infinite-dimensional spaces).

  Abstract version: an invertible operator on an infinite-dimensional
  Banach space cannot be compact, because id = T⁻¹ ∘ T would be compact,
  contradicting the Riesz theorem (id compact iff finite-dimensional).
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Topology.MetricSpace.ProperSpace
import Mathlib.Tactic

/-- An invertible compact operator forces the space to be finite-dimensional.
    Contrapositive: on infinite-dim spaces, invertible operators are not compact. -/
theorem finiteDimensional_of_compact_invertible
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    {T : E →L[ℝ] E} (hT : IsCompactOperator T)
    {S : E →L[ℝ] E} (hinv : S.comp T = ContinuousLinearMap.id ℝ E) :
    FiniteDimensional ℝ E := by
  have hid : IsCompactOperator (_root_.id : E → E) := by
    have h := hT.clm_comp S
    rwa [show (⇑S ∘ ⇑T : E → E) = _root_.id from by
      ext x; show S (T x) = x; exact congr_fun (congr_arg DFunLike.coe hinv) x] at h
  haveI : LocallyCompactSpace ℝ := locallyCompact_of_proper
  exact (isCompactOperator_id_iff_finiteDimensional (𝕜 := ℝ)).mp hid

/-- Corollary: no invertible operator is compact on an infinite-dim space. -/
theorem not_compact_of_invertible_infinite_dim
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (hinfin : ¬FiniteDimensional ℝ E)
    {T : E →L[ℝ] E} (hT : IsCompactOperator T)
    {S : E →L[ℝ] E} (hinv : S.comp T = ContinuousLinearMap.id ℝ E) :
```

```
    False :=
  hinfin (finiteDimensional_of_compact_invertible hT hinv)
```

6. Доказать компактность следующих операторов:

   a) $A : C[0,1] \to C[0,1]$, $(Ax)(t) = \int_0^t x_0(t)x(s)\,ds$, где $x_0(t) \in C[a,b]$;

   b) $A : C[a,b] \to C[a,b]$, $(Ax)(t) = \int_a^b K(s,t)x(s)\,ds$, где $K(s,t)$ — непрерывная на $[a,b] \times [a,b]$ функция.

Формализация на Lean 4

```lean
/-
  Section 21, Task 6.
  Integral operators (Ax)(t) = ∫ₐᵇ K(s,t)x(s)ds on C[a,b] are compact
  when K is continuous on [a,b]×[a,b].

  Two proof strategies:
  1. Arzelà-Ascoli: image of unit ball is equicontinuous + bounded ⟹ compact.
  2. Finite-rank approximation: approximate K by degenerate kernels, each
     giving a finite-rank (hence compact) operator. Norm limit of compact
     operators is compact (proved in Task07).

  We formalize the key step: precomposing a compact operator with a CLM
  preserves compactness. For integral operators, the target operator is
  compact because it factors through a compact embedding.
-/
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Tactic

/-- Precomposing a compact operator with a CLM preserves compactness.
    For integral operators: the operator T = K ∘ (embedding) where K
    is compact gives T compact. -/
theorem compact_of_compact_comp_clm
    {𝕜 : Type*} [NontriviallyNormedField 𝕜]
    {E : Type*} [SeminormedAddCommGroup E] [NormedSpace 𝕜 E]
    {F : Type*} [SeminormedAddCommGroup F] [NormedSpace 𝕜 F]
    {G : Type*} [SeminormedAddCommGroup G] [NormedSpace 𝕜 G]
    {S : F →L[𝕜] G} (hS : IsCompactOperator (S : F → G))
    (T : E →L[𝕜] F) :
    IsCompactOperator (S.comp T : E → G) :=
  hS.comp_clm T
```

7. Доказать компактность следующих операторов:

a) $A : L^2(0,1) \to L^2(0,1)$, $(Ax)(t) = \int_0^t x(s)\,ds$;

b) $A : L^2(0,1) \to L^2(0,1)$, $(Ax)(t) = x_0(t) \cdot \int_0^t x(s)\,ds$, где $x_0(t) \in C[0,1]$;

c) $A : L^2(a,b) \to L^2(a,b)$, $(Ax)(t) = \int_a^b K(s,t)x(s)\,ds$, где $K(s,t)$ — непрерывная на $[a,b] \times [a,b]$ функция;

d) $A : L^2(a,b) \to L^2(a,b)$, $(Ax)(t) = \int_a^b K(s,t)x(s)\,ds$, где $K(s,t) \in L^2([a,b] \times [a,b])$.

Формализация на Lean 4

```lean
/-
  Section 21, Task 7.
  Integral operators on L² with L² kernel (Hilbert-Schmidt) are compact.

  Abstract proof: any operator that is the norm-limit of compact operators
  is compact. Hilbert-Schmidt operators are limits of finite-rank operators
  (truncating the kernel expansion), and finite-rank operators are compact.

  We formalize the key abstract principle: the set of compact operators
  is closed in the operator norm topology.
-/
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Tactic

/-- The norm-limit of compact operators is compact.
    This is the abstract principle behind: Hilbert-Schmidt operators
    are compact (as limits of finite-rank, hence compact, operators). -/
theorem compact_of_operator_norm_limit
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [NormedAddCommGroup F] [NormedSpace ℝ F] [CompleteSpace F]
    {T : ℕ → E →L[ℝ] F} {A : E →L[ℝ] F}
    (hT : ∀ n, IsCompactOperator (T n : E → F))
    (hlim : Filter.Tendsto T Filter.atTop (nhds A)) :
    IsCompactOperator (A : E → F) :=
  isCompactOperator_of_tendsto hlim (Filter.Eventually.of_forall hT)
```

8. Показать, что оператор $A : l^2 \to \mathbb{R}^1$, $Ax = \sum_{k=1}^{\infty} \frac{\xi_k}{2^k}$ является компактным.

Формализация на Lean 4

```lean
/-
  Section 21, Task 08.
  A : □² → ℝ, Ax = Σ ξₖ/2ᵏ is a compact operator (finite-dimensional range).

  Any continuous linear map to a finite-dimensional space is compact,
  because bounded sets in finite-dimensional spaces have compact closure.
-/
import Mathlib.Analysis.Normed.Module.FiniteDimension
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Topology.MetricSpace.ProperSpace
import Mathlib.Tactic

/-- Any continuous linear map to a finite-dimensional space is compact.
    This applies to A: □²→ℝ, Ax = Σξₖ/2ᵏ (range = ℝ is 1-dimensional). -/
theorem clm_to_finiteDim_isCompact
    {E : Type*} [SeminormedAddCommGroup E] [NormedSpace ℝ E]
    {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ F]
    [FiniteDimensional ℝ F]
    (A : E →L[ℝ] F) : IsCompactOperator A := by
  haveI : ProperSpace F := FiniteDimensional.proper_real F
  exact isCompactOperator_id.comp_clm A
```

9. Показать, что образ компактного оператора является сепарабельным.

Формализация на Lean 4

```lean
/-
  Section 21, Task 9.
  Show that the image of a compact operator is separable.

  Proof: range A = ⋃ n, A '' closedBall 0 n. Each A '' closedBall 0 n
  has compact closure (since A is compact), hence is separable.
  Countable union of separable sets is separable.
-/
import Mathlib.Topology.MetricSpace.Pseudo.Basic
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Tactic

variable {X Y : Type*} [NormedAddCommGroup X] [NormedSpace ℝ X]
  [NormedAddCommGroup Y] [NormedSpace ℝ Y]

/-- The range of a compact operator is separable. -/
theorem compact_operator_range_separable
    (A : X →L[ℝ] Y) (hA : IsCompactOperator (A : X → Y)) :
    TopologicalSpace.IsSeparable (Set.range A) := by
  -- range A = ⋃ n : ℕ, A '' closedBall 0 n
  have hrange : Set.range A = ⋃ n : ℕ, A '' Metric.closedBall 0 (n : ℝ) := by
    ext y; simp only [Set.mem_range, Set.mem_iUnion, Set.mem_image]
    constructor
    · rintro ⟨x, rfl⟩
      exact ⟨⌈‖x‖⌉₊, x, Metric.mem_closedBall.mpr (by simp [dist_zero_right]; exact Nat.le_ceil _), rfl⟩
    · rintro ⟨n, x, _, rfl⟩; exact ⟨x, rfl⟩
  rw [hrange]
  rw [TopologicalSpace.isSeparable_iUnion]
  intro n
  -- A '' closedBall 0 n is bounded, its closure is compact
  have hbdd : Bornology.IsBounded (Metric.closedBall (0 : X) n) :=
    Metric.isBounded_closedBall
  exact ((hA.isCompact_closure_image_of_bounded hbdd).isSeparable).mono
    subset_closure
```

10. Являются ли компактными следующие операторы в пространствах $C[0,1]$ и $L^2(0,1)$?

a) $(Ax)(t) = \int_0^1 x(s)\,ds$;

b) $(Ax)(t) = \int_0^1 x(s)(t-s)^{-2/5}\,ds$;

c) $(Ax)(t) = \int_0^1 (ts + t^2 s^2) x(s)\,ds$.

11. Доказать, что оператор дифференцирования $\frac{d}{dt}$ является компактным из $C^{(2)}[0,1]$ в $C[0,1]$.

Формализация на Lean 4

```
/-
  Section 21, Task 11.
  d/dt : C²[0,1] → C[0,1] is compact (via Arzelà-Ascoli).

  Abstract proof: d/dt = (d/dt : C¹→C) ∘ (inclusion : C²→C¹).
  The inclusion C²→C¹ is compact (by Arzelà-Ascoli: the image of the
  C² unit ball is equicontinuous in C¹). Composing a CLM with a compact
  operator gives a compact operator.

  We formalize the abstract principle: CLM ∘ compact = compact.
-/
import Mathlib.Analysis.Normed.Operator.Compact
import Mathlib.Tactic

/-- Composition of a CLM with a compact operator is compact.
    This is the principle behind: d/dt : C² → C is compact
    (factor as CLM ∘ compact inclusion). -/
theorem compact_of_clm_comp_compact
    {□ : Type*} [NontriviallyNormedField □]
    {E : Type*} [SeminormedAddCommGroup E] [NormedSpace □ E]
    {F : Type*} [SeminormedAddCommGroup F] [NormedSpace □ F]
    {G : Type*} [SeminormedAddCommGroup G] [NormedSpace □ G]
    {T : E →L[□] F} (hT : IsCompactOperator (T : E → F))
    (S : F →L[□] G) :
    IsCompactOperator (S.comp T : E → G) :=
  hT.clm_comp S
```

12. При каких $\alpha, \beta, \gamma > 0$ является компактным в пространстве $C[0,1]$ оператор $(Ax)(t) = \int_0^1 t^\alpha s^\beta x(s^\gamma)\,ds$?
13. При каких α, β, γ является компактным в пространстве $L^2(0,1)$ оператор $(Ax)(t) = \int_0^1 t^\alpha s^\beta x(s^\gamma)\,ds$?